M07 — Prácticas integrales: observabilidad y resiliencia con .NET 8

Autor del curso: JAIME GALLO

Cómo utilizar el manual

Continúa exactamente `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`: 29 proyectos, 18 proyectos de tests, 283 pruebas y 34 documentos Markdown en `docs`. Construye cada una de las cuatro etapas en una carpeta nueva copiando la etapa anterior. Sigue los 96 pasos y aplica los bloques en el orden indicado.

Requisitos: Visual Studio con desarrollo ASP.NET, SDK .NET 8 y SQL Server Express o LocalDB. Conserva `global.json`, los proyectos hexagonales, el dominio de Producción y RabbitMQ con outbox/inbox de M6. Los catálogos son ejercicios de análisis: sus indicadores no instalan middleware ni prueban conexiones o recuperación. La implementación funcional se construye progresivamente junto al catálogo. Los pasos 7.1.17, 7.2.16, 7.3.16 y 7.4.17 completan el registro de los componentes y comprueban la solución; las pruebas de comportamiento aportan evidencia separada.

SQL Server para las pruebas. Conserva `tests/Compartido/SqlServerPruebas.cs`. El helper lee `ACERIA_TEST_SQLSERVER`, conecta a `master` y crea una base con nombre único por prueba; la cuenta necesita permiso para crearla y eliminarla. Ejemplo PowerShell con LocalDB: `$env:ACERIA_TEST_SQLSERVER = "Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True"`. Para Express sustituye el servidor por `.\SQLEXPRESS`. Sin variable, el helper utiliza Docker y `Testcontainers.MsSql` 3.10.0 con SQL Server 2022 Express y la imagen fija `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Una conexión inválida hace fallar las pruebas; no se omiten ni se sustituyen por memoria. RabbitMQ debe estar disponible para las pruebas heredadas y los recorridos funcionales de mensajería y readiness.

Convención de código. Cada bloque identifica archivo y operación. Crear exige un archivo nuevo y reemplazar exige un archivo existente; el bloque muestra su contenido completo. Las rutas son relativas a la raíz de la etapa. Las anotaciones numeradas se reservan para decisiones significativas y remiten a las líneas físicas de los bloques; no se comentan de forma repetida llaves, importaciones habituales ni líneas vacías. Los cuatro retos se aplican en copias optativas independientes y añaden una prueba cada uno; no se acumulan en las etapas obligatorias. Guarda todos los archivos e informes en UTF-8.

Preparar Docker Desktop en Windows para consumir RabbitMQ

El alumno trabaja en **Windows con PowerShell, Visual Studio, .NET 8 y SQL Server Express LocalDB**. En este módulo Docker Desktop solo aloja **RabbitMQ**: Producción, Inventario y (en M08) YARP se ejecutan directamente con `dotnet run`. El despliegue conjunto de todas las aplicaciones en contenedores queda fuera de esta práctica.

1. Instala y abre Docker Desktop en Windows con **WSL 2** y contenedores **Linux**. Antes de arrancar el broker ejecuta `docker version`, `docker compose version` y `docker info --format '{{.OSType}}'`; la última comprobación debe mostrar `linux`. Si Docker no responde, inicia Docker Desktop y repite la comprobación.
2. Abre PowerShell en la **carpeta de la etapa concreta** que estés trabajando, donde debe existir `compose.yml` (se crea en M06 y se hereda en M07/M08). Comprueba `Test-Path .\compose.yml`; debe devolver `True`. Ejecuta `docker compose -f .\compose.yml config --quiet`; cualquier error indica que no puedes continuar.

3. Ejecuta `docker compose -f .\compose.yml up -d rabbitmq` y comprueba el estado con `docker compose -f .\compose.yml ps`. Espera al arranque real con `docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping`; si devuelve error, espera unos segundos y repite, sin dar la práctica por superada.
4. Comprueba los puertos locales `5672` (AMQP) y `15672` (administración): `Test-NetConnection localhost -Port 5672` y `Test-NetConnection localhost -Port 15672` deben devolver `TcpTestSucceeded=True`. Abre `http://localhost:15672/` con `curso` / `curso-local` **solo en este laboratorio**.
5. En **cada terminal PowerShell que ejecute un servicio**, configura `$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'` y, cuando el ejercicio indique mensajería, `$env:Comunicacion__WorkersActivos = 'true'`. Los procesos no heredan las variables de otras ventanas. Conserva LocalDB como motor SQL, salvo que el paso exija explícitamente otra instalación.
6. Si ya tienes otro broker en `5672`, no levantes dos Compose con los mismos puertos: utiliza el que está activo si coincide con la configuración del ejercicio o detén primero el anterior. Para detener sin borrar mensajes: `docker compose -f .\compose.yml stop rabbitmq`; para recuperar: `docker compose -f .\compose.yml start rabbitmq` y repite el diagnóstico. No uses `down -v` cuando debas conservar el volumen y las proyecciones.
7. Si aparecen errores, consulta `docker compose -f .\compose.yml logs --tail 80 rabbitmq`, revisa los puertos ocupados y la salud del broker. Un contenedor arrancado no demuestra que el flujo de outbox/inbox funcione: ejecuta después las peticiones, comprobaciones y pruebas E2E del punto.

Criterio para continuar: broker disponible, conexión AMQP verificada y servicios Windows listos para consumirlo. La validación definitiva de mensajería se realiza en los procedimientos funcionales del módulo, no solo mediante `docker compose ps`.

Resultados de comprobación

Punto	Tests nuevos de catálogo	Tests nuevos funcionales	Tests de la etapa	Proyectos	Proyectos de tests	Documentos
Base M06 6.3	—	—	283	29	18	34
7.1	12	14	309	31	19	36
7.2	13	9	331	32	20	38
7.3	13	9	353	33	21	40
7.4	18	32	403	34	22	42

Punto 7.1 — Logging

Observar las peticiones entre servicios con información útil

El registro de eventos es necesario para comprender qué ocurrió cuando una solicitud atraviesa Producción, Inventario y RabbitMQ. No basta con imprimir mensajes libres en la consola: un log estructurado debe aportar nivel, instante, identificadores de correlación y contexto suficiente para investigar errores sin exponer datos sensibles. La misma correlación permite relacionar los eventos de procesos distintos.

Partiremos del estado completo de M06, conservando sus comunicaciones HTTP y asíncronas. Las clases de catálogo del punto 7.1 ayudarán a discutir decisiones de logging, mientras Serilog y la configuración efectiva de las API ofrecerán el comportamiento que debemos comprobar. Una propiedad declarada como logging estructurado no demuestra por sí sola que se hayan emitido registros JSON; estos se verifican durante la ejecución.

Carpeta de trabajo: `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar el análisis y el comportamiento ejecutable. La etapa termina con 309 pruebas en 31 proyectos y 19 proyectos de tests.

Ejercicio guiado. Incorpora `ConfiguracionLogging`, `ValidadorLogging` y `CatalogoLogging`, dos documentos nuevos y 12 pruebas de catálogo. Los bloques funcionales se aplican progresivamente; el paso 17 completa su registro y comprueba la compilación del punto.

PASO 7.1.1 — Crear la estructura del módulo 7 y del punto 7.1

El punto 7.1 comienza con una copia completa de M06 6.3 y conserva las comunicaciones previas. Comparar el estado heredado con la etapa nueva permite localizar regresiones y evita alterar la referencia histórica.

1. Localiza la etapa completa `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging` y copia en ella el contenido de `6.3-cuando-usar-cada-enfoque`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `7.1-logging`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `6.3-cuando-usar-cada-enfoque` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `6.3-cuando-usar-cada-enfoque`: verifica la ruta antes de editar.

PASO 7.1.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `7.1-logging` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 29 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 29 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 7.1.3 — Verificar que la carpeta `docs` contiene los documentos del punto 6.3

El punto conserva 34 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 34 archivos Markdown.
2. Localiza los ADR-010 a ADR-026 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`.
4. Comprueba que aún no existen `docs/logging.md` ni `docs/ADR-027-logging.md`.

Verificación concreta. Hay 34 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 7.1.4 — Crear el documento de logging en `docs`

La ficha separa el análisis de los contratos y la evidencia ejecutable.

1. Crea `docs/logging.md` dentro de la copia de `7.1-logging`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/logging.md` existe y que los 34 documentos anteriores permanecen intactos.

Archivo: `docs/logging.md`. **Operación:** Crear.

```
# Logging — Punto 7.1

## Definición

Registro de eventos que ocurren durante la ejecución del sistema.

## Niveles de log

- Trace: información muy detallada.
- Debug: información de depuración.
- Information: operación normal.
- Warning: situación anómala.
- Error: error que impide una operación.
- Critical: error que afecta a todo el sistema.

## Logging estructurado

Registra una plantilla de mensaje y propiedades nombradas y tipadas. El mensaje sigue siendo texto; las
propiedades permiten buscar y agregar sin analizarlo con expresiones regulares.

## Serilog

Biblioteca de logging estructurado para .NET.

## Integración con ASP.NET Core

    builder.Host.UseSerilog((context, config) =>
    {
        config
            .MinimumLevel.Information()
            .Enrich.FromLogContext()
            .WriteTo.Console(new JsonFormatter());
    });

## Correlación
```

```
Identificador común que se propaga entre servicios con la cabecera `X-Correlation-Id`.

## Logging de errores

Registra el tipo de fallo y los identificadores aprobados. No vuelques mensajes libres, cuerpos, query
strings, cabeceras Authorization ni excepciones completas que puedan contener secretos. La respuesta pública
tampoco expone detalles internos.

## Buenas prácticas

1. Logging estructurado.
2. Plantillas con propiedades.
3. Sin información sensible.
4. Nivel mínimo por entorno.
5. Correlación con identificador.

## Configuración por entorno

El nivel mínimo se ajusta por entorno en `appsettings.json`.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Biblioteca | Serilog |
| Sinks | Consola JSON en el ejercicio; un archivo requeriría retención, permisos y rotación |
| Enrichers | LogContext |
| Correlación | X-Correlation-Id |
| Nivel | Configurado por entorno |
| Validación | Test de coherencia |
## Contrato del ejercicio

Los seis niveles Trace, Debug, Information, Warning, Error y Critical corresponden a
`Microsoft.Extensions.Logging`. Serilog los representa mediante Verbose, Debug, Information, Warning, Error y
Fatal; `None` desactiva el registro y no es un nivel de evento. El catálogo exige logging estructurado,
correlación, protección de datos y configuración por entorno como política del curso, sin imponer que todas
las aplicaciones deban aceptar un header del cliente.

El middleware valida `X-Correlation-ID`: entre 1 y 64 caracteres ASCII alfanuméricos, guion o subrayado.
Sustituye valores inválidos, conserva un identificador por petición y lo devuelve y propaga al cliente HTTP.
Un identificador de correlación no demuestra por sí solo un sistema de trazas distribuido.
```

Verificación concreta. `docs/logging.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/logging.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.1.5 — Añadir Serilog.AspNetCore a la biblioteca de observabilidad

La dependencia de Serilog debe incorporarse al proyecto que realmente emite los registros. Comprueba el paquete y el arranque efectivo: declarar una clase con el nombre Logging no activa el proveedor.

1. Abre la carpeta `src` de la etapa actual y localiza los proyectos API de Producción e Inventario.
2. Aplica el archivo completo `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj` mostrado en este paso y comprueba la referencia a `Serilog.AspNetCore 8.0.3`.
3. Comprueba `global.json` y ejecuta `dotnet --version` para confirmar el SDK .NET 8 compatible antes de restaurar paquetes.
4. Añade la biblioteca a `AceriaApp.sln` si aún no aparece, ejecuta `dotnet restore src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj` y comprueba que la restauración termina sin errores.

Verificación concreta. La biblioteca compartida está incluida en la solución y su archivo de proyecto restaura los paquetes fijados.

Diagnóstico si falla.

- Añadir un sink de archivo sin definir permisos y retención amplía el despliegue: el ejercicio emite consola JSON.
- Modificar el proyecto de una etapa anterior rompe su autonomía: comprueba el camino completo antes de editar.

Integración funcional — `src/AceriaApp.Api/AceriaApp.Api.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `src/AceriaApp.Api/AceriaApp.Api.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk.Web"><PropertyGroup><TargetFramework>net8.0</
TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</ImplicitUsings></
PropertyGroup><ItemGroup><PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
Version="8.0.21" /><PackageReference Include="FluentValidation.DependencyInjectionExtensions"
Version="11.11.0" /><PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" /></ItemGroup>
<ItemGroup>
    <ProjectReference Include="../AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Hexagonal.Infraestructura/
AceriaApp.Hexagonal.Infraestructura.csproj" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Produccion.Infraestructura/
AceriaApp.Produccion.Infraestructura.csproj" />
  </ItemGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Comunicacion.Infraestructura/
AceriaApp.Comunicacion.Infraestructura.csproj" /></ItemGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" /></
ItemGroup>
</Project>
```

Fija la dependencia usada por esta etapa del ejercicio. Conecta este proyecto con otro proyecto real de la solución.

Integración funcional — `src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Contratos/AceriaApp.Comunicacion.Contratos.csproj" /
>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
<ProjectReference Include="../AceriaApp.Comunicacion.Infraestructura/
AceriaApp.Comunicacion.Infraestructura.csproj" /></ItemGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" /></
ItemGroup>
</Project>
```

Integración funcional — `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
```

```
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Serilog.AspNetCore" Version="8.0.3" />
  </ItemGroup>
</Project>
```

PASO 7.1.6 — Crear la carpeta `Logging` en el proyecto API

`Logging` agrupa datos y reglas del ejercicio de análisis; los adaptadores de comunicación tienen sus propias responsabilidades.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `Logging` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/Logging` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `6.3-cuando-usar-cada-enfoque`.

Verificación concreta. Existe `src/AceriaApp.Api/Logging` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

PASO 7.1.7 — Crear la clase `ConfiguracionLogging`

ConfiguracionLogging describe los niveles, destinos y opciones que se desean aplicar. Diferencia las opciones del catálogo de la configuración con la que Serilog escribe registros durante una petición real.

1. Crea `src/AceriaApp.Api/Logging/ConfiguracionLogging.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Logging/ConfiguracionLogging.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Logging;

public class ConfiguracionLogging
{
    public string NivelMinimo { get; init; } = string.Empty;
    public bool UsaLoggingEstructurado { get; init; }
    public bool TieneCorrelacion { get; init; }
    public bool EvitaDatosSensibles { get; init; }
    public bool ConfiguradoPorEntorno { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `ConfiguracionLogging` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Logging` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.1.8 — Crear la clase `ValidadorLogging`

El validador puede rechazar una descripción inconsistente de logging, pero no sabe si se emitió un registro. Para demostrar esa segunda propiedad hay que inspeccionar la salida de las API ejecutadas.

1. Crea `src/AceriaApp.Api/Logging/ValidadorLogging.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Logging/ValidadorLogging.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Logging;

public class ValidadorLogging
{
    private static readonly string[] NivelesPermitidos = { "Trace", "Debug", "Information", "Warning",
"Error", "Critical" };

    public ResultadoLogging Validar(IEnumerable<ConfiguracionLogging> configuraciones)
    {
        var errores = new List<string>();
        var lista = configuraciones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay configuraciones de logging declaradas.");
        }

        foreach (var configuracion in lista)
        {
            if (!NivelesPermitidos.Contains(configuracion.NivelMinimo))
            {
                errores.Add($"Nivel mínimo no permitido: '{configuracion.NivelMinimo}'.");
            }

            if (!configuracion.UsaLoggingEstructurado)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no usa logging
estructurado.");
            }

            if (!configuracion.TieneCorrelacion)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no tiene
correlación.");
            }

            if (!configuracion.EvitaDatosSensibles)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no evita datos
sensibles.");
            }

            if (!configuracion.ConfiguradoPorEntorno)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no está configurada
por entorno.");
            }
        }

        return new ResultadoLogging(errores.Count == 0, errores);
    }
}

public record ResultadoLogging(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Devuelve validez y diagnósticos; no instala middleware ni ejecuta llamadas remotas. Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorLogging` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Logging` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

Integración funcional — `src/AceriaApp.Observabilidad/CorrelacionMiddleware.cs`.

La cabecera de entrada es un dato no confiable. Se admite únicamente un identificador corto formado por letras ASCII, números, guion o guion bajo. El valor aceptado se guarda en Items y en LogContext durante esta petición; los ámbitos se eliminan al terminar, incluso si hay una excepción.

Archivo: `src/AceriaApp.Observabilidad/CorrelacionMiddleware.cs`. **Operación:** Crear.

```
using System.Diagnostics;
using Microsoft.AspNetCore.Http;
using Serilog.Context;
namespace AceriaApp.Observabilidad;
public static class ContextoCorrelacion
{
    public const string Cabecera = "X-Correlation-ID";
    private static readonly object Clave = new();
    public static string Obtener(HttpContext context) =>
        context.Items.TryGetValue(Clave, out var value) ? (string)value! : "";
    internal static void Guardar(HttpContext context, string id) => context.Items[Clave] = id;
    internal static bool Valido(string value) => value.Length is > 0 and <= 64 &&
        value.All(c => char.IsAsciiLetterOrDigit(c) || c is '-' or '_');
}
public sealed class CorrelacionMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext context)
    {
        var entrada = context.Request.Headers[ContextoCorrelacion.Cabecera].ToString();
        var id = ContextoCorrelacion.Valido(entrada) ? entrada : Guid.NewGuid().ToString("N");
        ContextoCorrelacion.Guardar(context, id);
        context.Response.Headers[ContextoCorrelacion.Cabecera] = id;
        context.Response.OnStarting(() =>
        {
            context.Response.Headers[ContextoCorrelacion.Cabecera] = id;
            return Task.CompletedTask;
        });
        using (LogContext.PushProperty("CorrelationId", id))
        using (LogContext.PushProperty("TraceId", Activity.Current?.TraceId.ToString() ??
context.TraceIdentifier))
            await next(context);
    }
}
```

Define un escenario asíncrono cuyo resultado verifica la conducta descrita por el nombre. Devuelve al cliente el identificador aceptado o generado. Repone la cabecera justo antes de enviarla, también cuando el manejador limpió la respuesta tras un fallo.

Añade propiedades solo durante el ámbito de esta petición. Relaciona el evento con la traza técnica de la petición.

Integración funcional — `src/AceriaApp.Observabilidad/PropagarCorrelacionHandler.cs`.

El HttpClient obtiene el identificador desde la petición actual. No conserva ese dato en un campo del handler, que puede reutilizarse entre peticiones. Si no existe contexto HTTP, no inventa ni reutiliza una correlación anterior.

Archivo: `src/AceriaApp.Observabilidad/PropagarCorrelacionHandler.cs`. **Operación:** Crear.

```
using Microsoft.AspNetCore.Http;
namespace AceriaApp.Observabilidad;
public sealed class PropagarCorrelacionHandler(IHttpContextAccessor accessor) : DelegatingHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken ct)
    {
        request.Headers.Remove(ContextoCorrelacion.Cabecera);
        var id = accessor.HttpContext is { } context ? ContextoCorrelacion.Obtener(context) : "";
        if (id.Length > 0) request.Headers.Add(ContextoCorrelacion.Cabecera, id);
        return base.SendAsync(request, ct);
    }
}
```

Elimina un valor anterior antes de propagar la correlación actual.

Integración funcional — `src/AceriaApp.Observabilidad/RegistroLogging.cs`.

La configuración crea eventos Serilog JSON y añade el nombre del servicio. El nivel mínimo se lee de IConfiguration: appsettings.Production.json lo eleva a Warning. Se silencian categorías que podrían incluir detalles HTTP o excepciones libres; los eventos propios conservan los datos estructurados necesarios. Esta restricción no audita todos los logs de los workers heredados de M6.

Archivo: `src/AceriaApp.Observabilidad/RegistroLogging.cs`. **Operación:** Crear.

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Configuration;
using Serilog;
using Serilog.Core;
using Serilog.Events;
using Serilog.Formatting.Json;
namespace AceriaApp.Observabilidad;
public static class RegistroLogging
{
    public static IServiceCollection AddLoggingAceria(this IServiceCollection services, string servicio,
IConfiguration? config = null)
    {
        var nivel = config?.GetValue("Observabilidad:NivelMinimo", LogEventLevel.Information) ??
LogEventLevel.Information;
        services.AddLogging();
        services.AddHttpContextAccessor();
        services.AddTransient<PropagarCorrelacionHandler>();
        services.AddSerilog((provider, log) => log.MinimumLevel.Is(nivel)
            .MinimumLevel.Override("Microsoft.AspNetCore", LogEventLevel.Warning)
            .MinimumLevel.Override("Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware",
LogEventLevel.Fatal)
            .MinimumLevel.Override("Microsoft.EntityFrameworkCore", LogEventLevel.Warning)
            .MinimumLevel.Override("System.Net.Http.HttpClient", LogEventLevel.Warning)
            .MinimumLevel.Override("Polly", LogEventLevel.Fatal)
            .Enrich.FromLogContext().Enrich.With(new CamposHttpSeguros()).Enrich.WithProperty("Servicio",
servicio)
            .ReadFrom.Services(provider).WriteTo.Console(new JsonFormatter()));
        return services;
    }
    private sealed class CamposHttpSeguros : ILogEventEnricher
    {
        public void Enrich(LogEvent logEvent, ILogEventPropertyFactory factory) =>
            logEvent.RemovePropertyIfPresent("RequestPath");
    }
    public static IApplicationBuilder UseLoggingAceria(this IApplicationBuilder app) =>
        app.UseMiddleware<CorrelacionMiddleware>().UseMiddleware<ResumenPeticionMiddleware>();
}
```

Registra el logger JSON y los servicios que necesita la correlación. Lee el umbral de logging configurado para el entorno. Serializa eventos estructurados como JSON en consola.

Define una comprobación síncrona del contrato indicado por el nombre. Elimina la ruta concreta que los scopes automáticos de ASP.NET incorporan al evento. Abre la correlación y el resumen antes de ejecutar el resto del pipeline.

Integración funcional — `src/AceriaApp.Observabilidad/ResumenPeticionMiddleware.cs`.

Se registra un resumen por petición usando la plantilla de la ruta, nunca su valor material ni la consulta. El cronómetro engloba el resto del pipeline. El estado de cancelación es una observación del servidor; no promete que el cliente reciba una respuesta 499 si ya cortó la conexión.

Archivo: `src/AceriaApp.Observabilidad/ResumenPeticionMiddleware.cs`. **Operación:** Crear.

```
using System.Diagnostics;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Routing;
using Microsoft.Extensions.Logging;
namespace AceriaApp.Observabilidad;
public sealed class ResumenPeticionMiddleware(RequestDelegate next, ILogger<ResumenPeticionMiddleware>
logger)
{
    public async Task InvokeAsync(HttpContext context)
    {
        var inicio = Stopwatch.GetTimestamp();
        var ruta = (context.GetEndpoint() as RouteEndpoint)?.RoutePattern.RawText ?? "[sin-ruta]";
        var fallo = false;
        try { await next(context); }
        catch { fallo = true; throw; }
        finally
        {
            if (!context.Request.Path.StartsWithSegments("/health"))
            {
                var estado = context.RequestAborted.IsCancellationRequested ? 499 : fallo ? 500 :
context.Response.StatusCode;
                var nivel = estado >= 500 ? LogLevel.Error : estado >= 400 && estado != 499 ?
LogLevel.Warning : LogLevel.Information;
                logger.Log(nivel, "Peticion {Metodo} {Ruta} respondio {Estado} en {Milisegundos} ms",
                    context.Request.Method, ruta, estado,
Stopwatch.GetElapsedTime(inicio).TotalMilliseconds);
            }
        }
    }
}
```

Mide duración mediante un reloj monotónico. Obtiene la plantilla del endpoint en lugar del valor concreto de la URL. Distingue una cancelación del cliente de un fallo de la dependencia.

PASO 7.1.9 — Añadir los `using` necesarios en `ValidadorLogging`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Logging/ValidadorLogging.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Logging/ValidadorLogging.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
```

```
namespace AceriaApp.Api.Logging;

public class ValidadorLogging
{
    private static readonly string[] NivelesPermitidos = { "Trace", "Debug", "Information", "Warning",
"Error", "Critical" };

    public ResultadoLogging Validar(IEnumerable<ConfiguracionLogging> configuraciones)
    {
        var errores = new List<string>();
        var lista = configuraciones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay configuraciones de logging declaradas.");
        }

        foreach (var configuracion in lista)
        {
            if (!NivelesPermitidos.Contains(configuracion.NivelMinimo))
            {
                errores.Add($"Nivel mínimo no permitido: '{configuracion.NivelMinimo}'.");
            }

            if (!configuracion.UsaLoggingEstructurado)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no usa logging
estructurado.");
            }

            if (!configuracion.TieneCorrelacion)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no tiene
correlación.");
            }

            if (!configuracion.EvitaDatosSensibles)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no evita datos
sensibles.");
            }

            if (!configuracion.ConfiguradoPorEntorno)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no está configurada
por entorno.");
            }
        }

        return new ResultadoLogging(errores.Count == 0, errores);
    }
}

public record ResultadoLogging(bool EsValida, IReadOnlyList<string> Errores);
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorLogging.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.1.10 — Crear la clase `CatalogoLogging`

CatalogoLogging reúne casos de análisis sin configurar el host. Su valor didáctico consiste en permitir razonar sobre niveles y formatos antes de comprobar los logs reales de Producción e Inventario.

1. Crea `src/AceriaApp.Api/Logging/CatalogoLogging.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Logging/CatalogoLogging.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Logging;

public class CatalogoLogging
{
    public static List<ConfiguracionLogging> ConfiguracionesPorDefecto()
    {
        return new List<ConfiguracionLogging>
        {
            new()
            {
                NivelMinimo = "Debug",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            },
            new()
            {
                NivelMinimo = "Information",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            },
            new()
            {
                NivelMinimo = "Warning",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Completa la declaración o la operación mostrada en el bloque. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoLogging` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Logging` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.1.11 — Añadir el `using` necesario en `CatalogoLogging`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Logging/CatalogoLogging.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Logging/CatalogoLogging.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;

namespace AceriaApp.Api.Logging;

public class CatalogoLogging
{
    public static List<ConfiguracionLogging> ConfiguracionesPorDefecto()
    {
        return new List<ConfiguracionLogging>
        {
            new()
            {
                NivelMinimo = "Debug",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            },
            new()
            {
                NivelMinimo = "Information",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            },
            new()
            {
                NivelMinimo = "Warning",
                UsaLoggingEstructurado = true,
                TieneCorrelacion = true,
                EvitaDatosSensibles = true,
                ConfiguradoPorEntorno = true
            }
        };
    }
}
```

Verificación concreta. `CatalogoLogging.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.1.12 — Crear el proyecto de tests del punto 7.1

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.Logging.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 15 y 16 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj` y `dotnet restore tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 7.1.13 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj` y `dotnet build tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
```

```
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

Integración funcional — `src/AceriaApp.Api/appsettings.Production.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Api/appsettings.Production.json`. **Operación:** Crear.

```
{
  "Observabilidad": {
    "NivelMinimo": "Warning"
  }
}
```

Fija el umbral propio de logging y admite sobreescritura por variables de entorno.

Integración funcional — `src/AceriaApp.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM071;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM071;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM071;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m071",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

Agrupa las conexiones de laboratorio específicas de la etapa. Activa o desactiva los procesos de mensajería y su dependencia en readiness. Mantiene el endpoint de fallo desactivado por defecto.

Integración funcional — `src/AceriaApp.Inventario.Api/appsettings.Production.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Inventario.Api/appsettings.Production.json`. **Operación:** Crear.

```
{
  "Observabilidad": {
    "NivelMinimo": "Warning"
  }
}
```

Integración funcional — `src/AceriaApp.Inventario.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Inventario.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM071;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m071",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

PASO 7.1.14 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj` y `dotnet list tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

PASO 7.1.15 — Crear el archivo `ValidadorLoggingTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.Logging.Tests/ValidadorLoggingTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Logging.Tests/ValidadorLoggingTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Logging;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Logging.Tests;

public class ValidadorLoggingTests
{
    private readonly ValidadorLogging _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoLogging.ConfiguracionesPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinConfiguraciones_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ConfiguracionLogging>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay configuraciones"));
    }

    [Fact]
    public void Validar_NivelNoPermitido_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "NivelInventado", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Nivel mínimo no permitido"));
    }

    [Fact]
    public void Validar_SinLoggingEstructurado_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = false, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
```

```
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa logging estructurado"));
    }

    [Fact]
    public void Validar_SinCorrelacion_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = false,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene correlación"));
    }

    [Fact]
    public void Validar_DatosSensibles_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = false, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no evita datos sensibles"));
    }

    [Fact]
    public void Validar_SinConfiguracionPorEntorno_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = false }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está configurada por entorno"));
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Construye una entrada del catálogo o del caso de prueba con valores explícitos.

Verificación concreta. `ValidadorLoggingTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.

- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Integración funcional — `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Integración funcional — `tests/AceriaApp.Logging.Tests/LoggingFuncionalTests.cs`.

Estas pruebas ejecutan middleware, HttpClient y Serilog reales sobre contextos controlados: incluyen cabeceras inválidas, concurrencia, aislamiento, propagación y campos JSON. El recorrido entre dos procesos, al final del paso de registro, añade la evidencia HTTP que una prueba de componente no proporciona.

Archivo: `tests/AceriaApp.Logging.Tests/LoggingFuncionalTests.cs`. **Operación:** Crear.

```
using System.Collections.Concurrent;
using System.Text.Json;
using AceriaApp.Observabilidad;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Routing;
using Microsoft.AspNetCore.Routing.Patterns;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;
using Serilog;
using Serilog.Core;
using Serilog.Events;
using Serilog.Extensions.Logging;
using Serilog.Formatting.Json;
using Xunit;
namespace AceriaApp.Logging.Tests;
public class LoggingFuncionalTests
{
    private sealed class Captura : ILogEventSink
    {
        public ConcurrentQueue<LogEvent> Eventos { get; } = new();
        public void Emit(LogEvent evento) => Eventos.Enqueue(evento);
    }
    private static DefaultHttpContext Contexto(string? id = null)
    {
        var ctx = new DefaultHttpContext();
        ctx.Request.Method = "GET";
        ctx.Request.Path = "/material/secreto-en-ruta";
        ctx.Request.QueryString = new QueryString("?token=secreto-query");
        ctx.Request.Headers.Authorization = "Bearer secreto-header";
        ctx.Request.Body = new MemoryStream("secreto-body"u8.ToArray());
        if (id is not null) ctx.Request.Headers["X-Correlation-ID"] = id;
        ctx.SetEndpoint(new RouteEndpoint(_ => Task.CompletedTask,
            RoutePatternFactory.Parse("/material/{nombre}"), 0, EndpointMetadataCollection.Empty,
"material"));
        return ctx;
    }
    [Fact]
    public async Task Correlacion_valida_se_conserva_en_respuesta_y_contexto()
```

```
    {
        var ctx = Contexto("orden_123-A");
        string? leido = null;
        await new CorrelacionMiddleware(c => { leido = ContextoCorrelacion.Obtener(c); return
Task.CompletedTask; }).InvokeAsync(ctx);
        Assert.Equal("orden_123-A", leido);
        Assert.Equal("orden_123-A", ctx.Response.Headers["X-Correlation-ID"]);
    }
    [Theory]
    [InlineData("")]
    [InlineData("valor\r\ninyectado")]
    [InlineData("con espacio")]
    [InlineData("áéí")]
    [InlineData("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa")]
    public async Task Correlacion_invalida_se_sustituye_sin_propagar_la_entrada(string entrada)
    {
        var ctx = Contexto(entrada);
        await new CorrelacionMiddleware(_ => Task.CompletedTask).InvokeAsync(ctx);
        var id = ctx.Response.Headers["X-Correlation-ID"].ToString();
        Assert.Matches("^[a-f0-9]{32}$", id);
        Assert.NotEqual(entrada, id);
        Assert.Equal(id, ContextoCorrelacion.Obtener(ctx));
    }
    [Fact]
    public async Task Correlacion_ausente_se_genera()
    {
        var ctx = Contexto();
        await new CorrelacionMiddleware(_ => Task.CompletedTask).InvokeAsync(ctx);
        Assert.Matches("^[a-f0-9]{32}$", ContextoCorrelacion.Obtener(ctx));
    }
    [Fact]
    public async Task Resumen_JSON_contiene_propiedades_tipadas_y_no_datos_de_la_peticion()
    {
        var sink = new Captura();
        using var logger = new
LoggerConfiguration().Enrich.FromLogContext().WriteTo.Sink(sink).CreateLogger();
        using var factory = new SerilogLoggerFactory(logger, dispose: false);
        var resumen = new ResumenPeticionMiddleware(c => { c.Response.StatusCode = 503; return
Task.CompletedTask; },
            factory.CreateLogger<ResumenPeticionMiddleware>());
        await new CorrelacionMiddleware(resumen.InvokeAsync).InvokeAsync(Contexto("peticion-1"));
        var evento = Assert.Single(sink.Eventos);
        using var writer = new StringWriter(); new JsonFormatter().Format(evento, writer);
        var json = writer.ToString(); using var doc = JsonDocument.Parse(json);
        var props = doc.RootElement.GetProperty("Properties");
        Assert.Equal("peticion-1", props.GetProperty("CorrelationId").GetString());
        Assert.Equal("/material/{nombre}", props.GetProperty("Ruta").GetString());
        Assert.Equal(503, props.GetProperty("Estado").GetInt32());
        Assert.True(props.GetProperty("Milisegundos").GetDouble() >= 0);
        Assert.False(string.IsNullOrWhiteSpace(props.GetProperty("TraceId").GetString()));
        Assert.Equal(LogEventLevel.Error, evento.Level);
        Assert.DoesNotContain("secreto", json);
    }
    [Fact]
    public async Task Ruta_no_encontrada_no_registra_el_texto_libre_de_la_URL()
    {
        var sink = new Captura(); using var logger = new
LoggerConfiguration().Enrich.FromLogContext().WriteTo.Sink(sink).CreateLogger();
        using var factory = new SerilogLoggerFactory(logger, false);
        var ctx = Contexto("ausente"); ctx.SetEndpoint(null);
        var resumen = new ResumenPeticionMiddleware(c => { c.Response.StatusCode = 404; return
Task.CompletedTask; }, factory.CreateLogger<ResumenPeticionMiddleware>());
        await new CorrelacionMiddleware(resumen.InvokeAsync).InvokeAsync(ctx);
        var e = Assert.Single(sink.Eventos);
        Assert.Equal("[sin-ruta]", ((ScalarValue)e.Properties["Ruta"]).Value);
        Assert.DoesNotContain("secreto", e.RenderMessage());
    }
    [Fact]
    public async Task Peticiones_concurrentes_no_mezclan_su_correlacion()
    {
        var sink = new Captura(); using var logger = new
LoggerConfiguration().Enrich.FromLogContext().WriteTo.Sink(sink).CreateLogger();
        using var factory = new SerilogLoggerFactory(logger, false);
        var resumen = new ResumenPeticionMiddleware(async _ => await Task.Yield(),
factory.CreateLogger<ResumenPeticionMiddleware>());
        await Task.WhenAll(Enumerable.Range(0, 12).Select(i => new
```

```
CorrelacionMiddleware(resumen.InvokeAsync).InvokeAsync(Contexto("paralelo-" + i))));
        Assert.Equal(12, sink.Eventos.Count);
        var ids = sink.Eventos.Select(e => (string)
((ScalarValue)e.Properties["CorrelationId"]).Value!).Order().ToArray();
        Assert.Equal(Enumerable.Range(0, 12).Select(i => "paralelo-" + i).Order(), ids);
    }
    private sealed class Transporte : HttpMessageHandler
    {
        public string? Recibido;
        protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken
ct)
        {
            Recibido = request.Headers.TryGetValues("X-Correlation-ID", out var values) ? values.Single() :
null;
            return Task.FromResult(new HttpResponseMessage(System.Net.HttpStatusCode.OK));
        }
    }
    [Fact]
    public async Task Cliente_saliente_propaga_la_correlacion_validada()
    {
        var ctx = Contexto("cadena-servicios"); var accessor = new HttpContextAccessor { HttpContext = ctx };
var terminal = new Transporte();
        using var http = new HttpClient(new PropagarCorrelacionHandler(accessor) { InnerHandler =
terminal });
        await new CorrelacionMiddleware(async _ => { using var response = await http.GetAsync("http://
inventario/material"); }).InvokeAsync(ctx);
        Assert.Equal("cadena-servicios", terminal.Recibido);
    }
    [Fact]
    public async Task Cliente_sin_contexto_HTTP_no_reutiliza_correlacion_anterior()
    {
        var terminal = new Transporte(); var accessor = new HttpContextAccessor { HttpContext = null };
        using var http = new HttpClient(new PropagarCorrelacionHandler(accessor) { InnerHandler =
terminal });
        using var response = await http.GetAsync("http://inventario/material");
        Assert.Null(terminal.Recibido);
    }
    [Fact]
    public void Registro_del_servicio_emite_propiedades_estructuradas_por_Serilog()
    {
        var sink = new Captura(); var services = new ServiceCollection();
services.AddSingleton<ILogEventSink>(sink);
        services.AddLoggingAceria("Inventario");
        using var provider = services.BuildServiceProvider();
        provider.GetRequiredService<ILogger<LoggingFuncionalTests>>().LogInformation("Lote {Cantidad}", 23);
        var e = Assert.Single(sink.Eventos);
        Assert.Equal("Inventario", ((ScalarValue)e.Properties["Servicio"]).Value);
        Assert.Equal(23, ((ScalarValue)e.Properties["Cantidad"]).Value);
    }

    [Fact]
    public void Nivel_minimo_configurado_filtra_informacion_y_conserva_avisos()
    {
        var sink = new Captura(); var services = new ServiceCollection();
services.AddSingleton<ILogEventSink>(sink);
        var config = new ConfigurationBuilder().AddInMemoryCollection(new Dictionary<string, string?>
            { ["Observabilidad:NivelMinimo"] = "Warning" }).Build();
        services.AddLoggingAceria("Inventario", config);
        using var provider = services.BuildServiceProvider(); var log =
provider.GetRequiredService<ILogger<LoggingFuncionalTests>>();
        log.LogInformation("Lote {Cantidad}", 23); log.LogWarning("Intento {Intento}", 5);
        var e = Assert.Single(sink.Eventos); Assert.Equal(LogEventLevel.Warning, e.Level);
        Assert.Equal(5, ((ScalarValue)e.Properties["Intento"]).Value);
    }
}
```

PASO 7.1.16 — Crear el archivo `CatalogoLoggingTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.Logging.Tests/CatalogoLoggingTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.

3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Logging.Tests/CatalogoLoggingTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Logging;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Logging.Tests;

public class CatalogoLoggingTests
{
    [Fact]
    public void ConfiguracionesPorDefecto_ContieneTresConfiguraciones()
    {
        var configuraciones = CatalogoLogging.ConfiguracionesPorDefecto();

        configuraciones.Should().HaveCount(3);
    }

    [Fact]
    public void ConfiguracionesPorDefecto_TodasUsanLoggingEstructurado()
    {
        var configuraciones = CatalogoLogging.ConfiguracionesPorDefecto();

        configuraciones.Should().OnlyContain(c => c.UsaLoggingEstructurado);
    }

    [Fact]
    public void ConfiguracionesPorDefecto_TodasTienenCorrelacion()
    {
        var configuraciones = CatalogoLogging.ConfiguracionesPorDefecto();

        configuraciones.Should().OnlyContain(c => c.TieneCorrelacion);
    }

    [Fact]
    public void ConfiguracionesPorDefecto_TodasEvitanDatosSensibles()
    {
        var configuraciones = CatalogoLogging.ConfiguracionesPorDefecto();

        configuraciones.Should().OnlyContain(c => c.EvitaDatosSensibles);
    }

    [Fact]
    public void ConfiguracionesPorDefecto_TodasConfiguradasPorEntorno()
    {
        var configuraciones = CatalogoLogging.ConfiguracionesPorDefecto();

        configuraciones.Should().OnlyContain(c => c.ConfiguradoPorEntorno);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoLoggingTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 7.1.17 — Compilar la solución completa

La compilación debe incluir los proyectos de observabilidad y sus referencias. A continuación comprobaremos que ambas API generan logs estructurados con el identificador de correlación correcto.

1. Aplica los bloques de registro final de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 31 proyectos y termina sin errores.

Registro de biblioteca compartida. Ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`; los proyectos de tests se incorporan en sus pasos de creación. Después de reemplazar todos los archivos de este paso, compila la solución.

Verificación concreta. La solución de la etapa compila con 31 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

Integración funcional — `src/AceriaApp.Api/Program.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Api.Comunicacion;
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddLoggingAceria("Produccion", builder.Configuration);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
builder.Services.AddComunicacion(builder.Configuration);
builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>("inventario-instrumentado", http =>
{
    http.BaseAddress = new Uri(builder.Configuration["Comunicacion:InventarioUrl"] ?? "http://
localhost:5106/");
```

```
    http.Timeout = TimeSpan.FromSeconds(3);
}).AddHttpMessageHandler<PropagarCorrelacionHandler>();
var app=builder.Build();
app.UseLoggingAceria();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;
```

```
// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
app.MapProduccion();
app.MapComunicacion();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Propaga el identificador validado hacia Inventario.

Integración funcional — `src/AceriaApp.Inventario.Api/InventarioProgram.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddLoggingAceria("Inventario", builder.Configuration);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        var app = builder.Build();
        app.UseLoggingAceria();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapGet("/api/v1/materiales/{material}", async (string material, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapGet("/api/v1/planificaciones/{ordenId:guid}", async (Guid ordenId, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
valor.CantidadTotal });
        });
        await app.RunAsync();
    }
}
```

Recorrido funcional 7.1 — correlación y logs JSON de las dos API

Abre tres terminales **PowerShell 7** en la carpeta de esta etapa, `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging`. Todos los comandos siguientes usan rutas relativas a esa carpeta: A ejecutará Producción en `5105`, B ejecutará Inventario en `5106` y C enviará peticiones. Necesitas el SDK .NET 8 y SQL Server Express LocalDB. Cierra con `Ctrl+C` cualquier API de un recorrido anterior que ocupe esos puertos.

En C, prepara la compilación y arranca la instancia LocalDB existente:

```
dotnet --version
sqllocaldb start MSSQLLocalDB
if ($LASTEXITCODE -ne 0) { throw 'No se pudo iniciar MSSQLLocalDB.' }
dotnet restore .\AceriaApp.sln
if ($LASTEXITCODE -ne 0) { throw 'Falló la restauración.' }
dotnet build .\AceriaApp.sln -c Release --no-restore
if ($LASTEXITCODE -ne 0) { throw 'Falló la compilación.' }
```

Los `appsettings.json` de esta etapa usan `AceriaAppM071`, `AceriaHexagonalM071`, `AceriaProduccionM071` y `AceriaInventarioM071`. El arranque en `Development` crea las bases si no existen; Inventario incorpora `Acero=300` e `Inoxidable=120` **solo si su tabla de existencias está vacía**. Una base utilizada antes puede contener otros valores: observa el stock real y no lo sustituyas por un número esperado.

Alternativa SQL Server Express: si utilizas una instancia instalada `localhost\SQLEXPRESS` con autenticación de Windows, inicia ese servicio y ejecuta este bloque en A y B **antes** del comando de arranque. Lee los nombres de bases de la etapa actual y cambia únicamente el servidor; también sirve en las etapas siguientes. En ese caso, omite el comando `sqllocaldb` y usa esa misma instancia para las pruebas SQL.

```
$servidorSql = 'localhost\SQLEXPRESS'
@('.\src\AceriaApp.Api\appsettings.json', '.\src\AceriaApp.Inventario.Api\appsettings.json') |
    ForEach-Object {
        $configuracion = Get-Content $_ -Raw | ConvertFrom-Json
        foreach ($conexion in $configuracion.ConnectionStrings.PSObject.Properties) {
            $cadena = $conexion.Value.Replace('Server=(localdb)\MSSQLLocalDB;', "Server=$servidorSql;")
            Set-Item -Path ('Env:ConnectionStrings__' + $conexion.Name) -Value $cadena
        }
    }
```

En B, inicia Inventario y deja esta terminal abierta:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
$env:Observabilidad__DiagnosticoActivo = 'false'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106 2>&1 |
    Tee-Object -FilePath .\inventario-m071.jsonl
```

En A, inicia Producción y deja esta terminal abierta:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Comunicacion__InventarioUrl = 'http://localhost:5106/'
$env:Observabilidad__NivelMinimo = 'Information'
$env:Observabilidad__DiagnosticoActivo = 'false'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105 2>&1 |
    Tee-Object -FilePath .\produccion-m071.jsonl
```

`--no-launch-profile` evita que el perfil heredado cambie el puerto a `5082`. En este recorrido los workers están desactivados: la consulta de Inventario usa HTTP y SQL reales y no necesita RabbitMQ.

En C, comprueba primero Inventario y después atraviesa Producción con una correlación identificable:

```
Invoke-RestMethod 'http://localhost:5106/health'
$stockReal = Invoke-RestMethod 'http://localhost:5106/api/v1/materiales/Acero'
$correlacion = 'm071-' + [guid]::NewGuid().ToString('N')
$respuesta = Invoke-WebRequest 'http://localhost:5105/api/v1/comunicacion/materiales/Acero' `
    -Headers @{ 'X-Correlation-ID' = $correlacion } -SkipHttpErrorCheck
$respuesta.StatusCode
$respuesta.Headers['X-Correlation-ID']
$respuesta.Content
if ($respuesta.StatusCode -ne 200) { throw 'La consulta a través de Producción no terminó en 200.' }
if ($respuesta.Headers['X-Correlation-ID'] -ne $correlacion) { throw 'Se perdió la correlación de
respuesta.' }
$material = $respuesta.Content | ConvertFrom-Json
if ($material.material -ne $stockReal.material -or $material.stock -ne $stockReal.stock) {
    throw 'La respuesta no coincide con el Inventario real.'
}
```

Consulta los dos archivos mientras siguen abiertos los procesos. Se toleran líneas de arranque que no sean JSON; el filtro solo recoge la petición recién enviada:

```
$eventos = @(
    Get-Content .\produccion-m071.jsonl, .\inventario-m071.jsonl |
        ForEach-Object {
            try {
                $evento = $_ | ConvertFrom-Json -ErrorAction Stop
                if ($evento.Properties.CorrelationId -eq $correlacion) { $evento }
            } catch { }
        }
)
$eventos | Select-Object Timestamp, Level,
    @{ n = 'Servicio'; e = { $_.Properties.Servicio } },
    @{ n = 'CorrelationId'; e = { $_.Properties.CorrelationId } },
    @{ n = 'TraceId'; e = { $_.Properties.TraceId } },
    @{ n = 'Metodo'; e = { $_.Properties.Metodo } },
    @{ n = 'Ruta'; e = { $_.Properties.Ruta } },
    @{ n = 'Estado'; e = { $_.Properties.Estado } },
    @{ n = 'Milisegundos'; e = { $_.Properties.Milisegundos } } | Format-Table -AutoSize
$servicios = @($eventos.Properties.Servicio | Sort-Object -Unique)
if ('Produccion' -notin $servicios -or 'Inventario' -notin $servicios) {
    throw 'Falta la misma correlación en alguno de los dos servicios.'
}
```

Debes observar el mismo `CorrelationId` en ambos servicios, con rutas plantilla `/api/v1/comunicacion/materiales/{material}` y `/api/v1/materiales/{material}`, estado `200` y duración. El campo `TraceId` aporta contexto técnico; este recorrido no instala un exportador ni acredita una plataforma completa de tracing distribuido.

Comprueba también la sustitución de un identificador inválido:

```
$invalida = Invoke-WebRequest 'http://localhost:5105/api/v1/comunicacion/materiales/Acero' `
    -Headers @{ 'X-Correlation-ID' = 'con espacio' } -SkipHttpErrorCheck
$generada = [string]$invalida.Headers['X-Correlation-ID']
$generada
if ($generada -eq 'con espacio' -or $generada -notmatch '^[A-Za-z0-9_-]{1,64}$') {
    throw 'No se sustituyó la correlación inválida.'
}
Select-String -Path .\produccion-m071.jsonl, .\inventario-m071.jsonl -SimpleMatch $generada
dotnet test .\tests\AceriaApp.Logging.Tests\AceriaApp.Logging.Tests.csproj -c Release --no-build --no-restore
`
    --filter 'FullyQualifiedName~LoggingFuncionalTests'
```

Las pruebas funcionales ejercitan los middleware y Serilog reales con peticiones y transporte controlados: ausencia, longitud y caracteres inválidos del header, aislamiento concurrente, propagación, rutas desconocidas y exclusión de cuerpos, query y `Authorization` del resumen. El recorrido anterior añade dos procesos HTTP y SQL reales.

La garantía de minimización corresponde a los **eventos nuevos de M7**. Los workers heredados de M6 siguen teniendo avisos que reciben la excepción completa; no afirmes que todos los logs de cualquier componente ocultan esos detalles. `EventoId` y `OrdenId` son identificadores del flujo de eventos, no una prueba de correlación HTTP entre todos los procesos. Detén A y B con `Ctrl+C` antes de pasar a otra etapa.

PASO 7.1.18 — Ejecutar todos los tests del punto 7.1

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 12 tests de catálogo y las pruebas funcionales incorporadas progresivamente al proyecto.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 12 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 7.1.19 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 309 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04, M05 y M06: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 309 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

PASO 7.1.20 — Actualizar el README del punto 7.1

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `7.1-logging`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `6.3-cuando-usar-cada-enfoque` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 7.1 — Logging

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.

## Estado del proyecto

- Solución `AceriaApp.sln`: 31 proyectos, 19 proyectos de tests y 309 pruebas acumuladas.
- 12 pruebas nuevas de análisis en `tests/AceriaApp.Logging.Tests`.
- Tipos `ConfiguracionLogging`, `ValidadorLogging` y `CatalogoLogging` en `src/AceriaApp.Api/Logging`.
- Los catálogos no configuran logging, probes, manejadores ni políticas HTTP.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M07 para iniciar los adaptadores y observar logs, salud, errores y recuperación HTTP.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/logging.md` y el ADR-027. Los bloques funcionales se
aplican progresivamente y el paso 17 completa el registro y la compilación del punto. Se conservan los
documentos y pruebas heredados.

## Documentación

- [Tema del punto](docs/logging.md).
- [ADR-027](docs/ADR-027-logging.md).
- Los 34 documentos anteriores, incluidos los ADR-010 a ADR-026, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 31 proyectos, 309 pruebas y 12 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 7.1.21 — Crear el ADR-027 del punto 7.1

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-027-logging.md` dentro de la copia de `7.1-logging`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-027-logging.md` existe y que los 34 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-027-logging.md`. **Operación:** Crear.

```
# ADR-027: Logging

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita observar y limitar fallos distribuidos. Se conservan los
contratos del dominio, SQL Server, RabbitMQ y outbox/inbox de M6. Los catálogos de análisis y la
implementación funcional aportan evidencias diferentes y complementarias.

## Decisión

Integrar Serilog.AspNetCore 8.0.3 con consola JSON, propiedades estructuradas y nivel por entorno. Validar y
propagar X-Correlation-ID. Registrar servicio, identificadores y resumen de petición sin cuerpos, query
strings ni cabeceras de autenticación. El catálogo valida la política declarada y las pruebas funcionales
observan eventos reales.

## Alternativas consideradas

- Mantener solo los catálogos: descartado porque los indicadores no prueban el comportamiento del proceso.
- Duplicar los componentes en Producción e Inventario: descartado; la biblioteca AceriaApp.Observabilidad
comparte la integración ASP.NET sin acoplar el dominio.
- Introducir servicios de observabilidad externos: fuera del alcance del módulo; la consola JSON permite
inspeccionar resultados reales.

## Consecuencias

- Positivas: comportamiento uniforme, contratos explícitos y fallos reproducibles.
- Negativas: deben mantenerse límites, configuración, políticas y pruebas de integración.
- Mitigación: conservar pruebas heredadas, validar errores y cancelación, y separar señales operativas de los
datos sensibles.
```

Verificación concreta. `docs/ADR-027-logging.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-027-logging.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.1.22 — Verificar que no se ha modificado ningún archivo del punto 6.3

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque` y actual `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 34 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 34 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 7.1.23 — Verificar la estructura del punto 7.1 y el `.gitignore`

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, con todos los proyectos dentro de la etapa. Las reglas de la raíz ya excluyen los binarios y resultados de todas las etapas.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/Logging` y las dos clases de tests en `tests/AceriaApp.Logging.Tests`.
3. Comprueba los 36 documentos de `docs`, incluida la ficha del punto y el ADR-027.
4. Ejecuta `dotnet sln AceriaApp.sln list`, `git status --short` y `git check-ignore` sobre un archivo de `bin`, `obj` y `TestResults`; contrasta las rutas y conserva las exclusiones heredadas.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 31 proyectos. Git no propone binarios ni resultados temporales.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 7.1.24 — Hacer el commit del punto 7.1 y verificar el estado final

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 7.1: logging`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 7.1 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorLogging` y una prueba adicional. Trabaja en una copia optativa de `7.1-logging` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 7.1.1 — Definir el contrato del resumen

Reemplaza `src/AceriaApp.Api/Logging/ValidadorLogging.cs` por el archivo completo siguiente, que conserva las reglas y añade el contrato de resumen en el mismo namespace.

Archivo: `src/AceriaApp.Api/Logging/ValidadorLogging.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.Logging;

public class ValidadorLogging
{
    private static readonly string[] NivelesPermitidos = { "Trace", "Debug", "Information", "Warning",
"Error", "Critical" };

    public ResultadoLogging Validar(IEnumerable<ConfiguracionLogging> configuraciones)
    {
        var errores = new List<string>();
        var lista = configuraciones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay configuraciones de logging declaradas.");
        }

        foreach (var configuracion in lista)
        {
            if (!NivelesPermitidos.Contains(configuracion.NivelMinimo))
            {
                errores.Add($"Nivel mínimo no permitido: '{configuracion.NivelMinimo}'.");
            }

            if (!configuracion.UsaLoggingEstructurado)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no usa logging
estructurado.");
            }

            if (!configuracion.TieneCorrelacion)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no tiene
correlación.");
            }

            if (!configuracion.EvitaDatosSensibles)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no evita datos
sensibles.");
            }

            if (!configuracion.ConfiguradoPorEntorno)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no está configurada
por entorno.");
            }
        }

        return new ResultadoLogging(errores.Count == 0, errores);
    }
}

public record ResultadoLogging(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenLogging(int Total, int Estructurado, int ConCorrelacion, int PorEntorno);
```

Reto 7.1.3 — Calcular los recuentos

Reemplaza de nuevo `src/AceriaApp.Api/Logging/ValidadorLogging.cs` por el archivo completo siguiente. El método queda dentro del validador y materializa la secuencia una sola vez.

Archivo: `src/AceriaApp.Api/Logging/ValidadorLogging.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.Logging;
```

```
public class ValidadorLogging
{
    public static ResumenLogging Resumen(IEnumerable<ConfiguracionLogging> configuraciones)
    {
        var lista = configuraciones.ToList();
        var estructurado = lista.Count(c => c.UsaLoggingEstructurado);
        var conCorrelacion = lista.Count(c => c.TieneCorrelacion);
        var porEntorno = lista.Count(c => c.ConfiguradoPorEntorno);
        return new ResumenLogging(lista.Count, estructurado, conCorrelacion, porEntorno);
    }

    private static readonly string[] NivelesPermitidos = { "Trace", "Debug", "Information", "Warning",
"Error", "Critical" };

    public ResultadoLogging Validar(IEnumerable<ConfiguracionLogging> configuraciones)
    {
        var errores = new List<string>();
        var lista = configuraciones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay configuraciones de logging declaradas.");
        }

        foreach (var configuracion in lista)
        {
            if (!NivelesPermitidos.Contains(configuracion.NivelMinimo))
            {
                errores.Add($"Nivel mínimo no permitido: '{configuracion.NivelMinimo}'.");
            }

            if (!configuracion.UsaLoggingEstructurado)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no usa logging
estructurado.");
            }

            if (!configuracion.TieneCorrelacion)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no tiene
correlación.");
            }

            if (!configuracion.EvitaDatosSensibles)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no evita datos
sensibles.");
            }

            if (!configuracion.ConfiguradoPorEntorno)
            {
                errores.Add($"La configuración con nivel '{configuracion.NivelMinimo}' no está configurada
por entorno.");
            }
        }

        return new ResultadoLogging(errores.Count == 0, errores);
    }
}

public record ResultadoLogging(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenLogging(int Total, int Estructurado, int ConCorrelacion, int PorEntorno);
```

Cuenta las entradas que cumplen el criterio indicado. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 7.1.6 — Verificar el catálogo por defecto

Reemplaza `tests/AceriaApp.Logging.Tests/ValidadorLoggingTests.cs` por este archivo completo. Conserva todos los tests obligatorios y añade la comprobación de total 3, estructuradas 3, con correlación 3 y por entorno 3.

Archivo: `tests/AceriaApp.Logging.Tests/ValidadorLoggingTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Logging;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Logging.Tests;

public class ValidadorLoggingTests
{
    private readonly ValidadorLogging _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoLogging.ConfiguracionesPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinConfiguraciones_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ConfiguracionLogging>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay configuraciones"));
    }

    [Fact]
    public void Validar_NivelNoPermitido_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "NivelInventado", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Nivel mínimo no permitido"));
    }

    [Fact]
    public void Validar_SinLoggingEstructurado_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = false, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa logging estructurado"));
    }

    [Fact]
    public void Validar_SinCorrelacion_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = false,
```

```
EvitaDatosSensibles = true, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene correlación"));
    }

    [Fact]
    public void Validar_DatosSensibles_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = false, ConfiguradoPorEntorno = true }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no evita datos sensibles"));
    }

    [Fact]
    public void Validar_SinConfiguracionPorEntorno_DevuelveError()
    {
        var configuraciones = new List<ConfiguracionLogging>
        {
            new() { NivelMinimo = "Information", UsaLoggingEstructurado = true, TieneCorrelacion = true,
EvitaDatosSensibles = true, ConfiguradoPorEntorno = false }
        };

        var resultado = _validador.Validar(configuraciones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está configurada por entorno"));
    }
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorLogging.Resumen(CatalogoLogging.ConfiguracionesPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.Estructurado.Should().Be(3);
        resumen.ConCorrelacion.Should().Be(3);
        resumen.PorEntorno.Should().Be(3);
    }
}
```

Calcula el resumen sobre el catálogo por defecto. Compara el recuento obtenido con el valor esperado del catálogo.

Comprobación del reto. Guarda los archivos, compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.Logging.Tests/AceriaApp.Logging.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 12 a 13 tests; la solución debe ejecutar 309 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y compara los hashes de la etapa obligatoria: no debe haber cambiado. Revisa `git diff --check` y el índice antes de registrar el reto en un commit local separado.

Diagnóstico si falla. Si aparece un tipo duplicado, reemplaza el contenido completo en vez de pegarlo al final. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 12 pruebas de catálogo, la implementación funcional y 36 documentos Markdown. La solución incluye 31 proyectos, 19 proyectos de tests y 309 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Continuación. Copia esta etapa obligatoria para abordar 7.2: Health Checks. Conserva por separado la copia del reto.

Punto 7.2 — Health Checks

Distinguir un proceso vivo de un servicio preparado para trabajar

Un proceso puede seguir escuchando HTTP aunque haya perdido la conexión con SQL Server o con RabbitMQ. Por eso la comprobación de vida no responde a la misma pregunta que la preparación para atender tráfico real. En este punto configuraremos los health checks de ASP.NET Core, identificaremos sus dependencias y aprenderemos a interpretar una respuesta saludable, degradada o no disponible.

La práctica conserva los logs y la correlación del punto 7.1. Los indicadores de catálogo son material explicativo; los endpoints de salud deben comprobarse desde un cliente HTTP y reflejar el estado de las dependencias definidas. Esta separación resulta esencial para entender cómo un orquestador o balanceador decide si envía nuevas peticiones a una instancia.

Carpeta de trabajo: `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar el análisis y el comportamiento ejecutable. La etapa termina con 331 pruebas en 32 proyectos y 20 proyectos de tests.

Ejercicio guiado. Incorpora `HealthCheck`, `ValidadorHealthChecks` y `CatalogoHealthChecks`, dos documentos nuevos y 13 pruebas de catálogo. Los bloques funcionales se aplican progresivamente; el paso 16 completa su registro y comprueba la compilación del punto.

PASO 7.2.1 — Crear la carpeta del punto 7.2 a partir del punto 7.1

La etapa autónoma continúa 31 proyectos y 309 pruebas antes de añadir los cambios del punto.

1. Localiza la etapa completa `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks` y copia en ella el contenido de `7.1-logging`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `7.2-health-checks`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `7.1-logging` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `7.1-logging`: verifica la ruta antes de editar.

PASO 7.2.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `7.2-health-checks` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 31 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 31 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 7.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 7.1

El punto conserva 36 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 36 archivos Markdown.
2. Localiza los ADR-010 a ADR-027 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging`.
4. Comprueba que aún no existen `docs/health-checks.md` ni `docs/ADR-028-health-checks.md`.

Verificación concreta. Hay 36 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 7.2.4 — Crear el documento de health checks en `docs`

Una ficha de health checks debe explicar cuándo un proceso está vivo y cuándo sus dependencias están listas. La diferencia tiene consecuencias concretas para el tráfico recibido por cada instancia.

1. Crea `docs/health-checks.md` dentro de la copia de `7.2-health-checks`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/health-checks.md` existe y que los 36 documentos anteriores permanecen intactos.

Archivo: `docs/health-checks.md`. **Operación:** Crear.

```
# Health Checks — Punto 7.2

## Definición

Punto de entrada que expone el estado del servicio.

## Tipos
```

```
- Liveness: el proceso puede atender. Una plataforma configurada puede reiniciarlo tras sus umbrales; este endpoint no consulta SQL ni RabbitMQ.
- Readiness: el servicio puede atender con sus dependencias críticas. Un balanceador configurado deja de enviar tráfico a una instancia que falla.
- Startup: comprueba el arranque con un presupuesto propio. Una plataforma puede reiniciar el contenedor si agota el umbral; no espera indefinidamente.

## Disponibilidad

- 200 OK: las comprobaciones seleccionadas devuelven el estado admitido por el endpoint.
- 503 Service Unavailable: una comprobación seleccionada impide readiness.
- Sin respuesta: no se ha obtenido evidencia a tiempo; puede ser proceso, red o timeout. No prueba por sí solo que el proceso haya terminado.

## Integración con ASP.NET Core

    builder.Services.AddHealthChecks()
        .AddCheck("self", () => HealthCheckResult.Healthy(), tags: new[] { "live" })
        .AddCheck<SqlServicioHealthCheck>("sql", tags: new[] { "ready" });

    app.MapHealthChecks("/health/live", new HealthCheckOptions
    {
        Predicate = check => check.Tags.Contains("live")
    });
    app.MapHealthChecks("/health/ready", new HealthCheckOptions
    {
        Predicate = check => check.Tags.Contains("ready")
    });

## Health checks personalizados

Implementan `IHealthCheck`. El método `CheckHealthAsync` devuelve un `HealthCheckResult` con estado `Healthy`, `Degraded` o `Unhealthy`.

## Formato de la respuesta

Por defecto: texto plano. Configurable: JSON.

## Health checks en contenedores

- Docker: `HEALTHCHECK`.
- Kubernetes: `livenessProbe`, `readinessProbe`, `startupProbe`.

## Health checks en balanceadores

El balanceador consulta el endpoint de readiness y retira las instancias no disponibles.

## Buenas prácticas

1. Endpoints separados para liveness y readiness.
2. Solo recursos críticos en readiness.
3. Startup check para arranques lentos.
4. Sin información sensible.
5. Tiempos e intervalos configurados.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Endpoint liveness | /health/live |
| Endpoint readiness | /health/ready |
| Comprobaciones | Base de datos, broker |
| Formato | JSON |
| Validación | Test de coherencia |
## Contrato del ejercicio

El fragmento muestra la selección por tags; el archivo completo de registro añade RabbitMQ solo cuando los workers están habilitados, configura límites de espera y serializa una respuesta pública sin excepciones ni cadenas de conexión. Dos rutas sin `Predicate` ejecutarían todas las comprobaciones en ambas y no separarían liveness de readiness.

El catálogo declara recursos, tipo, timeout y protección de información. Su bandera `ConsultaDependenciaExterna` permite detectar un liveness acoplado a una dependencia. No conecta a ningún recurso; los checks funcionales ejecutan SQL y AMQP de verdad. Readiness acredita acceso, no el vaciado del outbox ni la finalización de los consumidores.
```

Verificación concreta. `docs/health-checks.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/health-checks.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.2.5 — Crear la carpeta `HealthChecks` en el proyecto API

`HealthChecks` agrupa datos y reglas del ejercicio de análisis; los adaptadores de comunicación tienen sus propias responsabilidades.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `HealthChecks` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/HealthChecks` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `7.1-logging`.

Verificación concreta. Existe `src/AceriaApp.Api/HealthChecks` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

Integración funcional — `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Serilog.AspNetCore" Version="8.0.3" />
    <PackageReference Include="Microsoft.Data.SqlClient" Version="5.2.2" />
    <PackageReference Include="RabbitMQ.Client" Version="7.1.2" />
  </ItemGroup>
</Project>
```

Fija la dependencia usada por esta etapa del ejercicio.

PASO 7.2.6 — Crear la clase `HealthCheck`

HealthCheck declara un escenario que analizará el catálogo; no prueba por sí mismo una conexión SQL o AMQP. Los endpoints reales deben interrogar las dependencias configuradas y responder según su resultado.

1. Crea `src/AceriaApp.Api/HealthChecks/HealthCheck.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/HealthChecks/HealthCheck.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.HealthChecks;

public class HealthCheck
```

```
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public string Recurso { get; init; } = string.Empty;
    public bool ConsultaDependenciaExterna { get; init; }
    public bool TieneTimeout { get; init; }
    public bool ExponeInformacionSensible { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `HealthCheck` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.HealthChecks` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.2.7 — Crear la clase `ValidadorHealthChecks`

El validador comprueba la coherencia de una descripción de salud. Esa prueba no demuestra readiness: el estado real tiene que obtenerse desde el endpoint con la dependencia disponible y también cuando falla.

1. Crea `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.HealthChecks;

public class ValidadorHealthChecks
{
    private static readonly string[] TiposPermitidos = { "Liveness", "Readiness", "Startup" };

    public ResultadoHealthChecks Validar(IEnumerable<HealthCheck> healthChecks)
    {
        var errores = new List<string>();
        var lista = healthChecks.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay health checks declarados.");
        }

        foreach (var healthCheck in lista)
        {
            if (string.IsNullOrWhiteSpace(healthCheck.Nombre))
            {
                errores.Add("Un health check no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(healthCheck.Tipo))
            {
                errores.Add($"Tipo no permitido en '{healthCheck.Nombre}': '{healthCheck.Tipo}'.");
            }

            if (string.IsNullOrWhiteSpace(healthCheck.Recurso))
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no declara el recurso.");
            }
```

```
            if (healthCheck.Tipo == "Liveness" && healthCheck.ConsultaDependenciaExterna)
            {
                errores.Add($"El liveness '{healthCheck.Nombre}' consulta una dependencia externa.");
            }

            if (!healthCheck.TieneTimeout)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no tiene timeout.");
            }

            if (healthCheck.ExponeInformacionSensible)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' expone información sensible.");
            }
        }

        return new ResultadoHealthChecks(errores.Count == 0, errores);
    }
}

public record ResultadoHealthChecks(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Devuelve validez y diagnósticos; no instala middleware ni ejecuta llamadas remotas. Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorHealthChecks` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.HealthChecks` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.2.8 — Añadir los `using` necesarios en `ValidadorHealthChecks`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.HealthChecks;

public class ValidadorHealthChecks
{
    private static readonly string[] TiposPermitidos = { "Liveness", "Readiness", "Startup" };
```

```
    public ResultadoHealthChecks Validar(IEnumerable<HealthCheck> healthChecks)
    {
        var errores = new List<string>();
        var lista = healthChecks.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay health checks declarados.");
        }

        foreach (var healthCheck in lista)
        {
            if (string.IsNullOrWhiteSpace(healthCheck.Nombre))
            {
                errores.Add("Un health check no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(healthCheck.Tipo))
            {
                errores.Add($"Tipo no permitido en '{healthCheck.Nombre}': '{healthCheck.Tipo}'.");
            }

            if (string.IsNullOrWhiteSpace(healthCheck.Recurso))
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no declara el recurso.");
            }

            if (healthCheck.Tipo == "Liveness" && healthCheck.ConsultaDependenciaExterna)
            {
                errores.Add($"El liveness '{healthCheck.Nombre}' consulta una dependencia externa.");
            }

            if (!healthCheck.TieneTimeout)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no tiene timeout.");
            }

            if (healthCheck.ExponeInformacionSensible)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' expone información sensible.");
            }
        }

        return new ResultadoHealthChecks(errores.Count == 0, errores);
    }
}

public record ResultadoHealthChecks(bool EsValida, IReadOnlyList<string> Errores);
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorHealthChecks.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Integración funcional — `src/AceriaApp.Observabilidad/RabbitServicioHealthCheck.cs`.

La comprobación acredita una conexión AMQP autenticada. Desactiva la recuperación automática porque la conexión solo pertenece a esta sonda; la cierra y libera al terminar. Una cancelación del request se propaga, mientras un fallo de dependencia produce Unhealthy sin publicar URI ni credenciales.

Archivo: `src/AceriaApp.Observabilidad/RabbitServicioHealthCheck.cs`. **Operación:** Crear.

```
using Microsoft.Extensions.Diagnostics.HealthChecks;
using RabbitMQ.Client;
namespace AceriaApp.Observabilidad;
public sealed class RabbitServicioHealthCheck(string uri) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext context, CancellationToken ct =
default)
    {
        ct.ThrowIfCancellationRequested();
        using var plazo = CancellationTokenSource.CreateLinkedTokenSource(ct);
        plazo.CancelAfter(TimeSpan.FromSeconds(2));
        IConnection? conexion = null;
        try
        {
            var factory = new ConnectionFactory { Uri = new Uri(uri), AutomaticRecoveryEnabled = false,
                RequestedConnectionTimeout = TimeSpan.FromSeconds(2) };
            conexion = await factory.CreateConnectionAsync(plazo.Token);
            return conexion.IsOpen ? HealthCheckResult.Healthy() : HealthCheckResult.Unhealthy("AMQP no
disponible.");
        }
        catch
        {
            ct.ThrowIfCancellationRequested();
            return HealthCheckResult.Unhealthy("AMQP no disponible dentro del plazo.");
        }
        finally
        {
            if (conexion is not null)
            {
                await conexion.AbortAsync(TimeSpan.FromMilliseconds(200));
                await conexion.DisposeAsync();
            }
        }
    }
}
```

Combina la cancelación del llamante con el límite propio de la sonda. Verifica el protocolo AMQP y la autenticación, no solo un puerto abierto. Acota el cierre de la conexión AMQP usada por la sonda.

Integración funcional — `src/AceriaApp.Observabilidad/RegistroSalud.cs`.

Liveness responde sobre el proceso y no ejecuta dependencias. Readiness selecciona los checks con etiqueta ready: las bases propias y RabbitMQ cuando los workers están activos. En esta aplicación, tanto Degraded como Unhealthy producen 503. La respuesta expone nombre, estado y duración, sin mensajes internos.

Archivo: `src/AceriaApp.Observabilidad/RegistroSalud.cs`. **Operación:** Crear.

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Routing;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Diagnostics.HealthChecks;
namespace AceriaApp.Observabilidad;
public static class RegistroSalud
{
    public static IServiceCollection AddSaludAceria(this IServiceCollection services, IConfiguration config,
bool inventario)
    {
        var health = services.AddHealthChecks();
        var bases = inventario ? new[] { "AceriaInventario" } : new[] { "AceriaApp", "AceriaHexagonal",
"AceriaProduccion" };
        foreach (var nombre in bases)
            health.AddCheck("sql-" + nombre, new SqlServicioHealthCheck(config.GetConnectionString(nombre) ??
""),
                tags: new[] { "ready" }, timeout: TimeSpan.FromSeconds(3));
        if (config.GetValue<bool>("Comunicacion:WorkersActivos"))
            health.AddCheck("rabbitmq", new RabbitServicioHealthCheck(config["Comunicacion:RabbitMqUri"] ??
""),
```

```
                tags: new[] { "ready" }, timeout: TimeSpan.FromSeconds(3));
        return services;
    }
    public static IEndpointRouteBuilder MapSaludAceria(this IEndpointRouteBuilder endpoints)
    {
        endpoints.MapHealthChecks("/health/live", Opciones(_ => false));
        endpoints.MapHealthChecks("/health/ready", Opciones(r => r.Tags.Contains("ready")));
        return endpoints;
    }
    private static HealthCheckOptions Opciones(Func<HealthCheckRegistration, bool> filtro) => new()
    {
        Predicate = filtro,
        ResultStatusCodes = { [HealthStatus.Healthy] = 200, [HealthStatus.Degraded] = 503,
[HealthStatus.Unhealthy] = 503 },
        ResponseWriter = (context, report) => context.Response.WriteAsJsonAsync(new
        {
            estado = report.Status.ToString(),
            componentes = report.Entries.Select(e => new
            { nombre = e.Key, estado = e.Value.Status.ToString(), milisegundos =
e.Value.Duration.TotalMilliseconds })
        }, cancellationToken: context.RequestAborted)
    };
}
```

Registra las dependencias propias de este servicio para readiness. Publica liveness y readiness como rutas independientes. Selecciona qué checks ejecuta este endpoint.

Hace explícito el código HTTP de cada estado de salud. Distingue una cancelación del cliente de un fallo de la dependencia.

Integración funcional — `src/AceriaApp.Observabilidad/SqlServicioHealthCheck.cs`.

El probe abre una conexión y ejecuta SELECT 1 en la base configurada. No basta con que responda el puerto del servidor. Se limita la conexión y la consulta, se propaga la cancelación del llamante y se devuelve una descripción genérica cuando falla una dependencia.

Archivo: `src/AceriaApp.Observabilidad/SqlServicioHealthCheck.cs`. **Operación:** Crear.

```
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Diagnostics.HealthChecks;
namespace AceriaApp.Observabilidad;
public sealed class SqlServicioHealthCheck(string cadena) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext context, CancellationToken ct =
default)
    {
        ct.ThrowIfCancellationRequested();
        using var plazo = CancellationTokenSource.CreateLinkedTokenSource(ct);
        plazo.CancelAfter(TimeSpan.FromSeconds(2));
        try
        {
            if (string.IsNullOrWhiteSpace(cadena)) return HealthCheckResult.Unhealthy("SQL sin configurar.");
            var opciones = new SqlConnectionStringBuilder(cadena) { ConnectTimeout = 2 };
            await using var conexion = new SqlConnection(opciones.ConnectionString);
            await conexion.OpenAsync(plazo.Token);
            await using var consulta = conexion.CreateCommand();
            consulta.CommandText = "SELECT 1"; consulta.CommandTimeout = 2;
            var valor = await consulta.ExecuteScalarAsync(plazo.Token);
            return Convert.ToInt32(valor) == 1 ? HealthCheckResult.Healthy() :
HealthCheckResult.Unhealthy("SQL no responde correctamente.");
        }
        catch (Exception ex) when (ex is SqlException or ArgumentException or OperationCanceledException)
        {
            ct.ThrowIfCancellationRequested();
            return HealthCheckResult.Unhealthy("SQL no disponible dentro del plazo.");
        }
    }
}
```

Abre realmente la conexión a la base de datos seleccionada. Comprueba la ejecución SQL dentro de la dependencia.

PASO 7.2.9 — Crear la clase `CatalogoHealthChecks`

Cada invocación devuelve una lista nueva con los ejemplos de análisis; cambiar una lista no modifica futuros catálogos.

1. Crea `src/AceriaApp.Api/HealthChecks/CatalogoHealthChecks.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/HealthChecks/CatalogoHealthChecks.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.HealthChecks;

public class CatalogoHealthChecks
{
    public static List<HealthCheck> HealthChecksPorDefecto()
    {
        return new List<HealthCheck>
        {
            new() { Nombre = "Self", Tipo = "Liveness", Recurso = "Aplicación", TieneTimeout = true,
ExponeInformacionSensible = false },
            new() { Nombre = "Database", Tipo = "Readiness", ConsultaDependenciaExterna = true, Recurso =
"SQL Server Express", TieneTimeout = true, ExponeInformacionSensible = false },
            new() { Nombre = "RabbitMQ", Tipo = "Readiness", ConsultaDependenciaExterna = true, Recurso =
"Broker de mensajería", TieneTimeout = true, ExponeInformacionSensible = false }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoHealthChecks` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.HealthChecks` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.2.10 — Añadir el `using` necesario en `CatalogoHealthChecks`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/HealthChecks/CatalogoHealthChecks.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/HealthChecks/CatalogoHealthChecks.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;

namespace AceriaApp.Api.HealthChecks;

public class CatalogoHealthChecks
{
    public static List<HealthCheck> HealthChecksPorDefecto()
    {
        return new List<HealthCheck>
        {
            new() { Nombre = "Self", Tipo = "Liveness", Recurso = "Aplicación", TieneTimeout = true,
ExponeInformacionSensible = false },
            new() { Nombre = "Database", Tipo = "Readiness", ConsultaDependenciaExterna = true, Recurso =
"SQL Server Express", TieneTimeout = true, ExponeInformacionSensible = false },
            new() { Nombre = "RabbitMQ", Tipo = "Readiness", ConsultaDependenciaExterna = true, Recurso =
"Broker de mensajería", TieneTimeout = true, ExponeInformacionSensible = false }
        };
    }
}
```

Verificación concreta. `CatalogoHealthChecks.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.2.11 — Crear el proyecto de tests del punto 7.2

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.HealthChecks.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 14 y 15 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj` y `dotnet restore tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.

- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 7.2.12 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj` y `dotnet build tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

PASO 7.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj` y `dotnet list tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
```

```
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

Integración funcional — `src/AceriaApp.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM072;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM072;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM072;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m072",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

Agrupa las conexiones de laboratorio específicas de la etapa. Activa o desactiva los procesos de mensajería y su dependencia en readiness. Fija el umbral propio de logging y admite sobreescritura por variables de entorno.

Mantiene el endpoint de fallo desactivado por defecto.

Integración funcional — `src/AceriaApp.Inventario.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Inventario.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM072;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
```

```
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m072",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

PASO 7.2.14 — Crear el archivo `ValidadorHealthChecksTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.HealthChecks.Tests/ValidadorHealthChecksTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.HealthChecks.Tests/ValidadorHealthChecksTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.HealthChecks;
using FluentAssertions;
using Xunit;

namespace AceriaApp.HealthChecks.Tests;

public class ValidadorHealthChecksTests
{
    private readonly ValidadorHealthChecks _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoHealthChecks.HealthChecksPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinHealthChecks_DevuelveError()
    {
        var resultado = _validador.Validar(new List<HealthCheck>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay health checks"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", Recurso = "A", TieneTimeout = true }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinRecurso_DevuelveError()
    {
```

```
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = string.Empty, TieneTimeout = true }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el recurso"));
    }

    [Fact]
    public void Validar_SinTimeout_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = "A", TieneTimeout = false }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene timeout"));
    }

    [Fact]
    public void Validar_InformacionSensible_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = "A", TieneTimeout = true,
ExponeInformacionSensible = true }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("expone información sensible"));
    }
    [Fact]
    public void Validar_LivenessConDependenciaExterna_DevuelveError()
    {
        var checks = new List<HealthCheck>
        {
            new() { Nombre = "Self", Tipo = "Liveness", Recurso = "SQL Server", ConsultaDependenciaExterna =
true, TieneTimeout = true }
        };

        var resultado = _validador.Validar(checks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("consulta una dependencia externa"));
    }

    [Fact]
    public void Validar_ReadinessConDependenciaExterna_DevuelveValido()
    {
        var checks = new List<HealthCheck>
        {
            new() { Nombre = "SQL", Tipo = "Readiness", Recurso = "SQL Server", ConsultaDependenciaExterna =
true, TieneTimeout = true }
        };

        var resultado = _validador.Validar(checks);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorHealthChecksTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 7.2.15 — Crear el archivo `CatalogoHealthChecksTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.HealthChecks.Tests/CatalogoHealthChecksTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.HealthChecks.Tests/CatalogoHealthChecksTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.HealthChecks;
using FluentAssertions;
using Xunit;

namespace AceriaApp.HealthChecks.Tests;

public class CatalogoHealthChecksTests
{
    [Fact]
    public void HealthChecksPorDefecto_ContieneTresHealthChecks()
    {
        var healthChecks = CatalogoHealthChecks.HealthChecksPorDefecto();

        healthChecks.Should().HaveCount(3);
    }

    [Fact]
    public void HealthChecksPorDefecto_ContieneUnLiveness()
    {
        var healthChecks = CatalogoHealthChecks.HealthChecksPorDefecto();

        healthChecks.Count(h => h.Tipo == "Liveness").Should().Be(1);
    }

    [Fact]
    public void HealthChecksPorDefecto_ContieneDosReadiness()
    {
        var healthChecks = CatalogoHealthChecks.HealthChecksPorDefecto();

        healthChecks.Count(h => h.Tipo == "Readiness").Should().Be(2);
    }

    [Fact]
    public void HealthChecksPorDefecto_TodosTienenTimeout()
    {
        var healthChecks = CatalogoHealthChecks.HealthChecksPorDefecto();

        healthChecks.Should().OnlyContain(h => h.TieneTimeout);
    }
```

```
    [Fact]
    public void HealthChecksPorDefecto_NingunoExponeInformacionSensible()
    {
        var healthChecks = CatalogoHealthChecks.HealthChecksPorDefecto();

        healthChecks.Should().OnlyContain(h => !h.ExponeInformacionSensible);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Compara el recuento obtenido con el valor esperado del catálogo.

Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoHealthChecksTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Integración funcional — `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />

    <PackageReference Include="Microsoft.AspNetCore.TestHost" Version="8.0.21" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Testcontainers.RabbitMq" Version="3.10.0" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Conecta este proyecto con otro proyecto real de la solución.

Integración funcional — `tests/AceriaApp.HealthChecks.Tests/SaludFuncionalTests.cs`.

La suite combina siete comprobaciones controladas de contratos con dos comprobaciones de infraestructura real: una base SQL existente frente a otra inexistente y una conexión AMQP autenticada. Los helpers SQL y RabbitMQ crean recursos aislados o usan las conexiones de laboratorio configuradas; un error de infraestructura hace fallar la prueba.

Archivo: `tests/AceriaApp.HealthChecks.Tests/SaludFuncionalTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Text.Json;
using AceriaApp.Observabilidad;
using AceriaApp.PruebasSqlServer;
using Microsoft.AspNetCore.Builder;
```

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.TestHost;
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Diagnostics.HealthChecks;
using Microsoft.Extensions.Hosting;
using Testcontainers.RabbitMq;
using Xunit;
namespace AceriaApp.HealthChecks.Tests;
public class SaludFuncionalTests
{
    private static Task<IHost> Servidor(Action<IHealthChecksBuilder> registrar) => new HostBuilder()
        .ConfigureWebHost(web => web.UseTestServer()
            .ConfigureServices(services => { services.AddRouting(); registrar(services.AddHealthChecks()); })
            .Configure(app => { app.UseRouting(); app.UseEndpoints(endpoints =>
endpoints.MapSaludAceria()); }))
        .StartAsync();
    [Fact]
    public async Task Liveness_no_ejecuta_dependencias_y_readiness_si()
    {
        var consultas = 0;
        using var host = await Servidor(b => b.AddCheck("sql", () => { consultas++; return
HealthCheckResult.Unhealthy("interno"); }, tags: new[] { "ready" }));
        using var http = host.GetTestClient();
        using var live = await http.GetAsync("/health/live");
        Assert.Equal(HttpStatusCode.OK, live.StatusCode); Assert.Equal(0, consultas);
        using var ready = await http.GetAsync("/health/ready");
        Assert.Equal(HttpStatusCode.ServiceUnavailable, ready.StatusCode); Assert.Equal(1, consultas);
    }
    [Fact]
    public async Task Readiness_expone_estado_y_nombre_sin_credenciales_ni_excepcion()
    {
        using var host = await Servidor(b => b.AddCheck("sql", () =>
HealthCheckResult.Unhealthy("Password=secreto-descripcion", new Exception("secreto-excepcion")), tags: new[]
{ "ready" }));
        using var http = host.GetTestClient(); using var response = await http.GetAsync("/health/ready");
        var texto = await response.Content.ReadAsStringAsync(); using var doc = JsonDocument.Parse(texto);
        Assert.Equal(HttpStatusCode.ServiceUnavailable, response.StatusCode);
        Assert.Equal("Unhealthy", doc.RootElement.GetProperty("estado").GetString());
        Assert.Equal("sql",
Assert.Single(doc.RootElement.GetProperty("componentes").EnumerateArray()).GetProperty("nombre").GetString())
;
        Assert.DoesNotContain("secreto", texto); Assert.DoesNotContain("Password", texto);
Assert.DoesNotContain("exception", texto, StringComparison.OrdinalIgnoreCase);
    }
    [Fact]
    public async Task Readiness_sana_responde_200_y_JSON_comprobable()
    {
        using var host = await Servidor(b => b.AddCheck("sql", () => HealthCheckResult.Healthy(), tags: new[]
{ "ready" }));
        using var http = host.GetTestClient(); using var response = await http.GetAsync("/health/ready");
        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
        using var doc = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        Assert.Equal("Healthy", doc.RootElement.GetProperty("estado").GetString());
    }
    [Fact]
    public async Task SQL_no_accesible_no_se_presenta_como_sano()
    {
        var check = new SqlServicioHealthCheck("Server=127.0.0.1,1;User Id=sa;Password=secreto-
sql;TrustServerCertificate=True;Connect Timeout=1");
        var resultado = await check.CheckHealthAsync(new HealthCheckContext());
        Assert.Equal(HealthStatus.Unhealthy, resultado.Status);
        Assert.Null(resultado.Exception); Assert.DoesNotContain("secreto", resultado.Description ?? "");
    }
    [Fact]
    public async Task SQL_cancelado_no_intenta_recuperarse_ni_inventa_resultado()
    {
        using var cancelacion = new CancellationTokenSource(); cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(() => new
SqlServicioHealthCheck("Server=127.0.0.1,1").CheckHealthAsync(new HealthCheckContext(), cancelacion.Token));
    }
    [Fact]
    public async Task Rabbit_no_accesible_devuelve_no_sano_sin_detalles_internos()
    {
        var resultado = await new RabbitServicioHealthCheck("amqp://usuario:secreto-
rabbit@127.0.0.1:1/").CheckHealthAsync(new HealthCheckContext());
```

```
        Assert.Equal(HealthStatus.Unhealthy, resultado.Status); Assert.Null(resultado.Exception);
        Assert.DoesNotContain("secreto", resultado.Description ?? "");
    }
    [Fact]
    public async Task Rabbit_cancelado_respeta_el_token_del_llamador()
    {
        using var cancelacion = new CancellationTokenSource(); cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(() => new RabbitServicioHealthCheck("amqp://
guest:guest@127.0.0.1:1/").CheckHealthAsync(new HealthCheckContext(), cancelacion.Token));
    }
    [Fact]
    public async Task Sql_real_distingue_base_existente_y_nombre_inexistente()
    {
        await using var bd = BaseSqlServer.Crear();
        var sana = await new SqlServicioHealthCheck(bd.ConnectionString).CheckHealthAsync(new
HealthCheckContext());
        var ausente = new SqlConnectionStringBuilder(bd.ConnectionString) { InitialCatalog = "AceriaMissing_"
+ Guid.NewGuid().ToString("N") };
        var mala = await new SqlServicioHealthCheck(ausente.ConnectionString).CheckHealthAsync(new
HealthCheckContext());
        Assert.Equal(HealthStatus.Healthy, sana.Status); Assert.Equal(HealthStatus.Unhealthy, mala.Status);
    }
    [Fact]
    public async Task Rabbit_real_acredita_conexion_AMQP()
    {
        RabbitMqContainer? temporal = null;
        var uri = Environment.GetEnvironmentVariable("ACERIA_TEST_RABBITMQ");
        try
        {
            if (string.IsNullOrWhiteSpace(uri))
            {
                temporal = new RabbitMqBuilder().WithImage("rabbitmq:4.1.5-
management").WithUsername("curso").WithPassword("curso-local").Build();
                await temporal.StartAsync(); uri = temporal.GetConnectionString();
            }
            var estado = await new RabbitServicioHealthCheck(uri).CheckHealthAsync(new HealthCheckContext());
            Assert.Equal(HealthStatus.Healthy, estado.Status);
        }
        finally { if (temporal is not null) await temporal.DisposeAsync(); }
    }
}
```

Define un escenario asíncrono cuyo resultado verifica la conducta descrita por el nombre.

PASO 7.2.16 — Compilar la solución completa

La compilación acredita el registro de los componentes. Las pruebas HTTP deben confirmar que la vida del proceso y la disponibilidad de SQL Server o RabbitMQ no se confunden al devolver el estado.

1. Aplica los bloques de registro final de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 32 proyectos y termina sin errores.

Registro de biblioteca compartida. Ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`; los proyectos de tests se incorporan en sus pasos de creación. Después de reemplazar todos los archivos de este paso, compila la solución.

Verificación concreta. La solución de la etapa compila con 32 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

Integración funcional — `src/AceriaApp.Api/Program.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Api.Comunicacion;
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddLoggingAceria("Produccion", builder.Configuration);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
builder.Services.AddComunicacion(builder.Configuration);
builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>("inventario-instrumentado", http =>
{
    http.BaseAddress = new Uri(builder.Configuration["Comunicacion:InventarioUrl"] ?? "http://
localhost:5106/");
    http.Timeout = TimeSpan.FromSeconds(3);
}).AddHttpMessageHandler<PropagarCorrelacionHandler>();
builder.Services.AddSaludAceria(builder.Configuration, inventario: false);
var app=builder.Build();
app.UseLoggingAceria();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
```

```
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
```

```
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
app.MapProduccion();
app.MapComunicacion();
app.MapSaludAceria();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Registra el logger JSON y los servicios que necesita la correlación. Propaga el identificador validado hacia Inventario. Abre la correlación y el resumen antes de ejecutar el resto del pipeline.

Integración funcional — `src/AceriaApp.Inventario.Api/InventarioProgram.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddLoggingAceria("Inventario", builder.Configuration);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
```

```
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        builder.Services.AddSaludAceria(builder.Configuration, inventario: true);
        var app = builder.Build();
        app.UseLoggingAceria();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapGet("/api/v1/materiales/{material}", async (string material, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapGet("/api/v1/planificaciones/{ordenId:guid}", async (Guid ordenId, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
valor.CantidadTotal });
        });
        app.MapSaludAceria();
        await app.RunAsync();
    }
}
```

Recorrido funcional 7.2 — proceso vivo y dependencias disponibles

Abre A, B y C en `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks`. Usa PowerShell 7, .NET 8, LocalDB y Docker Desktop con contenedores Linux. Los puertos `5105`, `5106`, `5672` y `15672` deben estar disponibles. La alternativa SQL Server Express del recorrido 7.1 permite conservar los nombres `AceriaAppM072`, `AceriaHexagonalM072`, `AceriaProduccionM072` y `AceriaInventarioM072` cambiando solo el servidor.

En C, prepara SQL, compila y arranca el servicio RabbitMQ del `compose.yml` heredado de M6:

```
sqllocaldb start MSSQLLocalDB
if ($LASTEXITCODE -ne 0) { throw 'No se pudo iniciar MSSQLLocalDB.' }
dotnet restore .\AceriaApp.sln
if ($LASTEXITCODE -ne 0) { throw 'Falló la restauración.' }
dotnet build .\AceriaApp.sln -c Release --no-restore
if ($LASTEXITCODE -ne 0) { throw 'Falló la compilación.' }
docker compose -f .\compose.yml up -d rabbitmq
if ($LASTEXITCODE -ne 0) { throw 'No se pudo arrancar RabbitMQ.' }
docker compose -f .\compose.yml exec -T rabbitmq rabbitmqctl await_startup --timeout 60
if ($LASTEXITCODE -ne 0) { throw 'RabbitMQ no terminó de arrancar.' }
docker compose -f .\compose.yml ps
```

`rabbitmqctl await_startup` espera a que la aplicación RabbitMQ arranque; el límite de espera evita una comprobación bloqueada indefinidamente (referencia del comando).

El archivo utiliza `rabbitmq:4.1.5-management`, AMQP en `127.0.0.1:5672`, usuario local `curso`, contraseña local `curso-local` y volumen `rabbitmq-datos`. No son credenciales de producción. Los dos procesos compartirán el prefijo de esta etapa, `aceria-m072`.

En B, inicia Inventario con workers activos:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'true'
$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'
$env:Comunicacion__Prefijo = 'aceria-m072'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106 2>&1 |
    Tee-Object -FilePath .\inventario-m072.jsonl
```

En A, inicia Producción con los mismos ajustes de mensajería:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'true'
$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'
$env:Comunicacion__Prefijo = 'aceria-m072'
$env:Comunicacion__InventarioUrl = 'http://localhost:5106/'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105 2>&1 |
    Tee-Object -FilePath .\produccion-m072.jsonl
```

En C, consulta los cuatro endpoints. No continúes hasta que las dos respuestas de readiness estén sanas:

```
foreach ($puerto in 5105, 5106) {
    foreach ($tipo in 'live', 'ready') {
        $r = Invoke-WebRequest "http://localhost:$puerto/health/$tipo" -SkipHttpErrorCheck
        [pscustomobject]@{ Puerto = $puerto; Prueba = $tipo; HTTP = $r.StatusCode; Cuerpo = $r.Content }
        if ($r.StatusCode -ne 200) { throw "El estado inicial $puerto/$tipo no es sano." }
    }
}
```

`live` devuelve `Healthy` con una colección vacía de componentes: no consulta SQL ni RabbitMQ. `ready` incluye `sql-AceriaApp`, `sql-AceriaHexagonal`, `sql-AceriaProduccion` y `rabbitmq` en Producción; en Inventario incluye `sql-AceriaInventario` y `rabbitmq`. La ruta heredada `/health` de Inventario conserva su significado anterior y **no sustituye** a `/health/ready`.

Detén únicamente el RabbitMQ de este proyecto y consulta otra vez, sin detener las API:

```
docker compose -f .\compose.yml stop rabbitmq
foreach ($puerto in 5105, 5106) {
    $live = Invoke-WebRequest "http://localhost:$puerto/health/live" -SkipHttpErrorCheck
    $ready = Invoke-WebRequest "http://localhost:$puerto/health/ready" -SkipHttpErrorCheck
    [pscustomobject]@{ Puerto = $puerto; Live = $live.StatusCode; Ready = $ready.StatusCode; Detalle =
$ready.Content }
    $salud = $ready.Content | ConvertFrom-Json
    if ($live.StatusCode -ne 200 -or $ready.StatusCode -ne 503) { throw 'Live y ready no distinguen la
caída.' }
    if (($salud.componentes | Where-Object nombre -eq 'rabbitmq').estado -ne 'Unhealthy') {
        throw 'RabbitMQ no aparece como dependencia caída.'
    }
}
```

```
docker compose -f .\compose.yml start rabbitmq
docker compose -f .\compose.yml exec -T rabbitmq rabbitmqctl await_startup --timeout 60
```

Repite las peticiones de readiness para comprobar que vuelven a `200`. Pueden aparecer avisos de reconexión de los workers M6. El JSON público de salud solo contiene estados, nombres y duraciones; no incluye la URI AMQP, credenciales ni mensajes de excepción. Readiness demuestra acceso a las dependencias; no garantiza que el outbox esté vacío ni que todos los mensajes hayan sido consumidos.

Caída SQL sin detener una instancia compartida. Detén solo A con `Ctrl+C`. En esa misma terminal, guarda la conexión previa y apunta la base de Producción a un nombre único que no vas a crear. Mantén el servidor real en ejecución; desactiva el worker para aislar esta comprobación:

```
$conexionAnterior = $env:ConnectionStrings__AceriaProduccion
$nombreAusente = 'AceriaNoExisteM072_' + [guid]::NewGuid().ToString('N')
$conexionEtapa = (Get-Content .\src\AceriaApp.Api\appsettings.json -Raw | ConvertFrom-
Json).ConnectionStrings.AceriaProduccion
if ($conexionAnterior) { $conexionEtapa = $conexionAnterior }
$constructor = [System.Data.Common.DbConnectionStringBuilder]::new()
$constructor.ConnectionString = $conexionEtapa
[void]$constructor.Remove('Initial Catalog')
$constructor['Database'] = $nombreAusente
$env:ConnectionStrings__AceriaProduccion = $constructor.ConnectionString
$env:SkipDatabaseInitialization = 'true'
$env:Comunicacion__WorkersActivos = 'false'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105
```

`SkipDatabaseInitialization=true` impide que `EnsureCreated` convierta esta base ausente en una base válida. En C:

```
$liveSql = Invoke-WebRequest 'http://localhost:5105/health/live' -SkipHttpErrorCheck
$readySql = Invoke-WebRequest 'http://localhost:5105/health/ready' -SkipHttpErrorCheck
$liveSql.StatusCode
$readySql.StatusCode
$readySql.Content
$estadoSql = $readySql.Content | ConvertFrom-Json
if ($liveSql.StatusCode -ne 200 -or $readySql.StatusCode -ne 503) { throw 'No se observó la caída SQL.' }
if (($estadoSql.componentes | Where-Object nombre -eq 'sql-AceriaProduccion').estado -ne 'Unhealthy') {
    throw 'No se identificó la base ausente.'
}
```

Los otros dos checks SQL deben seguir sanos. Aquí no aparece `rabbitmq` en Producción porque sus workers se desactivaron. No uses `sqllocaldb stop`, no borres bases y no detengas el servicio SQL compartido.

Para recuperar A, pulsa `Ctrl+C` y ejecuta:

```
if ($conexionAnterior) { $env:ConnectionStrings__AceriaProduccion = $conexionAnterior }
else { Remove-Item Env:ConnectionStrings__AceriaProduccion -ErrorAction SilentlyContinue }
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'true'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105
```

En C, comprueba la recuperación y ejecuta las pruebas funcionales contra LocalDB y el RabbitMQ real ya arrancado:

```
Invoke-WebRequest 'http://localhost:5105/health/ready' -SkipHttpErrorCheck | Select-Object StatusCode,
Content
$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)
\MSSQLLocalDB;Database=master;Trusted_Connection=True;TrustServerCertificate=True'
$env:ACERIA_TEST_RABBITMQ = 'amqp://curso:curso-local@localhost:5672/'
dotnet test .\tests\AceriaApp.HealthChecks.Tests\AceriaApp.HealthChecks.Tests.csproj -c Release --no-build --
no-restore `
    --filter 'FullyQualifiedName~SaludFuncionalTests'
```

Si utilizaste SQL Server Express, sustituye el servidor de `ACERIA_TEST_SQLSERVER` por `localhost\SQLEXPRESS`. El usuario de pruebas necesita crear y eliminar sus bases `AceriaTest_<guid>`; no se reemplaza SQL por memoria o SQLite. La suite combina pruebas HTTP controladas de formato, cancelación y selección de checks con conexiones reales SQL/AMQP. Finaliza con `Ctrl+C` en A y B; después puedes ejecutar `docker compose -f .\compose.yml stop rabbitmq`. No uses `down -v`: no necesitas borrar datos para esta comprobación.

PASO 7.2.17 — Ejecutar todos los tests del punto 7.2

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 13 tests de catálogo y las pruebas funcionales incorporadas progresivamente al proyecto.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 13 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 7.2.18 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 331 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04, M05 y M06: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 331 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

PASO 7.2.19 — Actualizar el README del punto 7.2

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `7.2-health-checks`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `7.1-logging` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 7.2 — Health Checks

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.

## Estado del proyecto

- Solución `AceriaApp.sln`: 32 proyectos, 20 proyectos de tests y 331 pruebas acumuladas.
- 13 pruebas nuevas de análisis en `tests/AceriaApp.HealthChecks.Tests`.
- Tipos `HealthCheck`, `ValidadorHealthChecks` y `CatalogoHealthChecks` en `src/AceriaApp.Api/HealthChecks`.
- Los catálogos no configuran logging, probes, manejadores ni políticas HTTP.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M07 para iniciar los adaptadores y observar logs, salud, errores y recuperación HTTP.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/health-checks.md` y el ADR-028. Los bloques funcionales
se aplican progresivamente y el paso 16 completa el registro y la compilación del punto. Se conservan los
documentos y pruebas heredados.

## Documentación

- [Tema del punto](docs/health-checks.md).
- [ADR-028](docs/ADR-028-health-checks.md).
- Los 36 documentos anteriores, incluidos los ADR-010 a ADR-027, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 32 proyectos, 331 pruebas y 13 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 7.2.20 — Crear el ADR-028 del punto 7.2

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-028-health-checks.md` dentro de la copia de `7.2-health-checks`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-028-health-checks.md` existe y que los 36 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-028-health-checks.md`. **Operación:** Crear.

```
# ADR-028: Health Checks

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita observar y limitar fallos distribuidos. Se conservan los
contratos del dominio, SQL Server, RabbitMQ y outbox/inbox de M6. Los catálogos de análisis y la
implementación funcional aportan evidencias diferentes y complementarias.
```

```
## Decisión

Separar /health/live de /health/ready mediante selección explícita. Liveness no consulta dependencias.
Readiness consulta SQL Server y RabbitMQ cuando los workers están habilitados; ambos respetan cancelación y
límites de espera. La respuesta pública omite secretos y excepciones. Se mantiene /health heredado de
Inventario.

## Alternativas consideradas

- Mantener solo los catálogos: descartado porque los indicadores no prueban el comportamiento del proceso.
- Duplicar los componentes en Producción e Inventario: descartado; la biblioteca AceriaApp.Observabilidad
comparte la integración ASP.NET sin acoplar el dominio.
- Introducir servicios de observabilidad externos: fuera del alcance del módulo; la consola JSON permite
inspeccionar resultados reales.

## Consecuencias

- Positivas: comportamiento uniforme, contratos explícitos y fallos reproducibles.
- Negativas: deben mantenerse límites, configuración, políticas y pruebas de integración.
- Mitigación: conservar pruebas heredadas, validar errores y cancelación, y separar señales operativas de los
datos sensibles.
```

Verificación concreta. `docs/ADR-028-health-checks.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-028-health-checks.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.2.21 — Verificar que no se ha modificado ningún archivo del punto 7.1

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.1-logging` y actual `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 36 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 36 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 7.2.22 — Verificar la estructura del punto 7.2

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, con todos los proyectos dentro de la etapa.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/HealthChecks` y las dos clases de tests en `tests/AceriaApp.HealthChecks.Tests`.

3. Comprueba los 38 documentos de `docs`, incluida la ficha del punto y el ADR-028.
4. Ejecuta `dotnet sln AceriaApp.sln list` y contrasta sus 32 proyectos con los `.csproj` presentes.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 32 proyectos.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 7.2.23 — Verificar el `.gitignore` del punto 7.2

Las exclusiones se aplican desde la raíz a las etapas; no es necesario inventar un `.gitignore` propio dentro de cada copia.

1. Abre el `.gitignore` de la raíz del repositorio del curso.
2. Comprueba las reglas `**/bin/`, `**/obj/` y `**/TestResults/` que ya excluyen los artefactos de todas las etapas.
3. Ejecuta `git status --short` y revisa que no aparezcan binarios, cachés ni resultados temporales.
4. Conserva las exclusiones existentes; si aparecen archivos locales de Visual Studio, excluye `.vs/`, `*.user` y `*.suo` mediante `.git/info/exclude`, sin ignorar código ni configuración entregable.

Verificación concreta. Los archivos compilados y resultados de tests están excluidos; los archivos del punto siguen visibles para Git.

Diagnóstico si falla.

- Ignorar todo `tests/` elimina la evidencia reproducible: excluye solo sus artefactos generados.
- Esperar un `.gitignore` dentro de cada etapa puede llevar a crear duplicados innecesarios: revisa primero el de la raíz.

PASO 7.2.24 — Hacer el commit del punto 7.2 y verificar el estado final

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 7.2: health checks`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 7.2 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorHealthChecks` y una prueba adicional. Trabaja en una copia optativa de `7.2-health-checks` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 7.2.1 — Definir el contrato del resumen

Reemplaza `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs` por el archivo completo siguiente, que conserva las reglas y añade el contrato de resumen en el mismo namespace.

Archivo: `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.HealthChecks;

public class ValidadorHealthChecks
{
    private static readonly string[] TiposPermitidos = { "Liveness", "Readiness", "Startup" };

    public ResultadoHealthChecks Validar(IEnumerable<HealthCheck> healthChecks)
    {
        var errores = new List<string>();
        var lista = healthChecks.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay health checks declarados.");
        }

        foreach (var healthCheck in lista)
        {
            if (string.IsNullOrWhiteSpace(healthCheck.Nombre))
            {
                errores.Add("Un health check no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(healthCheck.Tipo))
            {
                errores.Add($"Tipo no permitido en '{healthCheck.Nombre}': '{healthCheck.Tipo}'.");
            }

            if (string.IsNullOrWhiteSpace(healthCheck.Recurso))
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no declara el recurso.");
            }

            if (healthCheck.Tipo == "Liveness" && healthCheck.ConsultaDependenciaExterna)
            {
                errores.Add($"El liveness '{healthCheck.Nombre}' consulta una dependencia externa.");
            }

            if (!healthCheck.TieneTimeout)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no tiene timeout.");
            }

            if (healthCheck.ExponeInformacionSensible)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' expone información sensible.");
            }
        }

        return new ResultadoHealthChecks(errores.Count == 0, errores);
    }
}

public record ResultadoHealthChecks(bool EsValida, IReadOnlyList<string> Errores);
```

```
public record ResumenHealthChecks(int Total, int Liveness, int Readiness, int Startup);
```

Reto 7.2.3 — Calcular los recuentos

Reemplaza de nuevo `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs` por el archivo completo siguiente. El método queda dentro del validador y materializa la secuencia una sola vez.

Archivo: `src/AceriaApp.Api/HealthChecks/ValidadorHealthChecks.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.HealthChecks;

public class ValidadorHealthChecks
{
    public static ResumenHealthChecks Resumen(IEnumerable<HealthCheck> healthChecks)
    {
        var lista = healthChecks.ToList();
        var liveness = lista.Count(h => h.Tipo == "Liveness");
        var readiness = lista.Count(h => h.Tipo == "Readiness");
        var startup = lista.Count(h => h.Tipo == "Startup");
        return new ResumenHealthChecks(lista.Count, liveness, readiness, startup);
    }

    private static readonly string[] TiposPermitidos = { "Liveness", "Readiness", "Startup" };

    public ResultadoHealthChecks Validar(IEnumerable<HealthCheck> healthChecks)
    {
        var errores = new List<string>();
        var lista = healthChecks.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay health checks declarados.");
        }

        foreach (var healthCheck in lista)
        {
            if (string.IsNullOrWhiteSpace(healthCheck.Nombre))
            {
                errores.Add("Un health check no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(healthCheck.Tipo))
            {
                errores.Add($"Tipo no permitido en '{healthCheck.Nombre}': '{healthCheck.Tipo}'.");
            }

            if (string.IsNullOrWhiteSpace(healthCheck.Recurso))
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no declara el recurso.");
            }

            if (healthCheck.Tipo == "Liveness" && healthCheck.ConsultaDependenciaExterna)
            {
                errores.Add($"El liveness '{healthCheck.Nombre}' consulta una dependencia externa.");
            }

            if (!healthCheck.TieneTimeout)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' no tiene timeout.");
            }

            if (healthCheck.ExponeInformacionSensible)
            {
                errores.Add($"El health check '{healthCheck.Nombre}' expone información sensible.");
            }
        }

        return new ResultadoHealthChecks(errores.Count == 0, errores);
```

```
    }
}

public record ResultadoHealthChecks(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenHealthChecks(int Total, int Liveness, int Readiness, int Startup);
```

Cuenta las entradas que cumplen el criterio indicado. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 7.2.6 — Verificar el catálogo por defecto

Reemplaza `tests/AceriaApp.HealthChecks.Tests/ValidadorHealthChecksTests.cs` por este archivo completo. Conserva todos los tests obligatorios y añade la comprobación de total 3, liveness 1, readiness 2 y startup 0.

Archivo: `tests/AceriaApp.HealthChecks.Tests/ValidadorHealthChecksTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.HealthChecks;
using FluentAssertions;
using Xunit;

namespace AceriaApp.HealthChecks.Tests;

public class ValidadorHealthChecksTests
{
    private readonly ValidadorHealthChecks _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoHealthChecks.HealthChecksPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinHealthChecks_DevuelveError()
    {
        var resultado = _validador.Validar(new List<HealthCheck>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay health checks"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", Recurso = "A", TieneTimeout = true }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinRecurso_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = string.Empty, TieneTimeout = true }
        };

        var resultado = _validador.Validar(healthChecks);
```

```
        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el recurso"));
    }

    [Fact]
    public void Validar_SinTimeout_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = "A", TieneTimeout = false }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene timeout"));
    }

    [Fact]
    public void Validar_InformacionSensible_DevuelveError()
    {
        var healthChecks = new List<HealthCheck>
        {
            new() { Nombre = "X", Tipo = "Readiness", Recurso = "A", TieneTimeout = true,
ExponeInformacionSensible = true }
        };

        var resultado = _validador.Validar(healthChecks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("expone información sensible"));
    }
    [Fact]
    public void Validar_LivenessConDependenciaExterna_DevuelveError()
    {
        var checks = new List<HealthCheck>
        {
            new() { Nombre = "Self", Tipo = "Liveness", Recurso = "SQL Server", ConsultaDependenciaExterna =
true, TieneTimeout = true }
        };

        var resultado = _validador.Validar(checks);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("consulta una dependencia externa"));
    }

    [Fact]
    public void Validar_ReadinessConDependenciaExterna_DevuelveValido()
    {
        var checks = new List<HealthCheck>
        {
            new() { Nombre = "SQL", Tipo = "Readiness", Recurso = "SQL Server", ConsultaDependenciaExterna =
true, TieneTimeout = true }
        };

        var resultado = _validador.Validar(checks);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorHealthChecks.Resumen(CatalogoHealthChecks.HealthChecksPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.Liveness.Should().Be(1);
        resumen.Readiness.Should().Be(2);
        resumen.Startup.Should().Be(0);
    }
}
```

Calcula el resumen sobre el catálogo por defecto.

Comprobación del reto. Guarda los archivos, compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.HealthChecks.Tests/AceriaApp.HealthChecks.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 13 a 14 tests; la solución debe ejecutar 331 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y compara los hashes de la etapa obligatoria: no debe haber cambiado. Revisa `git diff --check` y el índice antes de registrar el reto en un commit local separado.

Diagnóstico si falla. Si aparece un tipo duplicado, reemplaza el contenido completo en vez de pegarlo al final. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 13 pruebas de catálogo, la implementación funcional y 38 documentos Markdown. La solución incluye 32 proyectos, 20 proyectos de tests y 331 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Continuación. Copia esta etapa obligatoria para abordar 7.3: Gestión de excepciones. Conserva por separado la copia del reto.

Punto 7.3 — Gestión de excepciones

Transformar los errores en respuestas coherentes sin ocultar fallos

Una API necesita diferenciar una entrada inválida, una orden inexistente, un conflicto de dominio y una excepción técnica inesperada. Responder siempre con 500 oculta el contrato funcional; devolver siempre 400 puede esconder fallos del sistema. En el tercer punto integraremos el tratamiento de excepciones con un contrato de error legible y registros que conserven la información diagnóstica adecuada.

El recorrido aprovecha los health checks y logs anteriores y comprueba las respuestas HTTP de casos positivos y negativos. Los catálogos de excepciones no interceptan por sí solos ninguna solicitud; la política efectiva pertenece al middleware y a los manejadores de ASP.NET Core. También verificaremos que la respuesta pública no filtre trazas internas o información sensible.

Carpeta de trabajo: `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar el análisis y el comportamiento ejecutable. La etapa termina con 353 pruebas en 33 proyectos y 21 proyectos de tests.

Ejercicio guiado. Incorpora `ManejadorExcepcion`, `ValidadorGestionExcepciones` y `CatalogoExcepciones`, dos documentos nuevos y 13 pruebas de catálogo. Los bloques funcionales se aplican progresivamente; el paso 16 completa su registro y comprueba la compilación del punto.

PASO 7.3.1 — Crear la carpeta del punto 7.3 a partir del punto 7.2

La etapa autónoma continúa 32 proyectos y 331 pruebas antes de añadir los cambios del punto.

1. Localiza la etapa completa `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones` y copia en ella el contenido de `7.2-health-checks`; en Windows usa `xcopy` con `/E /I /H /Y`.

3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `7.3-gestion-excepciones`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `7.2-health-checks` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `7.2-health-checks`: verifica la ruta antes de editar.

PASO 7.3.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `7.3-gestion-excepciones` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 32 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 32 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 7.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 7.2

El punto conserva 38 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 38 archivos Markdown.
2. Localiza los ADR-010 a ADR-028 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks`.
4. Comprueba que aún no existen `docs/gestion-excepciones.md` ni `docs/ADR-029-gestion-excepciones.md`.

Verificación concreta. Hay 38 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 7.3.4 — Crear el documento de gestión de excepciones en `docs`

La ficha separa el análisis de los contratos y la evidencia ejecutable.

1. Crea `docs/gestion-excepciones.md` dentro de la copia de `7.3-gestion-excepciones`.
2. Introduce el contenido completo del bloque siguiente.

3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/gestion-excepciones.md` existe y que los 38 documentos anteriores permanecen intactos.

Archivo: `docs/gestion-excepciones.md`. **Operación:** Crear.

```
# Gestión de excepciones — Punto 7.3

## Definición

Conjunto de mecanismos que permiten controlar los errores que ocurren durante la ejecución.

## Excepciones esperadas y excepcionales

- Esperadas: patrón Result.
- Excepcionales: excepciones.

## Excepciones personalizadas

Heredan de `Exception` e incluyen información contextual del dominio.

## Manejador global

`UseExceptionHandler` y un `IExceptionHandler` centralizan los fallos inesperados antes de iniciar la
respuesta. Si ya se han enviado cabeceras no pueden reescribirla. Una cancelación solicitada por el cliente
no se traduce en error interno 500.

## Formato problem details

RFC 9457 define `type`, `title`, `status`, `detail` e `instance`; no obliga a incluir detalles internos. El
ejercicio devuelve un título genérico y correlación, sin stack trace, mensajes de excepción ni credenciales.

## Traducción de excepciones a códigos HTTP

| Excepción | Código |
|-----------|--------|
| OrdenNoEncontradaException | 404 |
| ValidacionException | 400 |
| ConflictoException | 409 |
| InventarioNoDisponibleException | 503 |
| No reconocida | 500 |

## Excepciones y logging

Las excepciones se registran con el nivel adecuado antes de devolver la respuesta.

## Excepciones y patrón Result

Las excepciones esperadas se representan con Result. Las excepcionales se capturan en el manejador global.

## Buenas prácticas

1. No usar excepciones para el flujo normal.
2. Capturar solo las excepciones manejables.
3. No capturar excepciones genéricas sin propósito.
4. Registrar las excepciones con contexto.
5. Devolver respuestas uniformes.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Manejador global | UseExceptionHandler |
| Formato | Problem details |
| Excepciones esperadas | Result |
| Excepciones excepcionales | Manejador global |
| Registro | Serilog |
| Validación | Test de coherencia |
## Contrato del ejercicio

Los nombres de excepción del catálogo son ejemplos de clasificación. No obligan al dominio a lanzar
excepciones para errores esperados: las rutas existentes conservan Result y sus 400/404/409/422, y la
indisponibilidad sigue siendo 503. No todo 4xx merece un log de nivel Error; se decide según su naturaleza.
La regla `RegistraError` del catálogo se exige a fallos 5xx, sin registrar contenido libre del fallo.
```

> El endpoint de diagnóstico requiere entorno Development y una opción explícita. No existe en Production. Las pruebas diferencian el formato del error, la correlación, la cancelación, las respuestas ya iniciadas y la preservación de los estados de negocio.

Verificación concreta. `docs/gestion-excepciones.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/gestion-excepciones.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.3.5 — Crear la carpeta `GestionExcepciones` en el proyecto API

`GestionExcepciones` agrupa datos y reglas del ejercicio de análisis; los adaptadores de comunicación tienen sus propias responsabilidades.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `GestionExcepciones` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/GestionExcepciones` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `7.2-health-checks`.

Verificación concreta. Existe `src/AceriaApp.Api/GestionExcepciones` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

PASO 7.3.6 — Crear la clase `ManejadorExcepcion`

ManejadorExcepcion representa los criterios del ejercicio y su respuesta prevista. Para aplicarlos realmente necesitamos que el middleware capture el fallo y genere un ProblemDetails sin revelar detalles internos.

1. Crea `src/AceriaApp.Api/GestionExcepciones/ManejadorExcepcion.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/GestionExcepciones/ManejadorExcepcion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.GestionExcepciones;

public class ManejadorExcepcion
{
    public string TipoExcepcion { get; init; } = string.Empty;
    public int CodigoHttp { get; init; }
    public string Titulo { get; init; } = string.Empty;
    public bool UsaProblemDetails { get; init; }
    public bool RegistraError { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `ManejadorExcepcion` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.GestionExcepciones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.3.7 — Crear la clase `ValidadorGestionExcepciones`

El validador del catálogo examina casos descritos, pero no intercepta una excepción lanzada por ASP.NET Core. La evidencia funcional procede de provocar errores controlados y consultar la respuesta HTTP.

1. Crea `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.GestionExcepciones;

public class ValidadorGestionExcepciones
{
    private static readonly int[] CodigosPermitidos = { 400, 401, 403, 404, 409, 422, 500, 503 };

    public ResultadoGestionExcepciones Validar(IEnumerable<ManejadorExcepcion> manejadores)
    {
        var errores = new List<string>();
        var lista = manejadores.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay manejadores declarados.");
        }

        foreach (var manejador in lista)
        {
            if (string.IsNullOrWhiteSpace(manejador.TipoExcepcion))
            {
                errores.Add("Un manejador no declara el tipo de excepción.");
            }

            if (!CodigosPermitidos.Contains(manejador.CodigoHttp))
            {
                errores.Add($"Código HTTP no permitido en '{manejador.TipoExcepcion}':
'{manejador.CodigoHttp}'.");
            }

            if (string.IsNullOrWhiteSpace(manejador.Titulo))
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no declara título.");
            }

            if (!manejador.UsaProblemDetails)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no usa problem details.");
            }

            if (manejador.CodigoHttp >= 500 && !manejador.RegistraError)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no registra el error.");
            }
        }
```

```
        return new ResultadoGestionExcepciones(errores.Count == 0, errores);
    }
}

public record ResultadoGestionExcepciones(bool EsValida, IReadOnlyList<string> Errores);
```

Completa la declaración o la operación mostrada en el bloque. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Devuelve validez y diagnósticos; no instala middleware ni ejecuta llamadas remotas. Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorGestionExcepciones` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.GestionExcepciones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.3.8 — Añadir los `using` necesarios en `ValidadorGestionExcepciones`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.GestionExcepciones;

public class ValidadorGestionExcepciones
{
    private static readonly int[] CodigosPermitidos = { 400, 401, 403, 404, 409, 422, 500, 503 };

    public ResultadoGestionExcepciones Validar(IEnumerable<ManejadorExcepcion> manejadores)
    {
        var errores = new List<string>();
        var lista = manejadores.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay manejadores declarados.");
        }

        foreach (var manejador in lista)
        {
            if (string.IsNullOrWhiteSpace(manejador.TipoExcepcion))
            {
                errores.Add("Un manejador no declara el tipo de excepción.");
```

```
            }

            if (!CodigosPermitidos.Contains(manejador.CodigoHttp))
            {
                errores.Add($"Código HTTP no permitido en '{manejador.TipoExcepcion}':
'{manejador.CodigoHttp}'.");
            }

            if (string.IsNullOrWhiteSpace(manejador.Titulo))
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no declara título.");
            }

            if (!manejador.UsaProblemDetails)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no usa problem details.");
            }

            if (manejador.CodigoHttp >= 500 && !manejador.RegistraError)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no registra el error.");
            }
        }

        return new ResultadoGestionExcepciones(errores.Count == 0, errores);
    }
}

public record ResultadoGestionExcepciones(bool EsValida, IReadOnlyList<string> Errores);
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorGestionExcepciones.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Integración funcional — `src/AceriaApp.Observabilidad/ManejadorExcepciones.cs`.

El manejador trata excepciones inesperadas, no sustituye los Result del dominio. Si la respuesta ya empezó no intenta escribir otra. En el caso general registra solo el tipo de excepción y devuelve ProblemDetails con un título seguro, traceId y correlationId; el fallback mantiene JSON aunque Accept no permita al escritor predeterminado actuar.

Archivo: `src/AceriaApp.Observabilidad/ManejadorExcepciones.cs`. **Operación:** Crear.

```
using System.Text.Json;
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
namespace AceriaApp.Observabilidad;
public sealed class ManejadorExcepciones(IProblemDetailsService detalles, ILogger<ManejadorExcepciones>
logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception exception, CancellationToken
ct)
    {
        if (context.Response.HasStarted) return false;
        if (exception is OperationCanceledException && context.RequestAborted.IsCancellationRequested)
        {
            context.Response.StatusCode = 499;
            return true;
        }
        logger.LogError(new EventId(7301, "ExcepcionInesperada"), "Fallo inesperado de tipo {TipoExcepcion}",
exception.GetType().Name);
```

```
        var problem = new ProblemDetails { Status = 500, Title = "Se ha producido un error interno.", Type =
"about:blank" };
        RegistroErrores.Correlacionar(context, problem);
        context.Response.StatusCode = 500;
        if (!await detalles.TryWriteAsync(new ProblemDetailsContext { HttpContext = context, ProblemDetails =
problem }))
            await context.Response.WriteAsJsonAsync(problem, (JsonSerializerOptions?)null, "application/
problem+json", ct);
        return true;
    }
}
```

Evita reemplazar una respuesta cuyas cabeceras ya se enviaron. Distingue una cancelación del cliente de un fallo de la dependencia. Registra el fallo sin adjuntar el mensaje libre de la excepción.

Intenta escribir ProblemDetails mediante el servicio de ASP.NET.

Integración funcional — `src/AceriaApp.Observabilidad/RegistroErrores.cs`.

El registro separa servicios de middleware. El endpoint que provoca un fallo existe únicamente si el entorno es Development y el indicador de diagnóstico está activo. Al ejecutarse dentro del pipeline comparte correlación y tratamiento de excepciones con las rutas ordinarias.

Archivo: `src/AceriaApp.Observabilidad/RegistroErrores.cs`. **Operación:** Crear.

```
using System.Diagnostics;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
namespace AceriaApp.Observabilidad;
public static class RegistroErrores
{
    public static IServiceCollection AddErroresAceria(this IServiceCollection services)
    {
        services.AddProblemDetails(options => options.CustomizeProblemDetails = c =>
Correlacionar(c.HttpContext, c.ProblemDetails));
        services.AddExceptionHandler<ManejadorExcepciones>();
        return services;
    }
    internal static void Correlacionar(HttpContext context, ProblemDetails problem)
    {
        problem.Extensions["traceId"] = Activity.Current?.Id ?? context.TraceIdentifier;
        problem.Extensions["correlationId"] = ContextoCorrelacion.Obtener(context);
    }
    public static IApplicationBuilder UseErroresAceria(this IApplicationBuilder app) =>
app.UseExceptionHandler();
    public static WebApplication MapDiagnosticoAceria(this WebApplication app)
    {
        if (app.Environment.IsDevelopment() &&
app.Configuration.GetValue<bool>("Observabilidad:DiagnosticoActivo"))
            app.MapGet("/api/diagnostico/fallo", IResult () => throw new InvalidOperationException("Detalle
interno del fallo controlado."));
        return app;
    }
}
```

Añade ProblemDetails y el manejador de excepciones al contenedor. Relaciona el evento con la traza técnica de la petición. Sitúa el manejador dentro de los ámbitos de correlación y logging.

Aplica las dos condiciones de habilitación del diagnóstico.

PASO 7.3.9 — Crear la clase `CatalogoExcepciones`

Cada invocación devuelve una lista nueva con los ejemplos de análisis; cambiar una lista no modifica futuros catálogos.

1. Crea `src/AceriaApp.Api/GestionExcepciones/CatalogoExcepciones.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/GestionExcepciones/CatalogoExcepciones.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.GestionExcepciones;

public class CatalogoExcepciones
{
    public static List<ManejadorExcepcion> ManejadoresPorDefecto()
    {
        return new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "OrdenNoEncontradaException", CodigoHttp = 404, Titulo = "Recurso no
encontrado", UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "ValidacionException", CodigoHttp = 400, Titulo = "Error de validación",
UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "ConflictoException", CodigoHttp = 409, Titulo = "Conflicto de estado",
UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "RecursoNoProcesableException", CodigoHttp = 422, Titulo = "Entidad no
procesable", UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "Exception", CodigoHttp = 500, Titulo = "Error interno",
UsaProblemDetails = true, RegistraError = true }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoExcepciones` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.GestionExcepciones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.3.10 — Añadir el `using` necesario en `CatalogoExcepciones`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/GestionExcepciones/CatalogoExcepciones.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/GestionExcepciones/CatalogoExcepciones.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;

namespace AceriaApp.Api.GestionExcepciones;

public class CatalogoExcepciones
{
    public static List<ManejadorExcepcion> ManejadoresPorDefecto()
    {
        return new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "OrdenNoEncontradaException", CodigoHttp = 404, Titulo = "Recurso no
encontrado", UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "ValidacionException", CodigoHttp = 400, Titulo = "Error de validación",
UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "ConflictoException", CodigoHttp = 409, Titulo = "Conflicto de estado",
UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "RecursoNoProcesableException", CodigoHttp = 422, Titulo = "Entidad no
procesable", UsaProblemDetails = true, RegistraError = true },
            new() { TipoExcepcion = "Exception", CodigoHttp = 500, Titulo = "Error interno",
UsaProblemDetails = true, RegistraError = true }
        };
    }
}
```

Verificación concreta. `CatalogoExcepciones.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.3.11 — Crear el proyecto de tests del punto 7.3

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.GestionExcepciones.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 14 y 15 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj` y `dotnet restore tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
Operación: Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 7.3.12 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj` y `dotnet build tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

PASO 7.3.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj` y `dotnet list tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj package`.

4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

Integración funcional — `src/AceriaApp.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM073;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM073;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM073;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m073",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

Agrupa las conexiones de laboratorio específicas de la etapa. Activa o desactiva los procesos de mensajería y su dependencia en readiness. Fija el umbral propio de logging y admite sobreescritura por variables de entorno.

Mantiene el endpoint de fallo desactivado por defecto.

Integración funcional — `src/AceriaApp.Inventario.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Inventario.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM073;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m073",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

PASO 7.3.14 — Crear el archivo `ValidadorGestionExcepcionesTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.GestionExcepciones.Tests/ValidadorGestionExcepcionesTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/ValidadorGestionExcepcionesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.GestionExcepciones;
using FluentAssertions;
using Xunit;

namespace AceriaApp.GestionExcepciones.Tests;

public class ValidadorGestionExcepcionesTests
{
    private readonly ValidadorGestionExcepciones _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoExcepciones.ManejadoresPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinManejadores_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ManejadorExcepcion>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay manejadores"));
    }

    [Fact]
    public void Validar_CodigoNoPermitido_DevuelveError()
    {
```

```
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 999, Titulo = "T", UsaProblemDetails = true,
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Código HTTP no permitido"));
    }

    [Fact]
    public void Validar_SinTitulo_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 404, Titulo = string.Empty, UsaProblemDetails = true,
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara título"));
    }

    [Fact]
    public void Validar_SinProblemDetails_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 404, Titulo = "T", UsaProblemDetails = false,
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa problem details"));
    }

    [Fact]
    public void Validar_ErrorServidorSinRegistro_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 500, Titulo = "T", UsaProblemDetails = true,
RegistraError = false }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no registra el error"));
    }
    [Fact]
    public void Validar_Indisponibilidad503_DevuelveValido()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "InventarioNoDisponibleException", CodigoHttp = 503, Titulo = "Servicio
no disponible", UsaProblemDetails = true, RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_ErrorClienteSinNivelError_DevuelveValido()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
```

```
            new() { TipoExcepcion = "ValidacionException", CodigoHttp = 400, Titulo = "Validación",
UsaProblemDetails = true, RegistraError = false }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorGestionExcepcionesTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 7.3.15 — Crear el archivo `CatalogoExcepcionesTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.GestionExcepciones.Tests/CatalogoExcepcionesTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/CatalogoExcepcionesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.GestionExcepciones;
using FluentAssertions;
using Xunit;

namespace AceriaApp.GestionExcepciones.Tests;

public class CatalogoExcepcionesTests
{
    [Fact]
    public void ManejadoresPorDefecto_ContieneCincoManejadores()
    {
        var manejadores = CatalogoExcepciones.ManejadoresPorDefecto();

        manejadores.Should().HaveCount(5);
    }

    [Fact]
    public void ManejadoresPorDefecto_TodosUsanProblemDetails()
    {
        var manejadores = CatalogoExcepciones.ManejadoresPorDefecto();

        manejadores.Should().OnlyContain(m => m.UsaProblemDetails);
    }
```

```
    [Fact]
    public void ManejadoresPorDefecto_TodosRegistranError()
    {
        var manejadores = CatalogoExcepciones.ManejadoresPorDefecto();

        manejadores.Should().OnlyContain(m => m.RegistraError);
    }

    [Fact]
    public void ManejadoresPorDefecto_ContieneCuatroErroresDelClienteYUnErrorDelServidor()
    {
        var manejadores = CatalogoExcepciones.ManejadoresPorDefecto();

        manejadores.Count(m => m.CodigoHttp >= 400 && m.CodigoHttp < 500).Should().Be(4);
        manejadores.Count(m => m.CodigoHttp >= 500).Should().Be(1);
    }

    [Fact]
    public void ManejadoresPorDefecto_ContieneUnError500()
    {
        var manejadores = CatalogoExcepciones.ManejadoresPorDefecto();

        manejadores.Count(m => m.CodigoHttp == 500).Should().Be(1);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Comprueba la condición en cada entrada declarada del catálogo.

Compara el recuento obtenido con el valor esperado del catálogo.

Verificación concreta. `CatalogoExcepcionesTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Integración funcional — `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj`.
Operación: Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  <PackageReference Include="Microsoft.AspNetCore.TestHost" Version="8.0.21" />
  </ItemGroup>
</Project>
```

Conecta este proyecto con otro proyecto real de la solución. Fija la dependencia usada por esta etapa del ejercicio.

Integración funcional — `tests/AceriaApp.GestionExcepciones.Tests/ErroresFuncionalesTests.cs`.

TestServer ejecuta el pipeline ASP.NET real: comprueba contenido seguro, negociación, códigos esperados del dominio, diagnóstico por entorno, cancelación y respuesta iniciada. Se inspeccionan los eventos serializados para descartar el mensaje libre de la excepción.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/ErroresFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Collections.Concurrent;
using System.Net;
using System.Text.Json;
using AceriaApp.Observabilidad;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Http.Features;
using Microsoft.AspNetCore.TestHost;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Serilog.Core;
using Serilog.Events;
using Serilog.Formatting.Json;
using Xunit;
namespace AceriaApp.GestionExcepciones.Tests;
public class ErroresFuncionalesTests
{
    private sealed class Captura : ILogEventSink
    {
        public ConcurrentQueue<LogEvent> Eventos { get; } = new();
        public void Emit(LogEvent e) => Eventos.Enqueue(e);
    }
    private static async Task<WebApplication> Servidor(string entorno, bool diagnostico, Captura? sink =
null)
    {
        var builder = WebApplication.CreateBuilder(new WebApplicationOptions { EnvironmentName = entorno });
        builder.WebHost.UseTestServer();
        builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
{ ["Observabilidad:DiagnosticoActivo"] = diagnostico.ToString() });
        if (sink is not null) builder.Services.AddSingleton<ILogEventSink>(sink);
        builder.Services.AddLoggingAceria("Prueba"); builder.Services.AddErroresAceria();
        var app = builder.Build(); app.UseLoggingAceria(); app.UseErroresAceria();
app.MapDiagnosticoAceria();
        app.MapGet("/inesperado", IResult () => throw new InvalidOperationException("secreto-excepcion-no-
publicar"));
        app.MapGet("/inesperado/{secreto}", IResult () => throw new InvalidOperationException("secreto-
excepcion-no-publicar"));
        app.MapGet("/esperado", () => Results.Problem(statusCode: 409, title: "La orden ya está
planificada."));
        await app.StartAsync(); return app;
    }
    [Fact]
    public async Task Fallo_inesperado_devuelve_ProblemDetails_y_correlacion_sin_detalle_privado()
    {
        await using var app = await Servidor("Production", false); using var http = app.GetTestClient();
        http.DefaultRequestHeaders.Add("X-Correlation-ID", "error-123");
        using var response = await http.GetAsync("/inesperado/secreto-ruta?token=secreto-query");
        var text = await response.Content.ReadAsStringAsync(); using var doc = JsonDocument.Parse(text);
        Assert.Equal(HttpStatusCode.InternalServerError, response.StatusCode);
        Assert.Equal("application/problem+json", response.Content.Headers.ContentType!.MediaType);
        Assert.Equal(500, doc.RootElement.GetProperty("status").GetInt32());
        Assert.Equal("error-123", doc.RootElement.GetProperty("correlationId").GetString());
        Assert.Equal("error-123", Assert.Single(response.Headers.GetValues("X-Correlation-ID")));
        Assert.False(string.IsNullOrWhiteSpace(doc.RootElement.GetProperty("traceId").GetString()));
        Assert.DoesNotContain("secreto", text); Assert.DoesNotContain("InvalidOperationException", text);
    }
    [Fact]
    public async Task Accept_no_compatible_con_writer_sigue_recibiendo_error_JSON_seguro()
    {
        await using var app = await Servidor("Production", false); using var http = app.GetTestClient();
        http.DefaultRequestHeaders.Accept.ParseAdd("text/plain");
        using var response = await http.GetAsync("/inesperado");
        Assert.Equal(HttpStatusCode.InternalServerError, response.StatusCode);
        using var doc = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        Assert.Equal(500, doc.RootElement.GetProperty("status").GetInt32());
    }
    [Fact]
```

```
    public async Task Result_esperado_conserva_su_409()
    {
        await using var app = await Servidor("Production", false); using var http = app.GetTestClient();
        using var response = await http.GetAsync("/esperado");
        Assert.Equal(HttpStatusCode.Conflict, response.StatusCode);
        using var doc = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        Assert.Equal("La orden ya está planificada.", doc.RootElement.GetProperty("title").GetString());
    }
    [Theory]
    [InlineData("Production", true, 404)]
    [InlineData("Development", false, 404)]
    [InlineData("Development", true, 500)]
    public async Task Diagnostico_solo_se_habilita_en_Development_con_opcion(string entorno, bool activo, int
status)
    {
        await using var app = await Servidor(entorno, activo); using var http = app.GetTestClient();
        using var response = await http.GetAsync("/api/diagnostico/fallo");
        Assert.Equal(status, (int)response.StatusCode);
    }
    [Fact]
    public async Task Excepcion_registrada_no_incluye_mensaje_libre_ni_cuerpo()
    {
        var sink = new Captura(); await using var app = await Servidor("Production", false, sink); using var
http = app.GetTestClient();
        using var response = await http.GetAsync("/inesperado/secreto-ruta?token=secreto-query");
        Assert.Equal(HttpStatusCode.InternalServerError, response.StatusCode);
        var error = Assert.Single(sink.Eventos, e => e.Properties.ContainsKey("TipoExcepcion"));
        Assert.Equal(LogEventLevel.Error, error.Level); Assert.Null(error.Exception);
        using var output = new StringWriter(); foreach (var e in sink.Eventos) new JsonFormatter().Format(e,
output);
        Assert.DoesNotContain("secreto-excepcion", output.ToString());
        Assert.DoesNotContain("secreto-ruta", output.ToString());
        Assert.DoesNotContain("secreto-query", output.ToString());
        Assert.DoesNotContain("RequestPath", output.ToString());
    }
    private static ServiceProvider Servicios()
    {
        var services = new ServiceCollection(); services.AddLoggingAceria("Prueba");
services.AddErroresAceria();
        services.AddSingleton<ManejadorExcepciones>(); return services.BuildServiceProvider();
    }
    [Fact]
    public async Task Cancelacion_del_usuario_no_se_convierte_en_500()
    {
        using var services = Servicios(); using var cancelacion = new CancellationTokenSource();
cancelacion.Cancel();
        var ctx = new DefaultHttpContext { RequestServices = services, RequestAborted = cancelacion.Token };
        ctx.Response.Body = new MemoryStream();
        Assert.True(await services.GetRequiredService<ManejadorExcepciones>().TryHandleAsync(ctx, new
OperationCanceledException(cancelacion.Token), cancelacion.Token));
        Assert.Equal(499, ctx.Response.StatusCode); Assert.Equal(0, ctx.Response.Body.Length);
    }
    private sealed class RespuestaIniciada : HttpResponseFeature { public override bool HasStarted => true; }
    [Fact]
    public async Task Respuesta_iniciada_no_se_reescribe()
    {
        using var services = Servicios(); var ctx = new DefaultHttpContext { RequestServices = services };
        ctx.Features.Set<IHttpResponseFeature>(new RespuestaIniciada()); ctx.Response.Body = new
MemoryStream();
        Assert.False(await services.GetRequiredService<ManejadorExcepciones>().TryHandleAsync(ctx, new
Exception("interno"), default));
        Assert.Equal(0, ctx.Response.Body.Length);
    }
}
```

Define una comprobación síncrona del contrato indicado por el nombre. Registra el logger JSON y los servicios que necesita la correlación. Abre la correlación y el resumen antes de ejecutar el resto del pipeline.

Define un escenario asíncrono cuyo resultado verifica la conducta descrita por el nombre.

PASO 7.3.16 — Compilar la solución completa

La integración necesita conservar los errores de dominio y registrar los fallos inesperados. Comprueba que las respuestas no se convierten indiscriminadamente en 500 ni exponen excepciones internas.

1. Aplica los bloques de registro final de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 33 proyectos y termina sin errores.

Registro de biblioteca compartida. Ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`; los proyectos de tests se incorporan en sus pasos de creación. Después de reemplazar todos los archivos de este paso, compila la solución.

Verificación concreta. La solución de la etapa compila con 33 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

Integración funcional — `src/AceriaApp.Api/Program.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Api.Comunicacion;
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddLoggingAceria("Produccion", builder.Configuration);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
builder.Services.AddComunicacion(builder.Configuration);
builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>("inventario-instrumentado", http =>
{
    http.BaseAddress = new Uri(builder.Configuration["Comunicacion:InventarioUrl"] ?? "http://
localhost:5106/");
```

```
    http.Timeout = TimeSpan.FromSeconds(3);
}).AddHttpMessageHandler<PropagarCorrelacionHandler>();
builder.Services.AddSaludAceria(builder.Configuration, inventario: false);
builder.Services.AddErroresAceria();
var app=builder.Build();
app.UseLoggingAceria();
app.UseErroresAceria();
app.MapDiagnosticoAceria();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
```

```
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
```

```
app.MapProduccion();
app.MapComunicacion();
app.MapSaludAceria();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Propaga el identificador validado hacia Inventario. Registra las dependencias propias de este servicio para readiness. Publica liveness y readiness como rutas independientes.

Integración funcional — `src/AceriaApp.Inventario.Api/InventarioProgram.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddLoggingAceria("Inventario", builder.Configuration);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        builder.Services.AddSaludAceria(builder.Configuration, inventario: true);
        builder.Services.AddErroresAceria();
        var app = builder.Build();
        app.UseLoggingAceria();
        app.UseErroresAceria();
        app.MapDiagnosticoAceria();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapGet("/api/v1/materiales/{material}", async (string material, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapGet("/api/v1/planificaciones/{ordenId:guid}", async (Guid ordenId, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
```

```
valor.CantidadTotal });
        });
        app.MapSaludAceria();
        await app.RunAsync();
    }
}
```

Recorrido funcional 7.3 — ProblemDetails seguro y diagnóstico restringido

Abre A, B y C en `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones`, con PowerShell 7, .NET 8 y LocalDB. Esta etapa usa bases con sufijo `M073`. Los workers estarán desactivados; RabbitMQ no es necesario. Los dos puertos de las API deben estar libres.

En C:

```
sqllocaldb start MSSQLLocalDB
if ($LASTEXITCODE -ne 0) { throw 'No se pudo iniciar MSSQLLocalDB.' }
dotnet restore .\AceriaApp.sln
if ($LASTEXITCODE -ne 0) { throw 'Falló la restauración.' }
dotnet build .\AceriaApp.sln -c Release --no-restore
if ($LASTEXITCODE -ne 0) { throw 'Falló la compilación.' }
```

En B, inicia Inventario en Development con diagnóstico habilitado:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__DiagnosticoActivo = 'true'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106 2>&1 |
    Tee-Object -FilePath .\inventario-m073.jsonl
```

En A, inicia Producción con los mismos ajustes:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Comunicacion__InventarioUrl = 'http://localhost:5106/'
$env:Observabilidad__DiagnosticoActivo = 'true'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105 2>&1 |
    Tee-Object -FilePath .\produccion-m073.jsonl
```

En C, provoca el fallo controlado de ambos servicios:

```
foreach ($puerto in 5105, 5106) {
    $correlacion = 'm073-' + $puerto + '-' + [guid]::NewGuid().ToString('N')
    $r = Invoke-WebRequest "http://localhost:$puerto/api/diagnostico/fallo" `
        -Headers @{ 'X-Correlation-ID' = $correlacion; Accept = 'application/problem+json' }
-SkipHttpErrorCheck
    $r.StatusCode
    $r.Headers['Content-Type']
    $r.Content
    $problema = $r.Content | ConvertFrom-Json
    if ($r.StatusCode -ne 500 -or $problema.status -ne 500) { throw 'No se recibió ProblemDetails 500.' }
    if ($problema.title -ne 'Se ha producido un error interno.') { throw 'El título público no es el
genérico.' }
    if ($problema.correlationId -ne $correlacion -or -not $problema.traceId) { throw 'Falta contexto de
correlación.' }
    if ($r.Content -match 'Detalle interno|InvalidOperationException|StackTrace') { throw 'Se expusieron
detalles internos.' }
```

```
}
Get-Content .\produccion-m073.jsonl, .\inventario-m073.jsonl |
    ForEach-Object {
        try { $_ | ConvertFrom-Json -ErrorAction Stop } catch { }
    } |
    Where-Object { $_.Properties.EventId.Id -eq 7301 } |
    Select-Object Timestamp, Level, MessageTemplate, Properties | Format-List
```

El evento `7301` informa del tipo de excepción y conserva el contexto de la petición; no registra su mensaje libre ni una traza completa. La respuesta pública conserva `correlationId` y `traceId`, sin detalles de implementación. Los errores esperados no se convierten en 500: por ejemplo, un material inexistente conserva su 404:

```
$ausente = 'NO-EXISTE-' + [guid]::NewGuid().ToString('N')
$r404 = Invoke-WebRequest "http://localhost:5105/api/v1/comunicacion/materiales/$ausente" -SkipHttpErrorCheck
$r404.StatusCode
if ($r404.StatusCode -ne 404) { throw 'La ausencia de material dejó de ser 404.' }
```

Diagnóstico ausente en Production, incluso con el flag activo. Detén A y B con `Ctrl+C`. No borres las bases creadas en Development. En A ejecuta:

```
$env:DOTNET_ENVIRONMENT = 'Production'
$env:ASPNETCORE_ENVIRONMENT = 'Production'
$env:Observabilidad__DiagnosticoActivo = 'true'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105
```

En B ejecuta:

```
$env:DOTNET_ENVIRONMENT = 'Production'
$env:ASPNETCORE_ENVIRONMENT = 'Production'
$env:Observabilidad__DiagnosticoActivo = 'true'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106
```

En C comprueba tanto que los procesos están vivos como que la ruta de diagnóstico no existe:

```
foreach ($puerto in 5105, 5106) {
    $live = Invoke-WebRequest "http://localhost:$puerto/health/live" -SkipHttpErrorCheck
    $diagnostico = Invoke-WebRequest "http://localhost:$puerto/api/diagnostico/fallo" -SkipHttpErrorCheck
    [pscustomobject]@{ Puerto = $puerto; Live = $live.StatusCode; Diagnostico = $diagnostico.StatusCode }
    if ($live.StatusCode -ne 200 -or $diagnostico.StatusCode -ne 404) { throw 'El diagnóstico no quedó
restringido al entorno.' }
}
dotnet test .\tests\AceriaApp.GestionExcepciones.Tests\AceriaApp.GestionExcepciones.Tests.csproj -c Release
--no-build --no-restore `
    --filter 'FullyQualifiedName~ErroresFuncionalesTests'
```

El nivel `Information` de esta sesión se fija expresamente para observar también respuestas correctas; `appsettings.Production.json` establece `Warning` de forma predeterminada. El flag por sí solo nunca habilita la ruta: se exige también `Development`. La suite añade casos controlados que no conviene provocar de forma ambigua con un navegador: cancelación del llamador sin 500, respuesta ya iniciada que no se reescribe, `Accept` no compatible y conservación de errores esperados. Los dos procesos de este recorrido sí son las API reales. Al terminar, detén A y B y cierra esas terminales para no heredar el diagnóstico activo.

PASO 7.3.17 — Ejecutar todos los tests del punto 7.3

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 13 tests de catálogo y las pruebas funcionales incorporadas progresivamente al proyecto.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 13 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 7.3.18 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 353 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04, M05 y M06: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 353 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

PASO 7.3.19 — Actualizar el README del punto 7.3

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `7.3-gestion-excepciones`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `7.2-health-checks` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 7.3 — Gestión de excepciones

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base anterior.
```

```
## Estado del proyecto

- Solución `AceriaApp.sln`: 33 proyectos, 21 proyectos de tests y 353 pruebas acumuladas.
- 13 pruebas nuevas de análisis en `tests/AceriaApp.GestionExcepciones.Tests`.
- Tipos `ManejadorExcepcion`, `ValidadorGestionExcepciones` y `CatalogoExcepciones` en `src/AceriaApp.Api/
GestionExcepciones`.
- Los catálogos no configuran logging, probes, manejadores ni políticas HTTP.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M07 para iniciar los adaptadores y observar logs, salud, errores y recuperación HTTP.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/gestion-excepciones.md` y el ADR-029. Los bloques
funcionales se aplican progresivamente y el paso 16 completa el registro y la compilación del punto. Se
conservan los documentos y pruebas heredados.

## Documentación

- [Tema del punto](docs/gestion-excepciones.md).
- [ADR-029](docs/ADR-029-gestion-excepciones.md).
- Los 38 documentos anteriores, incluidos los ADR-010 a ADR-028, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 33 proyectos, 353 pruebas y 13 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 7.3.20 — Crear el ADR-029 del punto 7.3

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-029-gestion-excepciones.md` dentro de la copia de `7.3-gestion-excepciones`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-029-gestion-excepciones.md` existe y que los 38 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-029-gestion-excepciones.md`. **Operación:** Crear.

```
# ADR-029: Gestión de excepciones

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita observar y limitar fallos distribuidos. Se conservan los
contratos del dominio, SQL Server, RabbitMQ y outbox/inbox de M6. Los catálogos de análisis y la
implementación funcional aportan evidencias diferentes y complementarias.

## Decisión

Centralizar fallos inesperados mediante IExceptionHandler, UseExceptionHandler y ProblemDetails de ASP.NET
Core 8 conforme a RFC 9457. Conservar Result y los estados 400/404/409/422/503. No transformar cancelación en
500 ni reescribir una respuesta iniciada. El diagnóstico controlado solo existe en Development con una opción
activada.
```

```
## Alternativas consideradas

- Mantener solo los catálogos: descartado porque los indicadores no prueban el comportamiento del proceso.
- Duplicar los componentes en Producción e Inventario: descartado; la biblioteca AceriaApp.Observabilidad
comparte la integración ASP.NET sin acoplar el dominio.
- Introducir servicios de observabilidad externos: fuera del alcance del módulo; la consola JSON permite
inspeccionar resultados reales.

## Consecuencias

- Positivas: comportamiento uniforme, contratos explícitos y fallos reproducibles.
- Negativas: deben mantenerse límites, configuración, políticas y pruebas de integración.
- Mitigación: conservar pruebas heredadas, validar errores y cancelación, y separar señales operativas de los
datos sensibles.
```

Verificación concreta. `docs/ADR-029-gestion-excepciones.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-029-gestion-excepciones.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.3.21 — Verificar que no se ha modificado ningún archivo del punto 7.2

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.2-health-checks` y actual `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 38 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 38 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 7.3.22 — Verificar la estructura del punto 7.3

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, con todos los proyectos dentro de la etapa.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/GestionExcepciones` y las dos clases de tests en `tests/AceriaApp.GestionExcepciones.Tests`.
3. Comprueba los 40 documentos de `docs`, incluida la ficha del punto y el ADR-029.
4. Ejecuta `dotnet sln AceriaApp.sln list` y contrasta sus 33 proyectos con los `.csproj` presentes.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 33 proyectos.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 7.3.23 — Verificar el `.gitignore` del punto 7.3

Las exclusiones se aplican desde la raíz a las etapas; no es necesario inventar un `.gitignore` propio dentro de cada copia.

1. Abre el `.gitignore` de la raíz del repositorio del curso.
2. Comprueba las reglas `**/bin/`, `**/obj/` y `**/TestResults/` que ya excluyen los artefactos de todas las etapas.
3. Ejecuta `git status --short` y revisa que no aparezcan binarios, cachés ni resultados temporales.
4. Conserva las exclusiones existentes; si aparecen archivos locales de Visual Studio, excluye `.vs/`, `*.user` y `*.suo` mediante `.git/info/exclude`, sin ignorar código ni configuración entregable.

Verificación concreta. Los archivos compilados y resultados de tests están excluidos; los archivos del punto siguen visibles para Git.

Diagnóstico si falla.

- Ignorar todo `tests/` elimina la evidencia reproducible: excluye solo sus artefactos generados.
- Esperar un `.gitignore` dentro de cada etapa puede llevar a crear duplicados innecesarios: revisa primero el de la raíz.

PASO 7.3.24 — Hacer el commit del punto 7.3 y verificar el estado final

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 7.3: gestión de excepciones`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 7.3 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorGestionExcepciones` y una prueba adicional. Trabaja en una copia optativa de `7.3-gestion-excepciones` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 7.3.1 — Definir el contrato del resumen

Reemplaza `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs` por el archivo completo siguiente, que conserva las reglas y añade el contrato de resumen en el mismo namespace.

Archivo: `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.GestionExcepciones;

public class ValidadorGestionExcepciones
{
    private static readonly int[] CodigosPermitidos = { 400, 401, 403, 404, 409, 422, 500, 503 };

    public ResultadoGestionExcepciones Validar(IEnumerable<ManejadorExcepcion> manejadores)
    {
        var errores = new List<string>();
        var lista = manejadores.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay manejadores declarados.");
        }

        foreach (var manejador in lista)
        {
            if (string.IsNullOrWhiteSpace(manejador.TipoExcepcion))
            {
                errores.Add("Un manejador no declara el tipo de excepción.");
            }

            if (!CodigosPermitidos.Contains(manejador.CodigoHttp))
            {
                errores.Add($"Código HTTP no permitido en '{manejador.TipoExcepcion}':
'{manejador.CodigoHttp}'.");
            }

            if (string.IsNullOrWhiteSpace(manejador.Titulo))
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no declara título.");
            }

            if (!manejador.UsaProblemDetails)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no usa problem details.");
            }

            if (manejador.CodigoHttp >= 500 && !manejador.RegistraError)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no registra el error.");
            }
        }

        return new ResultadoGestionExcepciones(errores.Count == 0, errores);
    }
}

public record ResultadoGestionExcepciones(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenGestionExcepciones(int Total, int ErroresCliente, int ErroresServidor, int
ConProblemDetails);
```

Reto 7.3.3 — Calcular los recuentos

Reemplaza de nuevo `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs` por el archivo completo siguiente. El método queda dentro del validador y materializa la secuencia una sola vez.

Archivo: `src/AceriaApp.Api/GestionExcepciones/ValidadorGestionExcepciones.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.GestionExcepciones;

public class ValidadorGestionExcepciones
{
    public static ResumenGestionExcepciones Resumen(IEnumerable<ManejadorExcepcion> manejadores)
    {
        var lista = manejadores.ToList();
        var erroresCliente = lista.Count(m => m.CodigoHttp >= 400 && m.CodigoHttp < 500);
        var erroresServidor = lista.Count(m => m.CodigoHttp >= 500);
        var conProblemDetails = lista.Count(m => m.UsaProblemDetails);
        return new ResumenGestionExcepciones(lista.Count, erroresCliente, erroresServidor,
conProblemDetails);
    }

    private static readonly int[] CodigosPermitidos = { 400, 401, 403, 404, 409, 422, 500, 503 };

    public ResultadoGestionExcepciones Validar(IEnumerable<ManejadorExcepcion> manejadores)
    {
        var errores = new List<string>();
        var lista = manejadores.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay manejadores declarados.");
        }

        foreach (var manejador in lista)
        {
            if (string.IsNullOrWhiteSpace(manejador.TipoExcepcion))
            {
                errores.Add("Un manejador no declara el tipo de excepción.");
            }

            if (!CodigosPermitidos.Contains(manejador.CodigoHttp))
            {
                errores.Add($"Código HTTP no permitido en '{manejador.TipoExcepcion}':
'{manejador.CodigoHttp}'.");
            }

            if (string.IsNullOrWhiteSpace(manejador.Titulo))
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no declara título.");
            }

            if (!manejador.UsaProblemDetails)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no usa problem details.");
            }

            if (manejador.CodigoHttp >= 500 && !manejador.RegistraError)
            {
                errores.Add($"El manejador '{manejador.TipoExcepcion}' no registra el error.");
            }
        }

        return new ResultadoGestionExcepciones(errores.Count == 0, errores);
    }
}

public record ResultadoGestionExcepciones(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenGestionExcepciones(int Total, int ErroresCliente, int ErroresServidor, int
ConProblemDetails);
```

Cuenta las entradas que cumplen el criterio indicado. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 7.3.6 — Verificar el catálogo por defecto

Reemplaza `tests/AceriaApp.GestionExcepciones.Tests/ValidadorGestionExcepcionesTests.cs` por este archivo completo. Conserva todos los tests obligatorios y añade la comprobación de total 5, errores de cliente 4, errores de servidor 1 y con ProblemDetails 5.

Archivo: `tests/AceriaApp.GestionExcepciones.Tests/ValidadorGestionExcepcionesTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.GestionExcepciones;
using FluentAssertions;
using Xunit;

namespace AceriaApp.GestionExcepciones.Tests;

public class ValidadorGestionExcepcionesTests
{
    private readonly ValidadorGestionExcepciones _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoExcepciones.ManejadoresPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinManejadores_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ManejadorExcepcion>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay manejadores"));
    }

    [Fact]
    public void Validar_CodigoNoPermitido_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 999, Titulo = "T", UsaProblemDetails = true,
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Código HTTP no permitido"));
    }

    [Fact]
    public void Validar_SinTitulo_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 404, Titulo = string.Empty, UsaProblemDetails = true,
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara título"));
    }

    [Fact]
    public void Validar_SinProblemDetails_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 404, Titulo = "T", UsaProblemDetails = false,
```

```
RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa problem details"));
    }

    [Fact]
    public void Validar_ErrorServidorSinRegistro_DevuelveError()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "X", CodigoHttp = 500, Titulo = "T", UsaProblemDetails = true,
RegistraError = false }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no registra el error"));
    }
    [Fact]
    public void Validar_Indisponibilidad503_DevuelveValido()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "InventarioNoDisponibleException", CodigoHttp = 503, Titulo = "Servicio
no disponible", UsaProblemDetails = true, RegistraError = true }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_ErrorClienteSinNivelError_DevuelveValido()
    {
        var manejadores = new List<ManejadorExcepcion>
        {
            new() { TipoExcepcion = "ValidacionException", CodigoHttp = 400, Titulo = "Validación",
UsaProblemDetails = true, RegistraError = false }
        };

        var resultado = _validador.Validar(manejadores);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorGestionExcepciones.Resumen(CatalogoExcepciones.ManejadoresPorDefecto());

        resumen.Total.Should().Be(5);
        resumen.ErroresCliente.Should().Be(4);
        resumen.ErroresServidor.Should().Be(1);
        resumen.ConProblemDetails.Should().Be(5);
    }
}
```

Calcula el resumen sobre el catálogo por defecto.

Comprobación del reto. Guarda los archivos, compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.GestionExcepciones.Tests/AceriaApp.GestionExcepciones.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 13 a 14 tests; la solución debe ejecutar 353 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó

`Resumen_ConCatalogoPorDefecto_CalculaTotales` y compara los hashes de la etapa obligatoria: no debe haber cambiado. Revisa `git diff --check` y el índice antes de registrar el reto en un commit local separado.

Diagnóstico si falla. Si aparece un tipo duplicado, reemplaza el contenido completo en vez de pegarlo al final. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 13 pruebas de catálogo, la implementación funcional y 40 documentos Markdown. La solución incluye 33 proyectos, 21 proyectos de tests y 353 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Continuación. Copia esta etapa obligatoria para abordar 7.4: Retry y Circuit Breaker mediante Polly. Conserva por separado la copia del reto.

Punto 7.4 — Retry y Circuit Breaker mediante Polly

Reintentar con límites y abrir el circuito cuando fallan las dependencias

La resiliencia no consiste en repetir automáticamente cualquier solicitud. Un retry sin límites puede saturar el servicio que está fallando, y repetir operaciones no idempotentes puede duplicar sus efectos. Polly permite controlar tiempos, reintentos y circuit breaker, pero exige elegir condiciones precisas y observar los cambios entre funcionamiento normal, fallos consecutivos y recuperación.

En la última etapa configuraremos las políticas sobre clientes HTTP reales y comprobaremos su reacción a fallos introducidos deliberadamente. Un catálogo que declara TieneRetry o TieneCircuitBreaker solo describe una intención. El objetivo será contrastar esa declaración con la configuración efectiva, los resultados HTTP y las pruebas de recuperación, sin alterar el comportamiento de Producción, Inventario y la mensajería heredada.

Carpeta de trabajo: `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar el análisis y el comportamiento ejecutable. La etapa termina con 403 pruebas en 34 proyectos y 22 proyectos de tests.

Ejercicio guiado. Incorpora `PoliticaResiliencia`, `ValidadorResiliencia` y `CatalogoPoliticas`, dos documentos nuevos y 18 pruebas de catálogo. Los bloques funcionales se aplican progresivamente; el paso 17 completa su registro y comprueba la compilación del punto.

PASO 7.4.1 — Crear la carpeta del punto 7.4 a partir del punto 7.3

La etapa autónoma continúa 33 proyectos y 353 pruebas antes de añadir los cambios del punto.

1. Localiza la etapa completa `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly` y copia en ella el contenido de `7.3-gestion-excepciones`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `7.4-retry-circuit-breaker-polly`.

4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `7.3-gestion-excepciones` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `7.3-gestion-excepciones`: verifica la ruta antes de editar.

PASO 7.4.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `7.4-retry-circuit-breaker-polly` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 33 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 33 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 7.4.3 — Verificar que la carpeta `docs` contiene los documentos del punto 7.3

El punto conserva 40 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 40 archivos Markdown.
2. Localiza los ADR-010 a ADR-029 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones`.
4. Comprueba que aún no existen `docs/retry-circuit-breaker-polly.md` ni `docs/ADR-030-retry-circuit-breaker-polly.md`.

Verificación concreta. Hay 40 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 7.4.4 — Crear el documento de resiliencia en `docs`

La ficha separa el análisis de los contratos y la evidencia ejecutable.

1. Crea `docs/retry-circuit-breaker-polly.md` dentro de la copia de `7.4-retry-circuit-breaker-polly`.
2. Introduce el contenido completo del bloque siguiente.

3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/retry-circuit-breaker-polly.md` existe y que los 40 documentos anteriores permanecen intactos.

Archivo: `docs/retry-circuit-breaker-polly.md`. **Operación:** Crear.

```
# Retry y Circuit Breaker mediante Polly — Punto 7.4

## Resiliencia

Capacidad de limitar el impacto de fallos y recuperarse cuando las condiciones lo permiten. No garantiza
disponibilidad ni convierte una operación fallida en éxito.

## Patrones de resiliencia

- Retry: reintenta operaciones fallidas.
- CircuitBreaker: corta el flujo cuando el destino falla.
- Timeout: limita el tiempo de espera.
- Bulkhead: aísla recursos por operación.

## Retry

- Se aplica a fallos transitorios.
- Intervalo fijo o exponencial.
- Requiere idempotencia en las operaciones.

## Circuit Breaker

Estados: cerrado, abierto y semiabierto.

## Combinación de patrones

    // Orden exterior → interior del handler de resiliencia:
    // timeout total → retry → circuit breaker → timeout de intento.
    // Cada reintento atraviesa el mismo circuito y queda dentro del presupuesto total.

## Polly

Polly v8 ofrece pipelines de resiliencia. La integración HTTP del ejercicio usa
`Microsoft.Extensions.Http.Resilience` y `AddResilienceHandler`; no mezcla la API v7 `Policy.WrapAsync`/
`AddPolicyHandler` con la API v8.

## Configuración de políticas

    var retry = new HttpRetryStrategyOptions
    {
        MaxRetryAttempts = 2,
        Delay = TimeSpan.FromMilliseconds(200),
        BackoffType = DelayBackoffType.Exponential,
        UseJitter = true
    };
    var circuit = new HttpCircuitBreakerStrategyOptions
    {
        MinimumThroughput = 4,
        FailureRatio = 0.5,
        SamplingDuration = TimeSpan.FromSeconds(10),
        BreakDuration = TimeSpan.FromSeconds(5)
    };

## Fallback

Solo devuelve una alternativa si el negocio la ha aprobado y puede distinguirse de un dato actual. En esta
consulta no se inventa stock ni se devuelve éxito ficticio: circuito abierto o presupuesto agotado se
traducen en indisponibilidad.

## Buenas prácticas

1. Timeout en todas las llamadas externas.
2. Retry solo en operaciones idempotentes.
3. Circuit breaker en destinos que fallan.
4. Registrar reintentos y aperturas.
5. Ajustar parámetros al contexto.

## Cierre del módulo 7
```

```
El proyecto AceriaApp dispone de:

- Logging estructurado con Serilog.
- Health checks separados en liveness y readiness.
- Gestión de excepciones con problem details.
- Retry, circuit breaker y timeout con Polly.
- 403 pruebas acumuladas en 34 proyectos, con 22 proyectos de tests y 42 documentos en `docs`.
- Treinta puntos completados (1.1 a 1.5, 2.1 a 2.4, 3.1 a 3.4, 4.1 a 4.5, 5.1 a 5.5, 6.1 a 6.3 y 7.1 a 7.4).

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Cliente HTTP | Políticas de retry, circuit breaker y timeout |
| Parámetros | Ajustados por destino |
| Registro | Reintentos y aperturas |
| Validación | Test de coherencia |
## Contrato del ejercicio

El presupuesto total es de 6 segundos y cada intento tiene 1 segundo. Hay como máximo dos reintentos además
del intento inicial. Los fallos elegibles son HTTP 408/429/5xx, transporte y timeout del intento. No se
reintentan 400/401/403/404, cancelación del usuario ni métodos mutantes. El circuito se comparte por cliente/
destino; no se crea para cada llamada.

`MinimoDeMuestras` representa el mínimo de llamadas observado, no un número de fallos consecutivos. La
apertura combina ese mínimo con la proporción de fallos dentro de la ventana. Los valores 5 y 3 del catálogo
son ejemplos de mínimos para dos destinos; la configuración HTTP ejecutable usa 4. Las cinco entradas del
catálogo conservan sus dos retries, dos circuit breakers y un timeout; sus ocho reintentos declarados no son
los intentos de una petición ni la política del outbox de M6.

`OperacionIdempotente` expresa una condición del contrato, no implementa idempotencia. El verificador rechaza
un retry que no la declare y exige límites y registro. Las pruebas funcionales observan intentos,
cancelación, apertura y recuperación del circuito real de Polly.
```

Verificación concreta. `docs/retry-circuit-breaker-polly.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/retry-circuit-breaker-polly.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.4.5 — Añadir la integración de Polly v8 mediante Microsoft.Extensions.Http.Resilience a la biblioteca de observabilidad

La biblioteca de resiliencia para HttpClient proporciona los manejadores necesarios de Polly v8. Antes de activar una política, verifica cómo se registra en DI y qué solicitudes pueden repetirse sin efectos secundarios.

1. Abre la carpeta `src` de la etapa actual y localiza los proyectos API de Producción e Inventario.
2. Aplica el archivo completo `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj` mostrado en este paso y comprueba la referencia a `Microsoft.Extensions.Http.Resilience (versión fijada en el bloque funcional)`.
3. Comprueba `global.json` y ejecuta `dotnet --version` para confirmar el SDK .NET 8 compatible antes de restaurar paquetes.
4. Añade la biblioteca a `AceriaApp.sln` si aún no aparece, ejecuta `dotnet restore src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj` y comprueba que la restauración termina sin errores.

Verificación concreta. La biblioteca compartida está incluida en la solución y su archivo de proyecto restaura los paquetes fijados.

Diagnóstico si falla.

- Instalar Polly v7 y copiar Policy.WrapAsync junto a un handler v8 mezcla dos API: usa la referencia y el código completos del módulo.
- Modificar el proyecto de una etapa anterior rompe su autonomía: comprueba el camino completo antes de editar.

Integración funcional — `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Serilog.AspNetCore" Version="8.0.3" />
    <PackageReference Include="Microsoft.Data.SqlClient" Version="5.2.2" />
    <PackageReference Include="RabbitMQ.Client" Version="7.1.2" />
    <ProjectReference Include="../AceriaApp.Comunicacion.Infraestructura/
AceriaApp.Comunicacion.Infraestructura.csproj" />
    <PackageReference Include="Microsoft.Extensions.Http.Resilience" Version="8.10.0" />
  </ItemGroup>
</Project>
```

Fija la dependencia usada por esta etapa del ejercicio. Conecta este proyecto con otro proyecto real de la solución.

PASO 7.4.6 — Crear la carpeta `Resiliencia` en el proyecto API

`Resiliencia` agrupa datos y reglas del ejercicio de análisis; los adaptadores de comunicación tienen sus propias responsabilidades.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `Resiliencia` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/Resiliencia` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `7.3-gestion-excepciones`.

Verificación concreta. Existe `src/AceriaApp.Api/Resiliencia` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

PASO 7.4.7 — Crear la clase `PoliticaResiliencia`

PoliticaResiliencia describe restricciones y parámetros de un escenario. Ninguna propiedad del objeto abre o cierra un circuito real; la implementación se configura sobre el cliente HTTP que utiliza la aplicación.

1. Crea `src/AceriaApp.Api/Resiliencia/PoliticaResiliencia.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Resiliencia/PoliticaResiliencia.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Resiliencia;

public class PoliticaResiliencia
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public int NumeroDeReintentos { get; init; }
    public int MinimoDeMuestras { get; init; }
    public double ProporcionDeFallos { get; init; } = 0.5;
    public int VentanaMuestreoSegundos { get; init; } = 10;
    public int DuracionAperturaSegundos { get; init; } = 5;
    public bool OperacionIdempotente { get; init; }
    public bool TieneTimeout { get; init; }
    public bool RegistraEventos { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `PoliticaResiliencia` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Resiliencia` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.4.8 — Crear la clase `ValidadorResiliencia`

El validador comprueba que los valores declarados encajan en las reglas docentes. Es distinto de medir el número de intentos o el estado del circuito bajo fallos de red.

1. Crea `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Resiliencia;

public class ValidadorResiliencia
{
    private static readonly string[] TiposPermitidos = { "Retry", "CircuitBreaker", "Timeout" };

    public ResultadoResiliencia Validar(IEnumerable<PoliticaResiliencia> politicas)
    {
        var errores = new List<string>();
        var lista = politicas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay políticas declaradas.");
        }

        foreach (var politica in lista)
        {
            if (string.IsNullOrWhiteSpace(politica.Nombre))
            {
                errores.Add("Una política no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(politica.Tipo))
            {
                errores.Add($"Tipo no permitido en '{politica.Nombre}': '{politica.Tipo}'.");
```

```
            }

            if (politica.Tipo == "Retry" && politica.NumeroDeReintentos < 1)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara reintentos.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.MinimoDeMuestras < 2)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara mínimo de
muestras.");
            }

            if (politica.Tipo == "Retry" && !politica.OperacionIdempotente)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara una operación
idempotente.");
            }

            if (politica.Tipo == "CircuitBreaker" &&
                (!double.IsFinite(politica.ProporcionDeFallos) || politica.ProporcionDeFallos <= 0 ||
politica.ProporcionDeFallos > 1))
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' tiene una proporción de
fallos fuera de (0, 1].");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.VentanaMuestreoSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una ventana de
muestreo positiva.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.DuracionAperturaSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una duración de
apertura positiva.");
            }

            if (!politica.TieneTimeout)
            {
                errores.Add($"La política '{politica.Nombre}' no tiene timeout.");
            }

            if (!politica.RegistraEventos)
            {
                errores.Add($"La política '{politica.Nombre}' no registra eventos.");
            }
        }

        return new ResultadoResiliencia(errores.Count == 0, errores);
    }
}

public record ResultadoResiliencia(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Completa la declaración o la operación mostrada en el bloque. Devuelve validez y diagnósticos; no instala middleware ni ejecuta llamadas remotas.

Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorResiliencia` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Resiliencia` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

Integración funcional — `src/AceriaApp.Observabilidad/ClienteInventarioResiliente.cs`.

El adaptador reutiliza la semántica HTTP de M6 y traduce apertura del circuito o timeout a la indisponibilidad conocida por la aplicación. No devuelve stock inventado ni transforma una cancelación del usuario en un fallo de negocio.

Archivo: `src/AceriaApp.Observabilidad/ClienteInventarioResiliente.cs`. **Operación:** Crear.

```
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Polly.CircuitBreaker;
using Polly.Timeout;
namespace AceriaApp.Observabilidad;

public sealed class ClienteInventarioResiliente(HttpClient http) : IClienteInventario
{
    private readonly ClienteInventarioHttp cliente = new(http);

    public async Task<MaterialDto?> ObtenerAsync(string material, CancellationToken ct)
    {
        try { return await cliente.ObtenerAsync(material, ct); }
        catch (Exception ex) when (ex is BrokenCircuitException or TimeoutRejectedException)
        {
            ct.ThrowIfCancellationRequested();
            throw new InventarioNoDisponibleException("Inventario no está disponible temporalmente.", ex);
        }
    }
}
```

Sitúa esta responsabilidad dentro del proyecto de observabilidad.

Integración funcional — `src/AceriaApp.Observabilidad/ResilienciaInventario.cs`.

La política se aplica al puerto de lectura de Inventario después de AddComunicacion. El presupuesto total de seis segundos envuelve retry, circuito y límite de un segundo por intento. El cuerpo JSON se lee dentro del límite; este adaptador no es un cliente de streaming. El circuito comparte estado por nombre de cliente y autoridad, mientras cada autoridad conserva su propio circuito.

Archivo: `src/AceriaApp.Observabilidad/ResilienciaInventario.cs`. **Operación:** Crear.

```
using System.Net;
using AceriaApp.Comunicacion.Aplicacion;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using Microsoft.Extensions.Http.Resilience;
using Microsoft.Extensions.Logging;
using Polly;
using Polly.Timeout;
namespace AceriaApp.Observabilidad;

public static class ResilienciaInventario
{
    public static IServiceCollection AddResilienciaInventario(this IServiceCollection services,
IConfiguration config)
    {
        // Sustituye el puerto registrado en M6; su dominio y sus contratos no cambian.
        services.RemoveAll<IClienteInventario>();
        services.AddHttpClient<IClienteInventario, ClienteInventarioResiliente>("inventario-resiliente", http
=>
```

```
        {
            http.BaseAddress = new Uri(config["Comunicacion:InventarioUrl"] ?? "http://localhost:5106/");
            http.Timeout = Timeout.InfiniteTimeSpan; // Polly gobierna un único presupuesto.
        }).AddPoliticaResilienciaInventario();
        return services;
    }

    public static IHttpClientBuilder AddPoliticaResilienciaInventario(this IHttpClientBuilder cliente)
    {
        cliente.Services.TryAddSingleton(TimeProvider.System);
        cliente.AddResilienceHandler("lectura-inventario", (pipeline, contexto) =>
        {
            var reloj = contexto.ServiceProvider.GetRequiredService<TimeProvider>();
            var log =
contexto.ServiceProvider.GetRequiredService<ILoggerFactory>().CreateLogger("AceriaApp.Resiliencia");
            pipeline.TimeProvider = reloj;
            pipeline
                .AddTimeout(TimeSpan.FromSeconds(6))
                .AddRetry(new HttpRetryStrategyOptions
                {
                    MaxRetryAttempts = 2,
                    Delay = TimeSpan.FromMilliseconds(200),
                    BackoffType = DelayBackoffType.Exponential,
                    UseJitter = true,
                    ShouldHandle = args => ValueTask.FromResult(
                        EsLectura(args.Context.GetRequestMessage()?.Method) &&
                        EsTransitorio(args.Outcome, args.Context.CancellationToken)),
                    OnRetry = args =>
                    {
                        log.LogWarning(new EventId(7401, "ReintentoInventario"),
                            "Reintento de inventario {Intento} tras {DemoraMs} ms; estado HTTP {EstadoHttp}",
                            args.AttemptNumber + 1, args.RetryDelay.TotalMilliseconds,
(int?)args.Outcome.Result?.StatusCode);
                        return default;
                    },
                    // Se usa el mismo reloj para Retry-After en fecha y para las estrategias.
                    // El timeout exterior cancela incluso una espera Retry-After de varios minutos.
                    DelayGenerator = args =>
                    {
                        var cabecera = args.Outcome.Result?.Headers.RetryAfter;
                        TimeSpan? espera = cabecera?.Delta;
                        if (espera is null && cabecera?.Date is { } fecha)
                            espera = fecha - reloj.GetUtcNow();
                        return ValueTask.FromResult(espera is { } valor && valor >= TimeSpan.Zero ? espera :
null);
                    }
                })
                .AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions
                {
                    MinimumThroughput = 4,
                    FailureRatio = 0.5,
                    SamplingDuration = TimeSpan.FromSeconds(10),
                    BreakDuration = TimeSpan.FromSeconds(5),
                    ShouldHandle = args => ValueTask.FromResult(
                        EsTransitorio(args.Outcome, args.Context.CancellationToken)),
                    OnOpened = _ =>
                    {
                        log.LogWarning(new EventId(7402, "CircuitoInventarioAbierto"), "Circuito de
inventario {Estado}", "abierto");
                        return default;
                    },
                    OnClosed = _ =>
                    {
                        log.LogInformation(new EventId(7403, "CircuitoInventarioCerrado"), "Circuito de
inventario {Estado}", "cerrado");
                        return default;
                    }
                })
                .AddTimeout(TimeSpan.FromSeconds(1));
        }).SelectPipelineByAuthority();
        cliente.AddHttpMessageHandler(() => new LeerRespuestaCompletaHandler());
        // IHttpClientFactory conserva la política por nombre y autoridad entre clientes/peticiones.
        return cliente;
    }

    private sealed class LeerRespuestaCompletaHandler : DelegatingHandler
```

```
    {
        protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request,
CancellationToken ct)
        {
            var respuesta = await base.SendAsync(request, ct);
            try
            {
                // HttpClient normalmente almacena el cuerpo después de abandonar sus handlers.
                // Aquí la lectura queda dentro del timeout de intento y del presupuesto total.
                await respuesta.Content.ReadAsByteArrayAsync(ct);
                return respuesta;
            }
            catch { respuesta.Dispose(); throw; }
        }
    }

    private static bool EsLectura(HttpMethod? metodo) =>
        metodo == HttpMethod.Get || metodo == HttpMethod.Head || metodo == HttpMethod.Options;

    private static bool EsTransitorio(Outcome<HttpResponseMessage> resultado, CancellationToken ct)
    {
        if (ct.IsCancellationRequested) return false;
        if (resultado.Exception is HttpRequestException or TimeoutRejectedException) return true;
        return resultado.Result?.StatusCode is HttpStatusCode.RequestTimeout or
HttpStatusCode.TooManyRequests ||
            resultado.Result is { } respuesta && (int)respuesta.StatusCode is >= 500 and <= 599;
    }
}
```

Selecciona el adaptador resiliente después de los registros heredados. Permite aportar un reloj al contenedor sin imponer uno sobre el existente. Construye las estrategias del cliente HTTP mediante la integración oficial.

Añade un límite temporal en esta posición de la composición. Configura reintentos de los fallos transitorios seleccionados. Registra cada reintento mediante propiedades seguras.

Mantiene el estado del circuito y sus criterios de apertura. Emite el evento observable de apertura del circuito. Emite el evento observable de recuperación del circuito.

Separa el estado del circuito por esquema, host y puerto. Incluye la lectura del cuerpo dentro del presupuesto y propaga cancelación.

PASO 7.4.9 — Añadir los `using` necesarios en `ValidadorResiliencia`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs`.
2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.Resiliencia;

public class ValidadorResiliencia
{
    private static readonly string[] TiposPermitidos = { "Retry", "CircuitBreaker", "Timeout" };

    public ResultadoResiliencia Validar(IEnumerable<PoliticaResiliencia> politicas)
    {
```

```
        var errores = new List<string>();
        var lista = politicas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay políticas declaradas.");
        }

        foreach (var politica in lista)
        {
            if (string.IsNullOrWhiteSpace(politica.Nombre))
            {
                errores.Add("Una política no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(politica.Tipo))
            {
                errores.Add($"Tipo no permitido en '{politica.Nombre}': '{politica.Tipo}'.");
            }

            if (politica.Tipo == "Retry" && politica.NumeroDeReintentos < 1)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara reintentos.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.MinimoDeMuestras < 2)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara mínimo de
muestras.");
            }

            if (politica.Tipo == "Retry" && !politica.OperacionIdempotente)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara una operación
idempotente.");
            }

            if (politica.Tipo == "CircuitBreaker" &&
                (!double.IsFinite(politica.ProporcionDeFallos) || politica.ProporcionDeFallos <= 0 ||
politica.ProporcionDeFallos > 1))
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' tiene una proporción de
fallos fuera de (0, 1].");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.VentanaMuestreoSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una ventana de
muestreo positiva.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.DuracionAperturaSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una duración de
apertura positiva.");
            }

            if (!politica.TieneTimeout)
            {
                errores.Add($"La política '{politica.Nombre}' no tiene timeout.");
            }

            if (!politica.RegistraEventos)
            {
                errores.Add($"La política '{politica.Nombre}' no registra eventos.");
            }
        }

        return new ResultadoResiliencia(errores.Count == 0, errores);
    }
}

public record ResultadoResiliencia(bool EsValida, IReadOnlyList<string> Errores);
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorResiliencia.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.4.10 — Crear la clase `CatalogoPoliticas`

CatalogoPoliticas compara ejemplos de retry y circuit breaker. Su clasificación debe interpretarse junto a la ejecución funcional, porque el catálogo no modifica las solicitudes del HttpClient.

1. Crea `src/AceriaApp.Api/Resiliencia/CatalogoPoliticas.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Resiliencia/CatalogoPoliticas.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Resiliencia;

public class CatalogoPoliticas
{
    public static List<PoliticaResiliencia> PoliticasPorDefecto()
    {
        return new List<PoliticaResiliencia>
        {
            new() { Nombre = "RetryHttp", Tipo = "Retry", OperacionIdempotente = true, NumeroDeReintentos =
3, MinimoDeMuestras = 0, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "CircuitBreakerHttp", Tipo = "CircuitBreaker", NumeroDeReintentos = 0,
MinimoDeMuestras = 5, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "TimeoutHttp", Tipo = "Timeout", NumeroDeReintentos = 0, MinimoDeMuestras = 0,
TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "RetryEventos", Tipo = "Retry", OperacionIdempotente = true, NumeroDeReintentos
= 5, MinimoDeMuestras = 0, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "CircuitBreakerEventos", Tipo = "CircuitBreaker", NumeroDeReintentos = 0,
MinimoDeMuestras = 3, TieneTimeout = true, RegistraEventos = true }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoPoliticas` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Resiliencia` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 7.4.11 — Añadir el `using` necesario en `CatalogoPoliticas`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Resiliencia/CatalogoPoliticas.cs`.

2. Reemplaza el contenido por el archivo completo siguiente, que incorpora los using antes del namespace.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Resiliencia/CatalogoPoliticas.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;

namespace AceriaApp.Api.Resiliencia;

public class CatalogoPoliticas
{
    public static List<PoliticaResiliencia> PoliticasPorDefecto()
    {
        return new List<PoliticaResiliencia>
        {
            new() { Nombre = "RetryHttp", Tipo = "Retry", OperacionIdempotente = true, NumeroDeReintentos =
3, MinimoDeMuestras = 0, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "CircuitBreakerHttp", Tipo = "CircuitBreaker", NumeroDeReintentos = 0,
MinimoDeMuestras = 5, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "TimeoutHttp", Tipo = "Timeout", NumeroDeReintentos = 0, MinimoDeMuestras = 0,
TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "RetryEventos", Tipo = "Retry", OperacionIdempotente = true, NumeroDeReintentos
= 5, MinimoDeMuestras = 0, TieneTimeout = true, RegistraEventos = true },
            new() { Nombre = "CircuitBreakerEventos", Tipo = "CircuitBreaker", NumeroDeReintentos = 0,
MinimoDeMuestras = 3, TieneTimeout = true, RegistraEventos = true }
        };
    }
}
```

Verificación concreta. `CatalogoPoliticas.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 7.4.12 — Crear el proyecto de tests del punto 7.4

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.Resiliencia.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 15 y 16 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj` y `dotnet restore tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
```

```
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 7.4.13 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj` y `dotnet build tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

Integración funcional — `src/AceriaApp.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM074;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM074;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM074;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m074",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

Agrupa las conexiones de laboratorio específicas de la etapa. Activa o desactiva los procesos de mensajería y su dependencia en readiness. Fija el umbral propio de logging y admite sobreescritura por variables de entorno.

Mantiene el endpoint de fallo desactivado por defecto.

Integración funcional — `src/AceriaApp.Inventario.Api/appsettings.json`.

La configuración usa bases y prefijos propios de esta etapa. Las variables de entorno permiten sustituir conexiones y activar el laboratorio sin editar secretos en el repositorio.

Archivo: `src/AceriaApp.Inventario.Api/appsettings.json`. **Operación:** Reemplazar.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM074;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m074",
    "WorkersActivos": false
  },
  "Observabilidad": {
    "NivelMinimo": "Information",
    "DiagnosticoActivo": false
  }
}
```

PASO 7.4.14 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj` y `dotnet list tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

PASO 7.4.15 — Crear el archivo `ValidadorResilienciaTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.Resiliencia.Tests/ValidadorResilienciaTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Resiliencia.Tests/ValidadorResilienciaTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Resiliencia;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Resiliencia.Tests;

public class ValidadorResilienciaTests
{
    private readonly ValidadorResiliencia _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValida()
    {
        var resultado = _validador.Validar(CatalogoPoliticas.PoliticasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinPoliticas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<PoliticaResiliencia>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay políticas"));
    }
```

```
    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_RetrySinReintentos_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Retry", NumeroDeReintentos = 0, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara reintentos"));
    }

    [Fact]
    public void Validar_CircuitBreakerSinMinimoDeMuestras_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 0, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara mínimo de muestras"));
    }

    [Fact]
    public void Validar_SinTimeout_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Timeout", TieneTimeout = false, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene timeout"));
    }

    [Fact]
    public void Validar_SinRegistro_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Timeout", TieneTimeout = true, RegistraEventos = false }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no registra eventos"));
    }
    [Fact]
    public void Validar_RetryNoIdempotente_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
```

```
            new() { Nombre = "X", Tipo = "Retry", NumeroDeReintentos = 2, OperacionIdempotente = false,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara una operación idempotente"));
    }
    [Fact]
    public void Validar_CircuitBreakerConUnaMuestra_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 1, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara mínimo de muestras"));
    }
    [Fact]
    public void Validar_CircuitBreakerConProporcionCero_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, ProporcionDeFallos = 0,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("proporción de fallos fuera"));
    }
    [Fact]
    public void Validar_CircuitBreakerConProporcionInvalida_DevuelveError()
    {
        foreach (var proporcion in new[] { 1.1, double.NaN, double.PositiveInfinity,
double.NegativeInfinity })
        {
            var politicas = new List<PoliticaResiliencia>
            {
                new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, ProporcionDeFallos =
proporcion, TieneTimeout = true, RegistraEventos = true }
            };

            var resultado = _validador.Validar(politicas);

            resultado.EsValida.Should().BeFalse();
            resultado.Errores.Should().Contain(e => e.Contains("proporción de fallos fuera"));
        }
    }
    [Fact]
    public void Validar_CircuitBreakerSinVentana_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, VentanaMuestreoSegundos = 0,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("ventana de muestreo positiva"));
    }
    [Fact]
    public void Validar_CircuitBreakerSinDuracionApertura_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, DuracionAperturaSegundos =
0, TieneTimeout = true, RegistraEventos = true }
```

```
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("duración de apertura positiva"));
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorResilienciaTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Integración funcional — `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`.

El archivo completo fija las referencias y versiones que utilizará la etapa; añade el proyecto a la solución antes de compilar.

Archivo: `tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj`. **Operación:** Reemplazar.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="Microsoft.Extensions.TimeProvider.Testing" Version="8.10.0" />
  </ItemGroup>
</Project>
```

Integración funcional — `tests/AceriaApp.Resiliencia.Tests/ResilienciaFuncionalTests.cs`.

Las pruebas utilizan IHttpClientFactory y las estrategias Polly reales con transporte controlado y reloj virtual. Comprueban el máximo de intentos, métodos de escritura, Retry-After, cancelación, cuerpos bloqueados, liberación de respuestas y recuperación concurrente del circuito. El reloj virtual evita esperar segundos de pared para acreditar el estado de la política.

Archivo: `tests/AceriaApp.Resiliencia.Tests/ResilienciaFuncionalTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Net.Http.Headers;
using System.Text;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Observabilidad;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
```

```
using Microsoft.Extensions.Time.Testing;
using Polly.CircuitBreaker;
using Polly.Timeout;
using Xunit;
namespace AceriaApp.Resiliencia.Tests;

public sealed class ResilienciaFuncionalTests
{
    [Fact]
    public async Task Un_503_se_reintenta_y_libera_su_respuesta_antes_del_200()
    {
        var contenido = new ContenidoRastreado("fallo temporal");
        using var prueba = new Banco((_, intento, _) => Task.FromResult(intento == 1
            ? new HttpResponseMessage(HttpStatusCode.ServiceUnavailable) { Content = contenido }
            : Respuesta(HttpStatusCode.OK)));
        using var http = prueba.CrearCliente();
        using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal(2, prueba.Transporte.Intentos);
        Assert.True(contenido.Liberado);
        var evento = Assert.Single(prueba.Logs.Entradas, x => x.Id == 7401);
        Assert.Equal("AceriaApp.Resiliencia", evento.Categoria);
        Assert.Null(evento.Excepcion);
        Assert.Equal(1, evento.Propiedades["Intento"]);
        Assert.Equal(503, evento.Propiedades["EstadoHttp"]);
        Assert.DoesNotContain("fallo temporal", evento.Mensaje);
    }

    [Theory]
    [InlineData(408)] [InlineData(429)] [InlineData(500)] [InlineData(502)] [InlineData(503)]
[InlineData(504)]
    public async Task Estados_transitorios_permiten_recuperar_la_lectura(int estado)
    {
        using var prueba = new Banco((_, intento, _) => Task.FromResult(Respuesta(intento == 1 ?
(HttpStatusCode)estado : HttpStatusCode.OK)));
        using var http = prueba.CrearCliente();
        using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal(2, prueba.Transporte.Intentos);
    }

    [Theory]
    [InlineData("GET")] [InlineData("HEAD")] [InlineData("OPTIONS")]
    public async Task Solo_los_metodos_de_lectura_permiten_reintento(string metodo)
    {
        using var prueba = new Banco((_, intento, _) => Task.FromResult(Respuesta(intento == 1 ?
HttpStatusCode.ServiceUnavailable : HttpStatusCode.OK)));
        using var http = prueba.CrearCliente();
        using var peticion = new HttpRequestMessage(new HttpMethod(metodo), "material");
        using var respuesta = await prueba.CompletarAsync(http.SendAsync(peticion));
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal(2, prueba.Transporte.Intentos);
    }

    [Theory]
    [InlineData(400)] [InlineData(401)] [InlineData(403)] [InlineData(404)]
    public async Task Errores_no_transitorios_no_se_reintentan(int estado)
    {
        using var prueba = new Banco((_, _, _) => Task.FromResult(Respuesta((HttpStatusCode)estado)));
        using var http = prueba.CrearCliente();
        using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
        Assert.Equal((HttpStatusCode)estado, respuesta.StatusCode);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Theory]
    [InlineData("POST")] [InlineData("PUT")] [InlineData("PATCH")] [InlineData("DELETE")]
    public async Task Un_metodo_mutante_no_repite_el_cuerpo_tras_un_503(string metodo)
    {
        var cuerpos = new List<string>();
        using var prueba = new Banco(async (peticion, _, ct) =>
        {
            cuerpos.Add(await peticion.Content!.ReadAsStringAsync(ct));
            return Respuesta(HttpStatusCode.ServiceUnavailable);
        });
        using var http = prueba.CrearCliente();
```

```
        using var peticion = new HttpRequestMessage(new HttpMethod(metodo), "material") { Content = new
StringContent("orden irrepetible") };
        using var respuesta = await prueba.CompletarAsync(http.SendAsync(peticion));
        Assert.Equal(HttpStatusCode.ServiceUnavailable, respuesta.StatusCode);
        Assert.Equal(new[] { "orden irrepetible" }, cuerpos);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Una_lectura_agota_tres_intentos_y_conserva_la_ultima_respuesta()
    {
        var contenidos = new List<ContenidoRastreado>();
        using var prueba = new Banco((_, _, _) =>
        {
            var contenido = new ContenidoRastreado("sigue caído"); contenidos.Add(contenido);
            return Task.FromResult(new HttpResponseMessage(HttpStatusCode.ServiceUnavailable) { Content =
contenido });
        });
        using var http = prueba.CrearCliente();
        using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
        Assert.Equal(HttpStatusCode.ServiceUnavailable, respuesta.StatusCode);
        Assert.Equal(3, prueba.Transporte.Intentos);
        Assert.True(contenidos[0].Liberado);
        Assert.True(contenidos[1].Liberado);
        Assert.False(contenidos[2].Liberado);
        Assert.Equal("sigue caído", await respuesta.Content.ReadAsStringAsync());
    }

    [Fact]
    public async Task Un_error_de_transporte_se_reintenta()
    {
        using var prueba = new Banco((_, intento, _) => intento == 1
            ? Task.FromException<HttpResponseMessage>(new HttpRequestException("conexión rechazada"))
            : Task.FromResult(Respuesta(HttpStatusCode.OK)));
        using var http = prueba.CrearCliente();
        using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal(2, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Cancelar_la_peticion_interrumpe_el_transporte_sin_reintentar()
    {
        var entrada = new TaskCompletionSource(TaskCreationOptions.RunContinuationsAsynchronously);
        using var prueba = new Banco(async (_, _, ct) =>
        { entrada.TrySetResult(); await Task.Delay(Timeout.InfiniteTimeSpan, ct); return
Respuesta(HttpStatusCode.OK); });
        using var http = prueba.CrearCliente();
        using var cancelacion = new CancellationTokenSource();
        var tarea = new ClienteInventarioResiliente(http).ObtenerAsync("CH-01", cancelacion.Token);
        await entrada.Task.WaitAsync(TimeSpan.FromSeconds(3));
        cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(async () => await tarea);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Cancelar_durante_retry_after_impide_un_segundo_envio()
    {
        using var prueba = new Banco((_, _, _) =>
        {
            var respuesta = Respuesta(HttpStatusCode.TooManyRequests);
            respuesta.Headers.RetryAfter = new RetryConditionHeaderValue(TimeSpan.FromSeconds(30));
            return Task.FromResult(respuesta);
        });
        using var http = prueba.CrearCliente();
        using var cancelacion = new CancellationTokenSource();
        var tarea = http.GetAsync("material", cancelacion.Token);
        cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(async () => await tarea);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Un_timeout_de_intento_cancela_transporte_o_cuerpo_y_permite_recuperar()
    {
```

```
        foreach (var cuerpoLento in new[] { false, true })
        {
            var cancelados = 0;
            ContenidoLento? contenido = null;
            using var prueba = new Banco(async (_, intento, ct) =>
            {
                if (intento == 1)
                {
                    if (cuerpoLento)
                        return new HttpResponseMessage(HttpStatusCode.OK)
                        { Content = contenido = new ContenidoLento(() => Interlocked.Increment(ref
cancelados)) };
                    try { await Task.Delay(Timeout.InfiniteTimeSpan, ct); }
                    catch (OperationCanceledException) { Interlocked.Increment(ref cancelados); throw; }
                }
                return Respuesta(HttpStatusCode.OK);
            });
            using var http = prueba.CrearCliente();
            using var respuesta = await prueba.CompletarAsync(http.GetAsync("material"));
            Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
            Assert.Equal(2, prueba.Transporte.Intentos);
            Assert.Equal(1, cancelados);
            if (cuerpoLento) Assert.True(contenido!.Liberado);
        }
    }

    [Fact]
    public async Task Tres_intentos_lentos_terminan_en_indisponibilidad_y_no_exponen_Polly_al_puerto()
    {
        using var prueba = new Banco(async (_, _, ct) =>
        { await Task.Delay(Timeout.InfiniteTimeSpan, ct); return Respuesta(HttpStatusCode.OK); });
        using var http = prueba.CrearCliente();
        var error = await Assert.ThrowsAsync<InventarioNoDisponibleException>(async () =>
            await prueba.CompletarAsync(new ClienteInventarioResiliente(http).ObtenerAsync("CH-01",
CancellationToken.None)));
        Assert.IsType<TimeoutRejectedException>(error.InnerException);
        Assert.Equal(3, prueba.Transporte.Intentos);
    }

    [Theory]
    [InlineData(false)] [InlineData(true)]
    public async Task El_presupuesto_total_acota_retry_after_en_segundos_y_fecha(bool fecha)
    {
        using var prueba = new Banco((_, _, _) =>
Task.FromResult(Respuesta(HttpStatusCode.TooManyRequests)));
        prueba.Transporte.Responder = (_, _, _) =>
        {
            var respuesta = Respuesta(HttpStatusCode.TooManyRequests);
            respuesta.Headers.RetryAfter = fecha
                ? new RetryConditionHeaderValue(prueba.Reloj.GetUtcNow().AddMinutes(1))
                : new RetryConditionHeaderValue(TimeSpan.FromMinutes(1));
            return Task.FromResult(respuesta);
        };
        using var http = prueba.CrearCliente();
        var inicio = prueba.Reloj.GetUtcNow();
        await Assert.ThrowsAsync<TimeoutRejectedException>(async () => await
prueba.CompletarAsync(http.GetAsync("material")));
        Assert.InRange((prueba.Reloj.GetUtcNow() - inicio).TotalSeconds, 6, 7);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task El_circuito_se_comparte_entre_clientes_y_recupera_con_una_sonda_half_open()
    {
        var sondaEntrada = new TaskCompletionSource(TaskCreationOptions.RunContinuationsAsynchronously);
        var sondaSalida = new TaskCompletionSource(TaskCreationOptions.RunContinuationsAsynchronously);
        using var prueba = new Banco(async (_, intento, ct) =>
        {
            if (intento <= 4) return Respuesta(HttpStatusCode.ServiceUnavailable);
            if (intento == 5) { sondaEntrada.TrySetResult(); await sondaSalida.Task.WaitAsync(ct); }
            return Respuesta(HttpStatusCode.OK);
        });
        using (var primero = prueba.CrearCliente())
            for (var i = 0; i < 4; i++)
            { using var fallo = await primero.PostAsync("material", null);
Assert.Equal(HttpStatusCode.ServiceUnavailable, fallo.StatusCode); }
```

```
        using var segundo = prueba.CrearCliente();
        await Assert.ThrowsAsync<BrokenCircuitException>(async () => await
segundo.GetAsync("material").WaitAsync(TimeSpan.FromSeconds(2)));
        Assert.Equal(4, prueba.Transporte.Intentos);
        prueba.Reloj.Advance(TimeSpan.FromSeconds(5));
        var sonda = segundo.GetAsync("material");
        await sondaEntrada.Task.WaitAsync(TimeSpan.FromSeconds(3));
        using var tercero = prueba.CrearCliente();
        await Assert.ThrowsAsync<BrokenCircuitException>(async () => await
tercero.GetAsync("material").WaitAsync(TimeSpan.FromSeconds(2)));
        Assert.Equal(5, prueba.Transporte.Intentos);
        sondaSalida.TrySetResult();
        using var recuperada = await sonda;
        using var siguiente = await tercero.GetAsync("material");
        Assert.Equal(HttpStatusCode.OK, recuperada.StatusCode);
        Assert.Equal(HttpStatusCode.OK, siguiente.StatusCode);
        Assert.Equal(6, prueba.Transporte.Intentos);
        Assert.Single(prueba.Logs.Entradas, x => x.Id == 7402 && x.Propiedades["Estado"]?.ToString() ==
"abierto");
        Assert.Single(prueba.Logs.Entradas, x => x.Id == 7403 && x.Propiedades["Estado"]?.ToString() ==
"cerrado");
        Assert.All(prueba.Logs.Entradas.Where(x => x.Categoria == "AceriaApp.Resiliencia"), x =>
Assert.Null(x.Excepcion));
    }

    [Fact]
    public async Task Un_destino_caido_no_abre_el_circuito_de_otro_destino()
    {
        using var prueba = new Banco((peticion, _, _) => Task.FromResult(Respuesta(
            peticion.RequestUri!.Host == "caido.test" ? HttpStatusCode.ServiceUnavailable :
HttpStatusCode.OK)));
        using var http = prueba.CrearCliente();
        for (var i = 0; i < 4; i++) { using var fallo = await http.PostAsync("http://caido.test/material",
null); }
        await Assert.ThrowsAsync<BrokenCircuitException>(async () => await http.GetAsync("http://caido.test/
material"));
        using var sana = await http.GetAsync("http://sano.test/material");
        Assert.Equal(HttpStatusCode.OK, sana.StatusCode);
        Assert.Equal(5, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Los_404_y_las_cancelaciones_no_abren_el_circuito()
    {
        using var prueba = new Banco((_, _, _) => Task.FromResult(Respuesta(HttpStatusCode.NotFound)));
        using var http = prueba.CrearCliente();
        for (var i = 0; i < 5; i++) { using var noExiste = await http.GetAsync("material");
Assert.Equal(HttpStatusCode.NotFound, noExiste.StatusCode); }
        for (var i = 0; i < 5; i++)
        {
            using var cancelacion = new CancellationTokenSource();
            prueba.Transporte.Responder = async (_, _, ct) => { cancelacion.Cancel(); await
Task.Delay(Timeout.InfiniteTimeSpan, ct); return Respuesta(HttpStatusCode.OK); };
            await Assert.ThrowsAnyAsync<OperationCanceledException>(async () => await
http.GetAsync("material", cancelacion.Token));
        }
        prueba.Transporte.Responder = (_, _, _) => Task.FromResult(Respuesta(HttpStatusCode.OK));
        using var sana = await http.GetAsync("material");
        Assert.Equal(HttpStatusCode.OK, sana.StatusCode);
        Assert.Equal(11, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task El_registro_sustituye_el_cliente_M6_y_mantiene_el_contrato_del_puerto()
    {
        using var prueba = new Banco((_, intento, _) => Task.FromResult(intento == 1
            ? Respuesta(HttpStatusCode.ServiceUnavailable)
            : new HttpResponseMessage(HttpStatusCode.OK) { Content = new StringContent("{\"material\":
\"CH-01\",\"stock\":20}", Encoding.UTF8, "application/json") }), registroCompleto: true);
        var cliente = prueba.Servicios.GetRequiredService<IClienteInventario>();
        var material = await prueba.CompletarAsync(cliente.ObtenerAsync("CH-01", CancellationToken.None));
        Assert.NotNull(material);
        Assert.Equal("CH-01", material.Material);
        Assert.Equal(20, material.Stock);
        Assert.Equal(2, prueba.Transporte.Intentos);
        Assert.Equal("http://inventario.test/api/v1/materiales/CH-01",
```

```
prueba.Transporte.UltimaUri!.AbsoluteUri);
        Assert.Single(prueba.Servicios.GetServices<IClienteInventario>());
    }

    [Fact]
    public async Task El_adaptador_conserva_el_404_como_ausencia_de_material()
    {
        using var prueba = new Banco((_, _, _) => Task.FromResult(Respuesta(HttpStatusCode.NotFound)),
registroCompleto: true);
        var resultado = await prueba.Servicios.GetRequiredService<IClienteInventario>().ObtenerAsync("NO-
EXISTE", CancellationToken.None);
        Assert.Null(resultado);
        Assert.Equal(1, prueba.Transporte.Intentos);
    }

    [Fact]
    public async Task Un_circuito_abierto_se_traduce_a_indisponibilidad_en_el_puerto()
    {
        using var prueba = new Banco((_, _, _) =>
Task.FromResult(Respuesta(HttpStatusCode.ServiceUnavailable)));
        using var http = prueba.CrearCliente();
        for (var i = 0; i < 4; i++) { using var fallo = await http.PostAsync("material", null); }
        var error = await Assert.ThrowsAsync<InventarioNoDisponibleException>(() => new
ClienteInventarioResiliente(http).ObtenerAsync("CH-01", CancellationToken.None));
        Assert.IsAssignableFrom<BrokenCircuitException>(error.InnerException);
        Assert.Equal(4, prueba.Transporte.Intentos);
    }

    private static HttpResponseMessage Respuesta(HttpStatusCode estado) => new(estado) { Content = new
StringContent("respuesta") };

    private sealed class Banco : IDisposable
    {
        public FakeTimeProvider Reloj { get; } = new();
        public ProveedorCaptura Logs { get; } = new();
        public TransporteControlado Transporte { get; }
        public ServiceProvider Servicios { get; }
        private readonly string nombre;
        public Banco(Func<HttpRequestMessage, int, CancellationToken, Task<HttpResponseMessage>> responder,
bool registroCompleto = false)
        {
            Transporte = new TransporteControlado(responder);
            var services = new ServiceCollection();
            services.AddLogging(logging => logging.AddProvider(Logs));
            services.AddSingleton<TimeProvider>(Reloj);
            nombre = registroCompleto ? "inventario-resiliente" : "lectura-prueba";
            if (registroCompleto)
            {
                var config = new ConfigurationBuilder().AddInMemoryCollection(new Dictionary<string, string?>
                { ["Comunicacion:InventarioUrl"] = "http://inventario.test/" }).Build();
                services.AddComunicacion(config);
                services.AddResilienciaInventario(config);
                services.AddHttpClient(nombre).ConfigurePrimaryHttpMessageHandler(() => Transporte);
            }
            else
                services.AddHttpClient(nombre, http => { http.BaseAddress = new Uri("http://
inventario.test/"); http.Timeout = Timeout.InfiniteTimeSpan; })
                    .ConfigurePrimaryHttpMessageHandler(() => Transporte).AddPoliticaResilienciaInventario();
            Servicios = services.BuildServiceProvider();
        }
        public HttpClient CrearCliente() =>
Servicios.GetRequiredService<IHttpClientFactory>().CreateClient(nombre);
        public async Task<T> CompletarAsync<T>(Task<T> tarea)
        {
            // Cada paso cede al planificador; solo el reloj virtual hace avanzar las políticas.
            for (var paso = 0; paso < 100 && !tarea.IsCompleted; paso++)
            { await Task.Delay(1); if (!tarea.IsCompleted) Reloj.Advance(TimeSpan.FromMilliseconds(100)); }
            return await tarea.WaitAsync(TimeSpan.FromSeconds(2));
        }
        public void Dispose() { Servicios.Dispose(); Transporte.Dispose(); }
    }

    private sealed class TransporteControlado(Func<HttpRequestMessage, int, CancellationToken,
Task<HttpResponseMessage>> responder) : HttpMessageHandler
    {
        private int intentos;
```

```
        public int Intentos => Volatile.Read(ref intentos);
        public Uri? UltimaUri { get; private set; }
        public Func<HttpRequestMessage, int, CancellationToken, Task<HttpResponseMessage>> Responder { get;
set; } = responder;
        protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken
cancellationToken)
        { UltimaUri = request.RequestUri; return Responder(request, Interlocked.Increment(ref intentos),
cancellationToken); }
    }

    private sealed class ContenidoLento(Action cancelado) : HttpContent
    {
        public bool Liberado { get; private set; }
        protected override bool TryComputeLength(out long length) { length = 0; return false; }
        protected override Task SerializeToStreamAsync(Stream stream, TransportContext? context) =>
            SerializeToStreamAsync(stream, context, CancellationToken.None);
        protected override async Task SerializeToStreamAsync(Stream stream, TransportContext? context,
CancellationToken cancellationToken)
        {
            try { await Task.Delay(Timeout.InfiniteTimeSpan, cancellationToken); }
            catch (OperationCanceledException) { cancelado(); throw; }
        }
        protected override void Dispose(bool disposing) { Liberado = true; base.Dispose(disposing); }
    }

    private sealed record EntradaLog(string Categoria, int Id, string Mensaje, Exception? Excepcion,
Dictionary<string, object?> Propiedades);
    private sealed class ProveedorCaptura : ILoggerProvider
    {
        public System.Collections.Concurrent.ConcurrentQueue<EntradaLog> Entradas { get; } = new();
        public ILogger CreateLogger(string categoryName) => new Captura(categoryName, Entradas);
        public void Dispose() { }
        private sealed class Captura(string categoria,
System.Collections.Concurrent.ConcurrentQueue<EntradaLog> entradas) : ILogger
        {
            public IDisposable? BeginScope<TState>(TState state) where TState : notnull => null;
            public bool IsEnabled(LogLevel logLevel) => true;
            public void Log<TState>(LogLevel logLevel, EventId eventId, TState state, Exception? exception,
Func<TState, Exception?, string> formatter) =>
                entradas.Enqueue(new EntradaLog(categoria, eventId.Id, formatter(state, exception),
exception,
                    state is IEnumerable<KeyValuePair<string, object?>> propiedades ?
propiedades.ToDictionary(x => x.Key, x => x.Value) : new()));
        }
    }

    private sealed class ContenidoRastreado(string contenido) : StringContent(contenido)
    {
        public bool Liberado { get; private set; }
        protected override void Dispose(bool disposing) { Liberado = true; base.Dispose(disposing); }
    }
}
```

Define un escenario asíncrono cuyo resultado verifica la conducta descrita por el nombre.

PASO 7.4.16 — Crear el archivo `CatalogoPoliticasTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.Resiliencia.Tests/CatalogoPoliticasTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Resiliencia.Tests/CatalogoPoliticasTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Resiliencia;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Resiliencia.Tests;

public class CatalogoPoliticasTests
{
    [Fact]
    public void PoliticasPorDefecto_ContieneCincoPoliticas()
    {
        var politicas = CatalogoPoliticas.PoliticasPorDefecto();

        politicas.Should().HaveCount(5);
    }

    [Fact]
    public void PoliticasPorDefecto_ContieneDosRetry()
    {
        var politicas = CatalogoPoliticas.PoliticasPorDefecto();

        politicas.Count(p => p.Tipo == "Retry").Should().Be(2);
    }

    [Fact]
    public void PoliticasPorDefecto_ContieneDosCircuitBreaker()
    {
        var politicas = CatalogoPoliticas.PoliticasPorDefecto();

        politicas.Count(p => p.Tipo == "CircuitBreaker").Should().Be(2);
    }

    [Fact]
    public void PoliticasPorDefecto_ContieneUnTimeout()
    {
        var politicas = CatalogoPoliticas.PoliticasPorDefecto();

        politicas.Count(p => p.Tipo == "Timeout").Should().Be(1);
    }

    [Fact]
    public void PoliticasPorDefecto_TodasTienenTimeoutYRegistranEventos()
    {
        var politicas = CatalogoPoliticas.PoliticasPorDefecto();

        politicas.Should().OnlyContain(p => p.TieneTimeout);
        politicas.Should().OnlyContain(p => p.RegistraEventos);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Compara el recuento obtenido con el valor esperado del catálogo.

Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoPoliticasTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 7.4.17 — Compilar la solución completa

La última etapa debe compilar con Polly v8 y superar pruebas que introduzcan fallos controlados. Hay que observar tanto un retry limitado como la apertura y recuperación del circuito, manteniendo intactos los contratos HTTP anteriores.

1. Aplica los bloques de registro final de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 34 proyectos y termina sin errores.

Registro de biblioteca compartida. Ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj`; los proyectos de tests se incorporan en sus pasos de creación. Después de reemplazar todos los archivos de este paso, compila la solución.

Verificación concreta. La solución de la etapa compila con 34 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

Integración funcional — `src/AceriaApp.Api/Program.cs`.

Este archivo de composición conserva las rutas y la inicialización de M6 y registra los nuevos componentes de la etapa en ambas API. Reemplázalo completo para mantener un orden reproducible.

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Api.Comunicacion;
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddLoggingAceria("Produccion", builder.Configuration);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
builder.Services.AddComunicacion(builder.Configuration);
builder.Services.AddResilienciaInventario(builder.Configuration);
builder.Services.AddHttpClient("inventario-resiliente").AddHttpMessageHandler<PropagarCorrelacionHandler>();
builder.Services.AddSaludAceria(builder.Configuration, inventario: false);
```

```
builder.Services.AddErroresAceria();
var app=builder.Build();
app.UseLoggingAceria();
app.UseErroresAceria();
app.MapDiagnosticoAceria();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
```

```
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
app.MapProduccion();
app.MapComunicacion();
app.MapSaludAceria();
```

```
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Registra el logger JSON y los servicios que necesita la correlación. Propaga el identificador validado hacia Inventario. Registra las dependencias propias de este servicio para readiness.

Añade ProblemDetails y el manejador de excepciones al contenedor. Abre la correlación y el resumen antes de ejecutar el resto del pipeline. Sitúa el manejador dentro de los ámbitos de correlación y logging.

Aplica las dos condiciones de habilitación del diagnóstico. Publica liveness y readiness como rutas independientes.

Recorrido funcional 7.4 — retry, apertura y recuperación del circuito

Abre A, B y C en `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`. Usa PowerShell 7, .NET 8 y LocalDB. Esta etapa utiliza bases con sufijo `M074`. Mantendremos los workers desactivados: la avería que vas a observar es la conexión HTTP a Inventario, sin introducir RabbitMQ en el experimento.

En C, prepara la etapa:

```
sqllocaldb start MSSQLLocalDB
if ($LASTEXITCODE -ne 0) { throw 'No se pudo iniciar MSSQLLocalDB.' }
dotnet restore .\AceriaApp.sln
if ($LASTEXITCODE -ne 0) { throw 'Falló la restauración.' }
dotnet build .\AceriaApp.sln -c Release --no-restore
if ($LASTEXITCODE -ne 0) { throw 'Falló la compilación.' }
```

En B, inicia Inventario:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__DiagnosticoActivo = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106 2>&1 |
    Tee-Object -FilePath .\inventario-m074.jsonl
```

En A, inicia Producción. **No vuelvas a arrancar este proceso hasta terminar la recuperación**: reiniciarlo eliminaría la evidencia del circuito compartido en memoria.

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Comunicacion__InventarioUrl = 'http://localhost:5106/'
$env:Observabilidad__DiagnosticoActivo = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Api -c Release --no-build --no-restore --no-launch-profile -- --urls
http://localhost:5105 2>&1 |
    Tee-Object -FilePath .\produccion-m074.jsonl
```

En C, comprueba el camino sano y conserva el identificador del proceso que escucha en `5105`:

```
$pidProduccionAntes = @(Get-NetTCPConnection -LocalPort 5105 -State Listen |
    Select-Object -ExpandProperty OwningProcess -Unique)[0]
Get-Process -Id $pidProduccionAntes | Select-Object Id, ProcessName, StartTime
$inventarioInicial = Invoke-RestMethod 'http://localhost:5106/api/v1/materiales/Acero'
$consultaInicial = Invoke-RestMethod 'http://localhost:5105/api/v1/comunicacion/materiales/Acero'
```

```
$inventarioInicial
$consultaInicial
if ($consultaInicial.material -ne $inventarioInicial.material -or $consultaInicial.stock -ne
$inventarioInicial.stock) {
    throw 'La lectura inicial no coincide con Inventario.'
}
```

Detén **solo Inventario en B con `Ctrl+C`**. Mantén A abierto. En C, envía cuatro consultas seguidas, sin pausas entre ellas, para acumular suficientes muestras dentro de la ventana del circuito:

```
$fallos = foreach ($numero in 1..4) {
    $correlacion = 'm074-caida-' + $numero + '-' + [guid]::NewGuid().ToString('N')
    $cronometro = [System.Diagnostics.Stopwatch]::StartNew()
    $r = Invoke-WebRequest 'http://localhost:5105/api/v1/comunicacion/materiales/Acero' `
        -Headers @{ 'X-Correlation-ID' = $correlacion } -SkipHttpErrorCheck -TimeoutSec 15
    $cronometro.Stop()
    if ($r.StatusCode -ne 503) { throw 'La caída debe devolverse como indisponibilidad, no como stock.' }
    $problema = $r.Content | ConvertFrom-Json
    if ($problema.PSObject.Properties.Name -contains 'stock') { throw 'Se devolvió stock ficticio durante la
caída.' }
    [pscustomobject]@{ Consulta = $numero; HTTP = $r.StatusCode; Milisegundos =
$cronometro.Elapsed.TotalMilliseconds; CorrelationId = $correlacion; Cuerpo = $r.Content }
}
$fallos | Format-List
$eventosCaida = @(
    Get-Content .\produccion-m074.jsonl |
        ForEach-Object { try { $_ | ConvertFrom-Json -ErrorAction Stop } catch { } } |
        Where-Object { $_.Properties.SourceContext -eq 'AceriaApp.Resiliencia' }
)
$eventosCaida | Select-Object Timestamp, Level,
    @{ n = 'Evento'; e = { $_.Properties.EventId.Id } },
    @{ n = 'CorrelationId'; e = { $_.Properties.CorrelationId } },
    @{ n = 'Intento'; e = { $_.Properties.Intento } },
    @{ n = 'DemoraMs'; e = { $_.Properties.DemoraMs } },
    @{ n = 'EstadoHttp'; e = { $_.Properties.EstadoHttp } },
    @{ n = 'EstadoCircuito'; e = { $_.Properties.Estado } } | Format-Table -AutoSize
if (-not ($eventosCaida | Where-Object { $_.Properties.EventId.Id -eq 7401 })) { throw 'No se observó el
retry.' }
if (-not ($eventosCaida | Where-Object { $_.Properties.EventId.Id -eq 7402 })) { throw 'No se observó la
apertura del circuito.' }
```

El evento `7401` registra cada reintento; `EstadoHttp` será nulo cuando el fallo sea de conexión, porque no hubo respuesta HTTP remota. El evento `7402` indica apertura. La primera consulta puede realizar varios envíos; las posteriores, mientras el circuito esté abierto, reciben `503` sin llegar al transporte. La menor duración de esas consultas es un indicio observable, no una prueba suficiente por sí sola del número de envíos; la suite comprueba esa ausencia de invocación con un transporte controlado.

La configuración permite **dos reintentos: tres intentos totales como máximo**, nunca tres reintentos adicionales. El circuito puede cortar antes. Solo GET/HEAD/OPTIONS se reintentan ante 408, 429, 5xx, fallos de transporte o timeout del intento; no se repiten POST/PUT/PATCH/DELETE, 404 ni cancelaciones del usuario. Cada intento tiene un segundo, y la política exterior limita a seis segundos la operación remota, incluidas esperas, `Retry-After` y lectura del cuerpo. La duración vista desde PowerShell incluye además planificación, middleware y transporte local: no la uses como un cronómetro exacto del límite interno.

Recuperación manteniendo Producción. Vuelve a ejecutar en B el comando completo de Inventario, sin cambiar A:

```
$env:DOTNET_ENVIRONMENT = 'Development'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:SkipDatabaseInitialization = 'false'
$env:Comunicacion__WorkersActivos = 'false'
$env:Observabilidad__DiagnosticoActivo = 'false'
$env:Observabilidad__NivelMinimo = 'Information'
dotnet run --project .\src\AceriaApp.Inventario.Api -c Release --no-build --no-restore --no-launch-profile --
--urls http://localhost:5106 2>&1 |
    Tee-Object -FilePath .\inventario-m074-recuperado.jsonl
```

En C, confirma que Inventario ha arrancado. Cuando esta petición devuelva `200`, deja transcurrir el intervalo de apertura de cinco segundos antes de probar Producción:

```
$readyInventario = Invoke-WebRequest 'http://localhost:5106/health/ready' -SkipHttpErrorCheck
$readyInventario.StatusCode
$readyInventario.Content
if ($readyInventario.StatusCode -ne 200) { throw 'Inventario todavía no está listo; no pruebes recuperación
hasta que lo esté.' }
Start-Sleep -Seconds 6
$correlacionRecuperada = 'm074-recuperada-' + [guid]::NewGuid().ToString('N')
$recuperada = Invoke-WebRequest 'http://localhost:5105/api/v1/comunicacion/materiales/Acero' `
    -Headers @{ 'X-Correlation-ID' = $correlacionRecuperada } -SkipHttpErrorCheck
$recuperada.StatusCode
$recuperada.Content
if ($recuperada.StatusCode -ne 200) { throw 'No se recuperó la consulta a Inventario.' }
$inventarioReal = Invoke-RestMethod 'http://localhost:5106/api/v1/materiales/Acero'
$materialRecuperado = $recuperada.Content | ConvertFrom-Json
if ($materialRecuperado.material -ne $inventarioReal.material -or $materialRecuperado.stock -ne
$inventarioReal.stock) {
    throw 'La recuperación no devuelve el stock real.'
}
$pidProduccionDespues = @(Get-NetTCPConnection -LocalPort 5105 -State Listen |
    Select-Object -ExpandProperty OwningProcess -Unique)[0]
if ($pidProduccionAntes -ne $pidProduccionDespues) { throw 'Producción fue reiniciado; repite el recorrido
manteniendo su proceso.' }
$cierre = @(
    Get-Content .\produccion-m074.jsonl |
        ForEach-Object { try { $_ | ConvertFrom-Json -ErrorAction Stop } catch { } } |
        Where-Object { $_.Properties.EventId.Id -eq 7403 -and $_.Properties.CorrelationId -eq
$correlacionRecuperada }
)
$cierre | Select-Object Timestamp, Level, MessageTemplate, Properties | Format-List
if ($cierre.Count -ne 1) { throw 'No se observó el cierre del circuito al recuperar la dependencia.' }
```

La primera petición admitida tras la apertura actúa como sonda `half-open`; su éxito cierra el circuito y produce `7403`. El nivel `Information` es necesario para ver ese cierre. La espera anterior pertenece a esta comprobación con procesos reales; las pruebas de políticas utilizan un reloj virtual y no esperan cinco segundos reales.

Por último, ejecuta las pruebas de resiliencia de la etapa:

```
dotnet test .\tests\AceriaApp.Resiliencia.Tests\AceriaApp.Resiliencia.Tests.csproj -c Release --no-build --
no-restore `
    --filter 'FullyQualifiedName~ResilienciaFuncionalTests'
```

Los 32 casos funcionales usan `IHttpClientFactory` y Polly reales, `FakeTimeProvider` y un `HttpMessageHandler` controlado. Comprueban 503→200, tres intentos como límite, respuestas liberadas, métodos mutantes sin repetir cuerpo, cancelación durante envío y `Retry-After`, timeout con cabeceras/cuerpo lento, presupuesto total, circuitos compartidos por cliente/destino, aislamiento entre autoridades, sonda half-open concurrente, recuperación y traducción a indisponibilidad. No presentan ese transporte controlado como una prueba de red ni de SQL. El recorrido manual sí realiza HTTP entre dos procesos y consulta SQL real. Los callbacks propios `7401/7402/7403` no registran excepciones libres, URL, cuerpos ni query; la telemetría genérica de Polly queda filtrada y sigue vigente la limitación sobre logs heredados de M6 indicada en 7.1.

Detén A y B con `Ctrl+C`. No se ha reservado ni modificado stock: todas las comprobaciones del fallo y recuperación usan la consulta GET de materiales.

PASO 7.4.18 — Ejecutar todos los tests del punto 7.4

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release --no-build --no-restore`.

2. Comprueba que se ejecutan los 18 tests de catálogo y las pruebas funcionales incorporadas progresivamente al proyecto.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 18 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 7.4.19 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 403 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04, M05 y M06: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 403 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

PASO 7.4.20 — Actualizar el README del punto 7.4

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `7.4-retry-circuit-breaker-polly`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `7.3-gestion-excepciones` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 7.4 — Retry y Circuit Breaker mediante Polly

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.

## Estado del proyecto

- Solución `AceriaApp.sln`: 34 proyectos, 22 proyectos de tests y 403 pruebas acumuladas.
- 18 pruebas nuevas de análisis en `tests/AceriaApp.Resiliencia.Tests`.
- Tipos `PoliticaResiliencia`, `ValidadorResiliencia` y `CatalogoPoliticas` en `src/AceriaApp.Api/
Resiliencia`.
```

```
- Los catálogos no configuran logging, probes, manejadores ni políticas HTTP.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M07 para iniciar los adaptadores y observar logs, salud, errores y recuperación HTTP.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/retry-circuit-breaker-polly.md` y el ADR-030. Los
bloques funcionales se aplican progresivamente y el paso 17 completa el registro y la compilación del punto.
Se conservan los documentos y pruebas heredados.

## Documentación

- [Tema del punto](docs/retry-circuit-breaker-polly.md).
- [ADR-030](docs/ADR-030-retry-circuit-breaker-polly.md).
- Los 40 documentos anteriores, incluidos los ADR-010 a ADR-029, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 34 proyectos, 403 pruebas y 18 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 7.4.21 — Crear el ADR-030 del punto 7.4

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-030-retry-circuit-breaker-polly.md` dentro de la copia de `7.4-retry-circuit-breaker-polly`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-030-retry-circuit-breaker-polly.md` existe y que los 40 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-030-retry-circuit-breaker-polly.md`. **Operación:** Crear.

```
# ADR-030: Retry y Circuit Breaker mediante Polly

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita observar y limitar fallos distribuidos. Se conservan los
contratos del dominio, SQL Server, RabbitMQ y outbox/inbox de M6. Los catálogos de análisis y la
implementación funcional aportan evidencias diferentes y complementarias.

## Decisión

Usar Polly v8 mediante Microsoft.Extensions.Http.Resilience solo en las consultas GET a Inventario. Aplicar
timeout total 6 s, dos reintentos de 200 ms exponenciales con jitter, circuito de mínimo cuatro muestras y
proporción 0,5 en diez segundos, apertura cinco segundos, y timeout de intento un segundo. No repetir métodos
mutantes ni estados permanentes ni cancelación del usuario. Conservar el circuito por cliente/destino y
traducir agotamiento a indisponibilidad.

## Alternativas consideradas

- Mantener solo los catálogos: descartado porque los indicadores no prueban el comportamiento del proceso.
- Duplicar los componentes en Producción e Inventario: descartado; la biblioteca AceriaApp.Observabilidad
comparte la integración ASP.NET sin acoplar el dominio.
```

```
- Introducir servicios de observabilidad externos: fuera del alcance del módulo; la consola JSON permite
inspeccionar resultados reales.

## Consecuencias

- Positivas: comportamiento uniforme, contratos explícitos y fallos reproducibles.
- Negativas: deben mantenerse límites, configuración, políticas y pruebas de integración.
- Mitigación: conservar pruebas heredadas, validar errores y cancelación, y separar señales operativas de los
datos sensibles.
```

Verificación concreta. `docs/ADR-030-retry-circuit-breaker-polly.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-030-retry-circuit-breaker-polly.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 7.4.22 — Verificar que no se ha modificado ningún archivo del punto 7.3

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.3-gestion-excepciones` y actual `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 40 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 40 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 7.4.23 — Verificar la estructura del punto 7.4 y el `.gitignore`

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, con todos los proyectos dentro de la etapa. Las reglas de la raíz ya excluyen los binarios y resultados de todas las etapas.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/Resiliencia` y las dos clases de tests en `tests/AceriaApp.Resiliencia.Tests`.
3. Comprueba los 42 documentos de `docs`, incluida la ficha del punto y el ADR-030.
4. Ejecuta `dotnet sln AceriaApp.sln list`, `git status --short` y `git check-ignore` sobre un archivo de `bin`, `obj` y `TestResults`; contrasta las rutas y conserva las exclusiones heredadas.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 34 proyectos. Git no propone binarios ni resultados temporales.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 7.4.24 — Hacer el commit del punto 7.4 y verificar el estado final del módulo 7

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 7.4: retry y circuit breaker mediante polly`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 7.4 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorResiliencia` y una prueba adicional. Trabaja en una copia optativa de `7.4-retry-circuit-breaker-polly` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 7.4.1 — Definir el contrato del resumen

Reemplaza `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs` por el archivo completo siguiente, que conserva las reglas y añade el contrato de resumen en el mismo namespace.

Archivo: `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.Resiliencia;

public class ValidadorResiliencia
{
    private static readonly string[] TiposPermitidos = { "Retry", "CircuitBreaker", "Timeout" };

    public ResultadoResiliencia Validar(IEnumerable<PoliticaResiliencia> politicas)
    {
        var errores = new List<string>();
        var lista = politicas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay políticas declaradas.");
        }
```

```
        foreach (var politica in lista)
        {
            if (string.IsNullOrWhiteSpace(politica.Nombre))
            {
                errores.Add("Una política no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(politica.Tipo))
            {
                errores.Add($"Tipo no permitido en '{politica.Nombre}': '{politica.Tipo}'.");
            }

            if (politica.Tipo == "Retry" && politica.NumeroDeReintentos < 1)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara reintentos.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.MinimoDeMuestras < 2)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara mínimo de
muestras.");
            }

            if (politica.Tipo == "Retry" && !politica.OperacionIdempotente)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara una operación
idempotente.");
            }

            if (politica.Tipo == "CircuitBreaker" &&
                (!double.IsFinite(politica.ProporcionDeFallos) || politica.ProporcionDeFallos <= 0 ||
politica.ProporcionDeFallos > 1))
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' tiene una proporción de
fallos fuera de (0, 1].");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.VentanaMuestreoSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una ventana de
muestreo positiva.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.DuracionAperturaSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una duración de
apertura positiva.");
            }

            if (!politica.TieneTimeout)
            {
                errores.Add($"La política '{politica.Nombre}' no tiene timeout.");
            }

            if (!politica.RegistraEventos)
            {
                errores.Add($"La política '{politica.Nombre}' no registra eventos.");
            }
        }

        return new ResultadoResiliencia(errores.Count == 0, errores);
    }
}

public record ResultadoResiliencia(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenResiliencia(int Total, int Retry, int CircuitBreaker, int Timeout, int TotalReintentos);
```

Reto 7.4.3 — Calcular los recuentos

Reemplaza de nuevo `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs` por el archivo completo siguiente. El método queda dentro del validador y materializa la secuencia una sola vez.

Archivo: `src/AceriaApp.Api/Resiliencia/ValidadorResiliencia.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;

namespace AceriaApp.Api.Resiliencia;

public class ValidadorResiliencia
{
    public static ResumenResiliencia Resumen(IEnumerable<PoliticaResiliencia> politicas)
    {
        var lista = politicas.ToList();
        var retry = lista.Count(p => p.Tipo == "Retry");
        var circuitBreaker = lista.Count(p => p.Tipo == "CircuitBreaker");
        var timeout = lista.Count(p => p.Tipo == "Timeout");
        var totalReintentos = lista.Sum(p => p.NumeroDeReintentos);
        return new ResumenResiliencia(lista.Count, retry, circuitBreaker, timeout, totalReintentos);
    }

    private static readonly string[] TiposPermitidos = { "Retry", "CircuitBreaker", "Timeout" };

    public ResultadoResiliencia Validar(IEnumerable<PoliticaResiliencia> politicas)
    {
        var errores = new List<string>();
        var lista = politicas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay políticas declaradas.");
        }

        foreach (var politica in lista)
        {
            if (string.IsNullOrWhiteSpace(politica.Nombre))
            {
                errores.Add("Una política no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(politica.Tipo))
            {
                errores.Add($"Tipo no permitido en '{politica.Nombre}': '{politica.Tipo}'.");
            }

            if (politica.Tipo == "Retry" && politica.NumeroDeReintentos < 1)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara reintentos.");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.MinimoDeMuestras < 2)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara mínimo de
muestras.");
            }

            if (politica.Tipo == "Retry" && !politica.OperacionIdempotente)
            {
                errores.Add($"La política de retry '{politica.Nombre}' no declara una operación
idempotente.");
            }

            if (politica.Tipo == "CircuitBreaker" &&
                (!double.IsFinite(politica.ProporcionDeFallos) || politica.ProporcionDeFallos <= 0 ||
politica.ProporcionDeFallos > 1))
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' tiene una proporción de
fallos fuera de (0, 1].");
            }

            if (politica.Tipo == "CircuitBreaker" && politica.VentanaMuestreoSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una ventana de
muestreo positiva.");
            }
```

```
            if (politica.Tipo == "CircuitBreaker" && politica.DuracionAperturaSegundos < 1)
            {
                errores.Add($"La política de circuit breaker '{politica.Nombre}' no declara una duración de
apertura positiva.");
            }

            if (!politica.TieneTimeout)
            {
                errores.Add($"La política '{politica.Nombre}' no tiene timeout.");
            }

            if (!politica.RegistraEventos)
            {
                errores.Add($"La política '{politica.Nombre}' no registra eventos.");
            }
        }

        return new ResultadoResiliencia(errores.Count == 0, errores);
    }
}

public record ResultadoResiliencia(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenResiliencia(int Total, int Retry, int CircuitBreaker, int Timeout, int TotalReintentos);
```

Cuenta las entradas que cumplen el criterio indicado. Suma los reintentos declarados por las políticas del catálogo; no cuenta intentos reales de HTTP. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 7.4.6 — Verificar el catálogo por defecto

Reemplaza `tests/AceriaApp.Resiliencia.Tests/ValidadorResilienciaTests.cs` por este archivo completo. Conserva todos los tests obligatorios y añade la comprobación de total 5, retry 2, circuit breaker 2, timeout 1 y suma de reintentos 8.

Archivo: `tests/AceriaApp.Resiliencia.Tests/ValidadorResilienciaTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Resiliencia;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Resiliencia.Tests;

public class ValidadorResilienciaTests
{
    private readonly ValidadorResiliencia _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValida()
    {
        var resultado = _validador.Validar(CatalogoPoliticas.PoliticasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinPoliticas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<PoliticaResiliencia>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay políticas"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
```

```
            new() { Nombre = "X", Tipo = "TipoInventado", TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_RetrySinReintentos_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Retry", NumeroDeReintentos = 0, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara reintentos"));
    }

    [Fact]
    public void Validar_CircuitBreakerSinMinimoDeMuestras_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 0, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara mínimo de muestras"));
    }

    [Fact]
    public void Validar_SinTimeout_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Timeout", TieneTimeout = false, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene timeout"));
    }

    [Fact]
    public void Validar_SinRegistro_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Timeout", TieneTimeout = true, RegistraEventos = false }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no registra eventos"));
    }
    [Fact]
    public void Validar_RetryNoIdempotente_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "Retry", NumeroDeReintentos = 2, OperacionIdempotente = false,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);
```

```
        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara una operación idempotente"));
    }
    [Fact]
    public void Validar_CircuitBreakerConUnaMuestra_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 1, TieneTimeout = true,
RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara mínimo de muestras"));
    }
    [Fact]
    public void Validar_CircuitBreakerConProporcionCero_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, ProporcionDeFallos = 0,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("proporción de fallos fuera"));
    }
    [Fact]
    public void Validar_CircuitBreakerConProporcionInvalida_DevuelveError()
    {
        foreach (var proporcion in new[] { 1.1, double.NaN, double.PositiveInfinity,
double.NegativeInfinity })
        {
            var politicas = new List<PoliticaResiliencia>
            {
                new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, ProporcionDeFallos =
proporcion, TieneTimeout = true, RegistraEventos = true }
            };

            var resultado = _validador.Validar(politicas);

            resultado.EsValida.Should().BeFalse();
            resultado.Errores.Should().Contain(e => e.Contains("proporción de fallos fuera"));
        }
    }
    [Fact]
    public void Validar_CircuitBreakerSinVentana_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, VentanaMuestreoSegundos = 0,
TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("ventana de muestreo positiva"));
    }
    [Fact]
    public void Validar_CircuitBreakerSinDuracionApertura_DevuelveError()
    {
        var politicas = new List<PoliticaResiliencia>
        {
            new() { Nombre = "X", Tipo = "CircuitBreaker", MinimoDeMuestras = 4, DuracionAperturaSegundos =
0, TieneTimeout = true, RegistraEventos = true }
        };

        var resultado = _validador.Validar(politicas);

        resultado.EsValida.Should().BeFalse();
```

```
        resultado.Errores.Should().Contain(e => e.Contains("duración de apertura positiva"));
    }
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorResiliencia.Resumen(CatalogoPoliticas.PoliticasPorDefecto());

        resumen.Total.Should().Be(5);
        resumen.Retry.Should().Be(2);
        resumen.CircuitBreaker.Should().Be(2);
        resumen.Timeout.Should().Be(1);
        resumen.TotalReintentos.Should().Be(8);
    }
}
```

Calcula el resumen sobre el catálogo por defecto.

Comprobación del reto. Guarda los archivos, compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.Resiliencia.Tests/AceriaApp.Resiliencia.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 18 a 19 tests; la solución debe ejecutar 403 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y compara los hashes de la etapa obligatoria: no debe haber cambiado. Revisa `git diff --check` y el índice antes de registrar el reto en un commit local separado.

Diagnóstico si falla. Si aparece un tipo duplicado, reemplaza el contenido completo en vez de pegarlo al final. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 18 pruebas de catálogo, la implementación funcional y 42 documentos Markdown. La solución incluye 34 proyectos, 22 proyectos de tests y 403 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Cierre del módulo. Relaciona un evento JSON, un estado de readiness, un ProblemDetails correlacionado y una recuperación de Inventario con sus pruebas ejecutables. Consulta el README y los ADR-027 a ADR-030 para operar y revisar las cuatro decisiones.